CS 61C Fall 2022

Number Representation

Discussion 1

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and if false, correct the statement to make it true:

1.1 Depending on the context, the same sequence of bits may represent different things.

1.2 It is possible to get an overflow error in Two's Complement when adding numbers of opposite signs.

1.3 If you interpret a N bit Two's complement number as an unsigned number, negative numbers would be smaller than positive numbers.

1.4 If you interpret an N bit Bias notation number as an unsigned number (assume there are negative numbers for the given bias), negative numbers would be smaller than positive numbers.

1.5 We can represent fractions and decimals in our given number representation formats (unsigned, biased, and Two's Complement).

2 Unsigned Integers

2.1 If we have an n-digit unsigned numeral $d_{n-1}d_{n-2}\ldots d_0$ in *radix* (or *base*) r, then the value of that numeral is $\sum_{i=0}^{n-1} r^i d_i$, which is just fancy notation to say that instead of a 10's or 100's place we have an r's or r^2's place. For the three radices binary, decimal, and hex, we just let r be 2, 10, and 16, respectively.

Let's try this by hand.

(a) Convert the following numbers from their initial radix into the other two common radices:

1. 0b10010011
2. 63
3. 0b00100100
4. 0
5. 39

6. 437
7. 0x0123

(b) Convert the following numbers from hex to binary:

1. 0xD3AD
2. 0xB33F
3. 0x7EC4

2.2 Our preferred tool for writing large numbers is the IEC prefixing system, which is similar to scientific notation but with powers of 2 rather than 10:

Ki (Kibi) = 2^{10}	Gi (Gibi) = 2^{30}	Pi (Pebi) = 2^{50}	Zi (Zebi) = 2^{70}
Mi (Mebi) = 2^{20}	Ti (Tebi) = 2^{40}	Ei (Exbi) = 2^{60}	Yi (Yobi) = 2^{80}

For example, we would write 2^{81} as $2 * 2^{80} = 2$ Yi.

(a) Write the following numbers using IEC prefixes:

- 2^{16}
- 2^{27}
- 2^{43}
- 2^{36}
- 2^{34}
- 2^{61}
- 2^{47}
- 2^{59}

(b) Write the following numbers as powers of 2:

- 2 Ki
- 512 Ki
- 16 Mi
- 256 Pi
- 64 Gi
- 128 Ei

3 Signed Integers

3.1 Unsigned binary numbers work for natural numbers, but many calculations use negative numbers as well. To deal with this, a number of different schemes have been used to represent signed numbers. Here are two common schemes:

Two's Complement:

- We can write the value of an n-digit two's complement number as $\sum_{i=0}^{n-2} 2^i d_i - 2^{n-1} d_{n-1}$.
- Negative numbers will have a 1 as their most significant bit (MSB). Plugging in $d_{n-1} = 1$ to the formula above gets us $\sum_{i=0}^{n-2} 2^i d_i - 2^{n-1}$.
- Meanwhile, positive numbers will have a 0 as their MSB. Plugging in $d_{n-1} = 0$ gets us $\sum_{i=0}^{n-2} 2^i d_i$, which is very similar to unsigned numbers.
- To negate a two's complement number: flip all the bits and add 1.
- Addition is exactly the same as with an unsigned number.
- Only one 0, and it's located at 0b0.

Biased Representation:

- The number line is shifted so that the smallest number we want to be representable would be 0b0...0.
- To find out what the represented number is, read the representation as if it was an unsigned number, then add the bias.
- We can shift to any arbitrary bias we want to suit our needs. To represent (nearly) as much negative numbers as positive, a commonly-used bias for N bits is $-(2^{N-1}-1)$.

For questions (a) through (c), assume an 8-bit integer and answer each one for the case of an unsigned number, biased number with a bias of -127, and two's complement number. Indicate if it cannot be answered with a specific representation.

(a) What is the largest integer? What is the result of adding one to that number?

1. Unsigned?
2. Biased?
3. Two's Complement?

(b) How would you represent the numbers 0, 1, and -1?

1. Unsigned?
2. Biased?
3. Two's Complement?

(c) How would you represent 17 and -17?

1. Unsigned?
2. Biased?
3. Two's Complement?

3.2 Prove that the two's complement inversion trick is valid (i.e. that x and $\overline{x}+1$ sum to 0).

3.3 We now have three major radices (or bases) that allow us to represent numbers using a finite amount of symbols: binary, decimal, hexadecimal. Why do we use each of these radices, and why are each of them preferred over other bases in a given context?

4 Arithmetic and Counting

4.1 Addition and subtraction of binary/hex numbers can be done in a similar fashion as with decimal digits by working right to left and carrying over extra digits to the next place. However, sometimes this may result in an overflow if the number of bits can no longer represent the true sum. Overflow occurs if and only if two numbers with the same sign are added and the result has the opposite sign.

(a) Compute the decimal result of the following arithmetic expressions involving 6-bit Two's Complement numbers as they would be calculated on a computer. Do any of these result in an overflow? Are all these operations possible?

1. 0b011001 − 0b000111
2. 0b100011 + 0b111010
3. 0x3B + 0x06
4. 0xFF − 0xAA
5. 0b000100 − 0b001000

(b) What is the least number of bits needed to represent the following ranges using any number representation scheme?

1. 0 to 256
2. -7 to 56
3. 64 to 127 and -64 to -127
4. Address every byte of a 12 TiB chunk of memory

(c) How many distinct numbers can the following schemes represent? How many distinct *positive* numbers?

1. 10-bit unsigned
2. 8-bit Two's Complement
3. 8-bit One's Complement
4. 6-bit biased, with a bias of -30
5. 10-bit sign-magnitude

CS 61C Fall 2022

Number Representation

Discussion 1

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and if false, correct the statement to make it true:

1.1 Depending on the context, the same sequence of bits may represent different things.

True. The same bits can be interpreted in many different ways with the exact same bits! The bits can represent anything from an unsigned number to a signed number or even, as we will cover later, a program. It is all dependent on its agreed upon interpretation.

1.2 It is possible to get an overflow error in Two's Complement when adding numbers of opposite signs.

False. Overflow errors only occur when the correct result of the addition falls outside the range of $[-(2^{n-1}), 2^{n-1}-1]$. Adding numbers of opposite signs will not result in numbers outside of this range.

1.3 If you interpret a N bit Two's complement number as an unsigned number, negative numbers would be smaller than positive numbers.

False. In Two's Complement, the MSB is always 1 for a negative number. This means EVERY negative number in Two's Complement, when converted to unsigned, will be larger than the positive numbers.

1.4 If you interpret an N bit Bias notation number as an unsigned number (assume there are negative numbers for the given bias), negative numbers would be smaller than positive numbers.

True. In bias notation, we add a bias to the unsigned interpretation to create the value. Regardless of where we 'shift' the range of representable values, the negative numbers, when converted to unsigned, will always stay smaller than the positive numbers. This is unlike Two's Complement (see description above).

1.5 We can represent fractions and decimals in our given number representation formats (unsigned, biased, and Two's Complement).

False. Our current representation formats has a major limitation; we can only represent and do arithmetic with integers. To successfully represent fractional values

as well as numbers with extremely high magnitude beyond our current boundaries, we need another representation format.

2 Unsigned Integers

2.1 If we have an n-digit unsigned numeral $d_{n-1}d_{n-2}\ldots d_0$ in *radix* (or *base*) r, then the value of that numeral is $\sum_{i=0}^{n-1} r^i d_i$, which is just fancy notation to say that instead of a 10's or 100's place we have an r's or r^2's place. For the three radices binary, decimal, and hex, we just let r be 2, 10, and 16, respectively.

Let's try this by hand.

(a) Convert the following numbers from their initial radix into the other two common radices:

1. 0b10010011 = 147 = 0x93
2. 63 = 0b0011 1111 = 0x3F
3. 0b00100100 = 36 = 0x24
4. 0 = 0b0 = 0x0
5. 39 = 0b0010 0111 = 0x27
6. 437 = 0b0001 1011 0101 = 0x1B5
7. 0x0123 = 0b0000 0001 0010 0011 = 291

(b) Convert the following numbers from hex to binary:

1. 0xD3AD = 0b1101 0011 1010 1101 = 54189
2. 0xB33F = 0b1011 0011 0011 1111 = 45887
3. 0x7EC4 = 0b0111 1110 1100 0100 = 32452

2.2 Our preferred tool for writing large numbers is the IEC prefixing system, which is similar to scientific notation but with powers of 2 rather than 10:

Ki (Kibi) = 2^{10}	Gi (Gibi) = 2^{30}	Pi (Pebi) = 2^{50}	Zi (Zebi) = 2^{70}
Mi (Mebi) = 2^{20}	Ti (Tebi) = 2^{40}	Ei (Exbi) = 2^{60}	Yi (Yobi) = 2^{80}

For example, we would write 2^{81} as $2 * 2^{80} = 2$ Yi.

(a) Write the following numbers using IEC prefixes:

- 2^{16} = 64 Ki
- 2^{27} = 128 Mi
- 2^{43} = 8 Ti
- 2^{36} = 64 Gi
- 2^{34} = 16 Gi
- 2^{61} = 2 Ei
- 2^{47} = 128 Ti
- 2^{59} = 512 Pi

(b) Write the following numbers as powers of 2:

- 2 Ki = 2^{11}
- 512 Ki = 2^{19}
- 16 Mi = 2^{24}
- 256 Pi = 2^{58}
- 64 Gi = 2^{36}
- 128 Ei = 2^{67}

3 Signed Integers

3.1 Unsigned binary numbers work for natural numbers, but many calculations use negative numbers as well. To deal with this, a number of different schemes have been used to represent signed numbers. Here are two common schemes:

Two's Complement:

- We can write the value of an n-digit two's complement number as $\sum_{i=0}^{n-2} 2^i d_i - 2^{n-1} d_{n-1}$.
- Negative numbers will have a 1 as their most significant bit (MSB). Plugging in $d_{n-1} = 1$ to the formula above gets us $\sum_{i=0}^{n-2} 2^i d_i - 2^{n-1}$.
- Meanwhile, positive numbers will have a 0 as their MSB. Plugging in $d_{n-1} = 0$ gets us $\sum_{i=0}^{n-2} 2^i d_i$, which is very similar to unsigned numbers.
- To negate a two's complement number: flip all the bits and add 1.
- Addition is exactly the same as with an unsigned number.
- Only one 0, and it's located at 0b0.

Biased Representation:

- The number line is shifted so that the smallest number we want to be representable would be 0b0...0.
- To find out what the represented number is, read the representation as if it was an unsigned number, then add the bias.
- We can shift to any arbitrary bias we want to suit our needs. To represent (nearly) as much negative numbers as positive, a commonly-used bias for N bits is $-(2^{N-1} - 1)$.

For questions (a) through (c), assume an 8-bit integer and answer each one for the case of an unsigned number, biased number with a bias of -127, and two's complement number. Indicate if it cannot be answered with a specific representation.

(a) What is the largest integer? What is the result of adding one to that number?

1. Unsigned? 255, 0
2. Biased? 128, -127
3. Two's Complement? 127, -128

(b) How would you represent the numbers 0, 1, and -1?

1. Unsigned? 0b0000 0000, 0b0000 0001, not possible
2. Biased? 0b0111 1111, 0b1000 0000, 0b0111 1110
3. Two's Complement? 0b0000 0000, 0b0000 0001, 0b1111 1111

(c) How would you represent 17 and -17?

1. Unsigned? 0b0001 0001, not possible
2. Biased? 0b1001 0000, 0b0110 1110

3. Two's Complement? 0b0001 0001, 0b1110 1111

3.2 Prove that the two's complement inversion trick is valid (i.e. that x and $\overline{x}+1$ sum to 0).

Note that for any x we have $x+\overline{x}=0\text{b}1\ldots1$. Adding 0b1 to $0\text{b}1\ldots1$ will cause the value to overflow, meaning that $0\text{b}1\ldots1+0\text{b}1=0\text{b}0=0$. Therefore, $x+\overline{x}+1=0$

A straightforward hand calculation shows that $0\text{b}1\ldots1+0\text{b}1=0$.

3.3 We now have three major radices (or bases) that allow us to represent numbers using a finite amount of symbols: binary, decimal, hexadecimal. Why do we use each of these radices, and why are each of them preferred over other bases in a given context?

Decimal is the preferred radix for human hand calculations, likely related to the fact that humans have 10 fingers.

Binary numerals are particularly useful for computers. Binary signals are less likely to be garbled than higher radix signals, as there is more "distance" (voltage or current) between valid signals (HIGH and LOW). Additionally, binary signals are quite convenient to design circuits, as we'll see later in the course.

Hexadecimal numbers are a convenient shorthand for displaying binary numbers, owing to the fact that one hex digit corresponds exactly to four binary digits.

4 Arithmetic and Counting

4.1 Addition and subtraction of binary/hex numbers can be done in a similar fashion as with decimal digits by working right to left and carrying over extra digits to the next place. However, sometimes this may result in an overflow if the number of bits can no longer represent the true sum. Overflow occurs if and only if two numbers with the same sign are added and the result has the opposite sign.

(a) Compute the decimal result of the following arithmetic expressions involving 6-bit Two's Complement numbers as they would be calculated on a computer. Do any of these result in an overflow? Are all these operations possible?

1. 0b011001 − 0b000111

   0b010010 = 18, No overflow.

2. 0b100011 + 0b111010

   Adding together we get 0b1011101, however since we are working with 6-bit numbers we truncate the first digit to get 0b011101 = 29. Since we added two negative numbers and ended up with a positive number, this results in an overflow.

3. 0x3B + 0x06

   Converting to binary, we get 0b111011 + 0b000110 = (after truncating as the problem states we're working with 6-bit numbers) 0b000001 = 1.

Despite the extra truncated bit, this is not an overflow as -5 + 6 indeed equals 1!

4. 0xFF − 0xAA

   Trick question! This is not possible, as these hex numbers would need 8 bits to represent and we are working with 6 bit numbers.

5. 0b000100 − 0b001000

   The 2's complement of 0b001000 is 0b110111 + 1 = 0b111000. We add that to 0b000100 to get 0b111100.

   We can logically fact check this by converting everything to decimals: 0b000100 is 4 and 0b001000 is 8, so the subtraction should result in -4, which is 0b111100.

(b) What is the least number of bits needed to represent the following ranges using any number representation scheme?

1. 0 to 256

   In general n bits can be used to represent at most 2^n distinct things. As such 8 bits can represent $2^8 = 256$ numbers. However, this range actually contains 257 numbers so we need 9 bits.

2. -7 to 56

   Range of 64 numbers which can be represented through 6 bits as $2^6 = 64$

3. 64 to 127 and -64 to -127

   We are representing 128 numbers in total which requires 7 bits.

4. Address every byte of a 12 TiB chunk of memory

   Since a TiB is 2^{40} and the factor of 12 needs 4 bits, in total we can represent using 44 bits as 2^{43} bytes $<$ 12 TiB $< 2^{44}$ bytes

(c) How many distinct numbers can the following schemes represent? How many distinct *positive* numbers?

1. 10-bit unsigned 1024, 1023

   In unsigned representation, different bit-strings correspond to different numbers, so 10 bits can represent $2^{10} = 1024$ distinct numbers. Out of all of these, only the number 0 is non-positive, so we can represent 1023 distinct positive numbers.

2. 8-bit Two's Complement 256, 127

   Like unsigned, different bit-strings correspond to distinct numbers in Two's Complement, so 8 bits can represent $2^8 = 256$ numbers. Out of these, half of them have a MSB of 1, which are negative numbers, and one is the number zero, so we can represent $256/2 - 1 = 127$ distinct positive numbers.

3. 8-bit One's Complement 255, 127

   Recall that One's Complement represents negative numbers by flipping the bits of its positive representation (which must have leading zeros). This produces unique values except for one case, with 0b0...0 and 0b1...1 corresponding to 0 and -0. Thus, One's Complement can only represent $2^8 - 1 = 255$ distinct numbers. As all positive numbers must have a leading zero, our largest positive value would be 0b01...1, so we would have $2^7 - 1$ distinct positive numbers.

4. 6-bit biased, with a bias of -30 64, 33

   Also like unsigned, in biased notation, no two different bit-strings correspond to the same number, so 6 bits can represent $2^6 = 64$ numbers. With this bias, the largest number we can represent is 0b111111$= 63 - 30 = 33$, and the smallest is -30, so there are 33 distinct positive numbers ($1 \sim 33$).

5. 10-bit sign-magnitude 1023, 511

   Unlike all three previous representation schemes, two different bit-strings (0b0000000000 and 0b1000000000) actually correspond to the same number zero, so we can represent only $2^{10} - 1 = 1023$ distinct numbers. Out of these, every bit-string with a MSB of 0, except 0b0000000000, correspond to a different positive number, so we can represent $2^9 - 1 = 511$ distinct positive numbers.

CS 61C Fall 2022

C and Pointers

Discussion 2

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and if false, correct the statement to make it true:

1.1 True or False: C is a pass-by-value language.

1.2 The following is correct C syntax:

```
int num = 43
```

1.3 In compiled languages, the compile time is generally pretty fast, however the run-time is significantly slower than interpreted languages.

1.4 The correct way of declaring a character array is `char[]` array.

1.5 Bitwise and logical operations result in the same behaviour for given bitstrings.

2 Bit-wise Operations

2.1 In C, we have a few bit-wise operators at our disposal:

- `AND` (`&`)
- `NOT` ($\sim$)
- `OR` (`|`)
- `XOR` ($\wedge$)
- `SHIFT LEFT` (`<<`)
  - Example: `0b0001 << 2 = 0b0100`
- `SHIFT RIGHT` (`>>`)
  - Example: `0b0100 >> 2 = 0b0001`

a	**b**	**a & b**	**a | b**	**a ∧ b**	**∼a**
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

For your convenience, truth tables for the logical operators are provided above. With the binary numbers `a`, `b`, and `c` below, perform the following bit-wise operations:

```
a = 0b1000 1011
b = 0b0011 0101
c = 0b1111 0000
```

(a) `a & b`

(b) `a` ∧ `c`

(c) `a | 0`

(d) `a | (b >> 5)`

(e) ∼ ((`b | c`) `& a`)

3 Pass-by-who?

3.1 Implement the following functions so that they work as described.

(a) Swap the value of two **`int`**s. *Remain swapped after returning from this function.*
Hint: Our answer is around three lines long.

```
void swap(________________, ________________) {
```

(b) Return the number of bytes in a string. *Do not use* `strlen`.
Hint: Our answer is around 4 lines long.

```
int mystrlen(________________) {
```

4 Debugging

4.1 The following functions may contain logic or syntax errors. Find and correct them.

(a) Returns the sum of all the elements in `summands`.

```
1  int sum(int *summands) {
2      int sum = 0;
3      for (int i = 0; i < sizeof(summands); i++)
4          sum += *(summands + i);
5      return sum;
6  }
```

(b) Increments all of the letters in the `string` which is stored at the front of an array of arbitrary length, `n >= strlen(string)`. Does not modify any other parts of the array's memory.

```
1  void increment(char *string, int n) {
2      for (int i = 0; i < n; i++)
3          *(string + i)++;
4  }
```

(c) Copies the string `src` to `dst`.

```
1  void copy(char *src, char *dst) {
2      while (*dst++ = *src++);
3  }
```

(d) Overwrites an input string `src` with "61C is awesome!" if there's room. Does nothing if there is not. Assume that `length` correctly represents the length of `src`.

```
1  void cs61c(char *src, size_t length) {
2      char *srcptr, replaceptr;
3      char replacement[16] = "61C is awesome!";
4      srcptr = src;
5      replaceptr = replacement;
6      if (length >= 16) {
7          for (int i = 0; i < 16; i++)
8              *srcptr++ = *replaceptr++;
9      }
10 }
```

CS 61C Fall 2022

C and Pointers

Discussion 2

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and if false, correct the statement to make it true:

1.1 True or False: C is a pass-by-value language.

True. If you want to pass a reference to anything, you should use a pointer.

1.2 The following is correct C syntax:

```
int num = 43
```

False. Semicolon!!

```
int num = 43;
```

1.3 In compiled languages, the compile time is generally pretty fast, however the run-time is significantly slower than interpreted languages.

False. Reasonable compilation time, excellent run-time performance. It optimizes for a given processor type and operating system.

1.4 The correct way of declaring a character array is `char[]` array.

False. The correct way is `char array[]`.

1.5 Bitwise and logical operations result in the same behaviour for given bitstrings.

False. Bitwise and logical operations fundamentally speaking, perform the same operations, just in different contexts. Bitwise operations compare and operate on inputs bit-by-bit, from least to most significant bit in the bitstring. Logical operations compare and operate on inputs as a whole, where anything not 0 can be considered to be a 1.

Note that in 61C and both bitwise and logical operations, 0 can be considered as False and not-0 can be considered as True in comparisons!

2 Bit-wise Operations

2.1 In C, we have a few bit-wise operators at our disposal:

- `AND` (`&`)
- `NOT` ($\sim$)
- `OR` (`|`)
- `XOR` ($\wedge$)
- `SHIFT LEFT` (`<<`)
  - Example: `0b0001 << 2 = 0b0100`
- `SHIFT RIGHT` (`>>`)
  - Example: `0b0100 >> 2 = 0b0001`

a	**b**	**a & b**	**a | b**	**a $\wedge$ b**	**$\sim$a**
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

For your convenience, truth tables for the logical operators are provided above. With the binary numbers `a`, `b`, and `c` below, perform the following bit-wise operations:

```
a = 0b1000 1011
b = 0b0011 0101
c = 0b1111 0000
```

(a) `a & b`

`0b0000 0001`

(b) `a` $\wedge$ `c`

`0b0111 1011`

(c) `a | 0`

`0b1000 1011`
Anything `| 0` always evaluates to the original value, so this just returns the value of `a`.

(d) `a | (b >> 5)`

`0b1000 1011`
`b >> 5` evaluates to 0b0000 0001, so `a | 1` still returns the value of `a`.

(e) $\sim$ ((`b | c`) `& a`)

`0b0111 1110`

3 Pass-by-who?

3.1 Implement the following functions so that they work as described.

(a) Swap the value of two **int**s. *Remain swapped after returning from this function.*
Hint: Our answer is around three lines long.

```
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}
```

(b) Return the number of bytes in a string. *Do not use* `strlen`.
Hint: Our answer is around 4 lines long.

```
int mystrlen(char* str) {
    int count = 0;
    while (*str++) {
        count++;
    }
    return count;
}
```

4 Debugging

4.1 The following functions may contain logic or syntax errors. Find and correct them.

(a) Returns the sum of all the elements in `summands`.

It is necessary to pass a size alongside the pointer.

```
int sum(int* summands, size_t n) {
    int sum = 0;
    for (int i = 0; i < n; i++)
        sum += *(summands + i);
    return sum;
}
```

(b) Increments all of the letters in the `string` which is stored at the front of an array of arbitrary length, `n >= strlen(string)`. Does not modify any other parts of the array's memory.

The ends of strings are denoted by the null terminator rather than n. Simply having space for n characters in the array does not mean the string stored inside is also of length n.

```
void increment(char* string) {
    for (i = 0; string[i] != 0; i++)
        string[i]++; // or (*(string + i))++;
}
```

Another common bug to watch out for is the corner case that occurs when incrementing the character with the value 0xFF. Adding 1 to 0xFF will overflow back to 0, producing a null terminator and unintentionally shortening the string.

(c) Copies the string `src` to `dst`.

```
1  void copy(char *src, char *dst) {
2      while (*dst++ = *src++);
3  }
```

No errors.

(d) Overwrites an input string `src` with "61C is awesome!" if there's room. Does nothing if there is not. Assume that `length` correctly represents the length of `src`.

```
1  void cs61c(char *src, size_t length) {
2      char *srcptr, replaceptr;
3      char replacement[16] = "61C is awesome!";
4      srcptr = src;
5      replaceptr = replacement;
6      if (length >= 16) {
7          for (int i = 0; i < 16; i++)
8              *srcptr++ = *replaceptr++;
9      }
10 }
```

`char *srcptr, replaceptr` initializes a `char` pointer, and a `char`—not two `char` pointers.

The correct initialization should be, `char *srcptr, *replaceptr`.

CS 61C Fall 2022

Memory and Floating Point

Discussion 3

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 Memory sectors are defined by the hardware, and cannot be altered.

1.2 For large recursive functions, you should store your data on the heap over the stack.

1.3 True or False. The goals of floating point are to have a large range of values, a low amount of precision, and real arithmetic results

1.4 True or False. The distance between floating point numbers increases as the absolute value of the numbers increase.

1.5 True or False. Floating Point addition is associative.

2 Memory Management

C does not automatically handle memory for you. In each program, an address space is set aside, separated in 2 dynamically changing regions and 2 'static' regions.

- The Stack: local variables inside of functions, where data is garbage immediately after the *function in which it was defined* returns. Each function call creates a stack frame with its own arguments and local variables. The stack dynamically changes, growing downwards as multiple functions are called within each other (LIFO structure), and collapsing upwards as functions finish execution and return.
- The Heap: memory manually allocated by the programmer with `malloc`, `calloc`, or `realloc`. Used for data we want to persist beyond function calls, growing upwards to 'meet' the stack. Careful heap management is necessary to avoid Heisenbugs! Memory is freed only when the programmer explicitly frees it!
- Static data: global variables declared outside of functions, does not grow or shrink through function execution.

- Code (or Text): loaded at the start of the program and does not change after, contains executable instructions and any pre-processor macros.

There are a number of functions in C that can be used to dynamically allocate memory on the heap. The following are the ones we use in this class:

- `malloc(size_t size)` allocates a block of `size` bytes and returns the start of the block. The time it takes to search for a block is generally not dependent on `size`.
- `calloc(size_t count, size_t size)` allocates a block of `count * size` bytes, **sets every value in the block to zero**, then returns the start of the block.
- `realloc(void *ptr, size_t size)` "resizes" a previously-allocated block of memory to `size` bytes, returning the start of the resized block.
- `free(void *ptr)` deallocates a block of memory which starts at `ptr` that was previously allocated by the three previous functions.

2.1 For each part, choose one or more of the following memory segments where the data could be located: **code, static, heap, stack**.

(a) Static variables

(b) Local variables

(c) Global variables

(d) Constants

(e) Machine Instructions

(f) Result of Dynamic Memory Allocation(`malloc` or `calloc`)

(g) String Literals

2.2 Write the code necessary to allocate memory on the heap in the following scenarios

(a) An array `arr` of k integers

(b) A string `str` containing p characters

(c) An $n \times m$ matrix `mat` of integers initialized to zero.

2.3 Compare the following two implementations of a function which duplicates a string. Is either one correct? Which one runs faster?

```
1  char* strdup1(char* s) {
2      int n = strlen(s);
3      char* new_str = malloc((n + 1) * sizeof(char));
4      for (int i = 0; i < n; i++) new_str[i] = s[i];
5      return new_str;
6  }
```

```
7  char* strdup2(char* s) {
8      int n = strlen(s);
9      char* new_str = calloc(n + 1, sizeof(char));
10     for (int i = 0; i < n; i++) new_str[i] = s[i];
11     return new_str;
12 }
```

2.4 What's the main issue with the code snippet seen here? (Hint: `gets()` is a function that reads in user input and stores it in the array given in the argument.)

```
1  char* foo() {
2      char buffer[64];
3      gets(buffer);
4
5      char* important_stuff = (char*) malloc(11 * sizeof(char));
6
7      int i;
8      for (i = 0; i < 10; i++) important_stuff[i] = buffer[i];
9      important_stuff[i] = '\0';
10     return important_stuff;
11 }
```

3 Linked List

Suppose we've defined a linked list **struct** as follows. Assume `*lst` points to the first element of the list, or is `NULL` if the list is empty.

```
struct ll_node {
    int first;
    struct ll_node* rest;
}
```

3.1 Implement `prepend`, which adds one new `value` to the front of the linked list. Hint: why use ll_node $**lst$ instead of ll_node$*lst$?

```
void prepend(struct ll_node** lst, int value)
```

3.2 Implement `free_ll`, which frees all the memory consumed by the linked list.

```
void free_ll(struct ll_node** lst)
```

4 Floating Point

The IEEE 754 standard defines a binary representation for floating point values using three fields.

- The *sign* determines the sign of the number (0 for positive, 1 for negative).
- The *exponent* is in **biased notation**. For instance, the bias is -127 which comes from -$(2^{8-1}-1)$ for single-precision floating point numbers.
- The *significand* or *mantissa* is akin to unsigned integers, but used to store a fraction instead of an integer.

The below table shows the bit breakdown for the single precision (32-bit) representation. The leftmost bit is the MSB and the rightmost bit is the LSB.

1	8	23
Sign	Exponent	Mantissa/Significand/Fraction

For normalized floats:
Value = $(-1)^{Sign} * 2^{Exp+Bias} * 1.\textbf{significand}_2$
For denormalized floats:
Value = $(-1)^{Sign} * 2^{Exp+Bias+1} * 0.\textbf{significand}_2$

Exponent	**Significand**	**Meaning**
0	Anything	Denorm
1-254	Anything	Normal
255	0	Infinity
255	Nonzero	NaN

Note that in the above table, our exponent has values from 0 to 255. When translating between binary and decimal floating point values, we must remember that there is a bias for the exponent.

4.1 Convert the following single-precision floating point numbers from binary to decimal or from decimal to binary. You may leave your answer as an expression.

- 0x00000000
- 8.25
- 0x00000F00
- 39.5625
- 0xFF94BEEF
- -∞
- 1/3

5 More Floating Point Representation

As we saw above, not every number can be represented perfectly using floating point. For this question, we will only look at positive numbers.

5.1 What is the next smallest number larger than 2 that can be represented completely?

5.2 What is the next smallest number larger than 4 that can be represented completely?

5.3 What is the largest odd number that we can represent? Hint: Try applying the step size technique covered in lecture.

CS 61C Fall 2022

Memory and Floating Point

Discussion 3

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 Memory sectors are defined by the hardware, and cannot be altered.

False. The four major memory sectors, stack, heap, static/data, and text/code for any given process (application) are defined by the operating system and may differ depending on what kind of memory is needed for it to run.

What's an example of a process that might need significant stack space, but very little text, static, and heap space? (Almost any basic deep recursive scheme, since you're making many new function calls on top of each other without closing the previous ones, and thus, stack frames.)

What's an example of a text and static heavy process? (Perhaps a process that is incredibly complicated but has efficient stack usage and does not dynamically allocate memory.)

What's an example of a heap-heavy process? (Maybe if you're using a lot of dynamic memory that the user attempts to access.)

1.2 For large recursive functions, you should store your data on the heap over the stack.

False. Generally speaking, if you need to keep access to data over several separate function calls, use the heap. However, recursive functions call themselves, creating multiple stack frames and using each of their return values. If you store data on the heap in a recursive scheme, your `malloc` calls may lead to you rapidly running out of memory, or can lead to memory leaks as you lose where you allocate memory as each stack frame collapses.

1.3 True or False. The goals of floating point are to have a large range of values, a low amount of precision, and real arithmetic results

False. Although floating point DOES
- Provide support for a wide range of values. (Both very small and very large)
- Help programmers deal with errors in real arithmetic because floating point can represent + ∞, -∞, NaN (Not a number)

Floating point actually has HIGH precision. Recall that precision is a count of the number of bits in a computer word used to represent a value. Floating point helps

you keep as much precision as possible because we have so much freedom to interpret our bits as whatever negative powers of 2 are useful for specifying the number.

1.4 True or False. The distance between floating point numbers increases as the absolute value of the numbers increase.

True. The uneven spacing is due to the exponent representation of floating point numbers. There are a fixed number of bits in the significand. In IEEE 32 bit storage there are 23 bits for the significand, which means the LSB is 2^{-22} times the MSB. If the exponent is zero (after allowing for the offset) the difference between two neighboring floats will be 2^{-22}. If the exponent is 8, the difference between two neighboring floats will be 2^{-14} because the mantissa is multiplied by 2^8. Limited precision makes binary floating-point numbers discontinuous; there are gaps between them.

1.5 True or False. Floating Point addition is associative.

False. Because of rounding errors, you can find Big and Small numbers such that:
(Small + Big) + Big != Small + (Big + Big)
FP approximates results because it only has 23 bits for Significand

2 Memory Management

C does not automatically handle memory for you. In each program, an address space is set aside, separated in 2 dynamically changing regions and 2 'static' regions.

- The Stack: local variables inside of functions, where data is garbage immediately after the *function in which it was defined* returns. Each function call creates a stack frame with its own arguments and local variables. The stack dynamically changes, growing downwards as multiple functions are called within each other (LIFO structure), and collapsing upwards as functions finish execution and return.
- The Heap: memory manually allocated by the programmer with `malloc`, `calloc`, or `realloc`. Used for data we want to persist beyond function calls, growing upwards to 'meet' the stack. Careful heap management is necessary to avoid Heisenbugs! Memory is freed only when the programmer explicitly frees it!
- Static data: global variables declared outside of functions, does not grow or shrink through function execution.
- Code (or Text): loaded at the start of the program and does not change after, contains executable instructions and any pre-processor macros.

There are a number of functions in C that can be used to dynamically allocate memory on the heap. The following are the ones we use in this class:

- `malloc(size_t size)` allocates a block of `size` bytes and returns the start of the block. The time it takes to search for a block is generally not dependent on `size`.
- `calloc(size_t count, size_t size)` allocates a block of `count * size` bytes, **sets every value in the block to zero**, then returns the start of the block.
- `realloc(void *ptr, size_t size)` "resizes" a previously-allocated block of memory to `size` bytes, returning the start of the resized block.
- `free(void *ptr)` deallocates a block of memory which starts at `ptr` that was previously allocated by the three previous functions.

2.1 For each part, choose one or more of the following memory segments where the data could be located: **code, static, heap, stack**.

(a) Static variables

Static

(b) Local variables

Stack

(c) Global variables

Static

(d) Constants

Code, static, or stack

Constants can be compiled directly into the code. `x = x + 1` can compile with the number 1 stored directly in the machine instruction in the code. That instruction will always increment the value of the variable `x` by 1, so it can be stored directly in the machine instruction without reference to other memory. This can also occur with pre-processor macros.

```
1  #define y 5
2
3  int plus_y(int x) {
4      x = x + y;
5      return x;
6  }
```

Constants can also be found in the stack or static storage depending on if it's declared in a function or not.

```
1  const int x = 1;
2
3  int sum(int* arr) {
4      int total = 0;
5      ...
```

```
6 }
```

In this example, `x` is a variable whose value will be stored in the static storage, while `total` is a local variable whose value will be stored on the stack. Variables declared **`const`** are not allowed to change, but the usage of **`const`** can get more tricky when combined with pointers.

(e) Machine Instructions

Code

(f) Result of Dynamic Memory Allocation(`malloc` or `calloc`)

Heap

(g) String Literals

Static.

When declared in a function, string literals can only be stored in static memory. String literals are declared when a character pointer is assigned to a string declared within quotation marks, i.e. **`char`**`* s = "string"`. You'll often see a near identical alternative to declaring a string: **`char`** `s[7] = "string"`. This string array will be stored in the stack (when declared inside a function) and is mutable, though they cannot change in size. Note that the compiler will arrange for the char array to be initialised from the literal and be mutable.

2.2 Write the code necessary to allocate memory on the heap in the following scenarios

(a) An array `arr` of k integers

```
arr = (int *) malloc(sizeof(int) * k);
```

(b) A string `str` containing p characters

```
str = (char *) malloc(sizeof(char) * (p + 1));
```
Don't forget the null terminator!

(c) An $n \times m$ matrix `mat` of integers initialized to zero.

```
mat = (int *) calloc(n * m, sizeof(int));
```

Alternative solution. This might be needed if you wanted to efficiently permute the rows of the matrix.

```
1 mat = (int **) calloc(n, sizeof(int *));
2 for (int i = 0; i < n; i++)
3     mat[i] = (int *) calloc(m, sizeof(int));
```

2.3 Compare the following two implementations of a function which duplicates a string. Is either one correct? Which one runs faster?

```
1 char* strdup1(char* s) {
2     int n = strlen(s);
3     char* new_str = malloc((n + 1) * sizeof(char));
4     for (int i = 0; i < n; i++) new_str[i] = s[i];
```

```
    return new_str;
}
char* strdup2(char* s) {
    int n = strlen(s);
    char* new_str = calloc(n + 1, sizeof(char));
    for (int i = 0; i < n; i++) new_str[i] = s[i];
    return new_str;
}
```

The first implementation is incorrect because malloc doesn't initialize the allocated memory to any given value, so the new string may not be null-terminated. This is easily fixed, however, just by setting the last character in new_str to the null terminator. The second implementation is correct since calloc will set each character to zero, so the string is always null-terminated.

Between the two implementations, the first will run slightly faster since malloc doesn't set the memory values, and thus runs in O(1) time. calloc does set each memory location, so it runs in O(n) time. Effectively, we do "extra" work in the second implementation setting every character to zero, and then overwrite them with the copied values afterwards.

2.4 What's the main issue with the code snippet seen here? (Hint: `gets()` is a function that reads in user input and stores it in the array given in the argument.)

```
char* foo() {
    char buffer[64];
    gets(buffer);

    char* important_stuff = (char*) malloc(11 * sizeof(char));

    int i;
    for (i = 0; i < 10; i++) important_stuff[i] = buffer[i];
    important_stuff[i] = '\0';
    return important_stuff;
}
```

If the user input contains more than 63 characters, then the input will override other parts of the memory! (You will learn more about this and how it can be used to maliciously exploit programs in CS 161.)

Note that it's perfectly acceptable in C to create an array on the stack. It's often discouraged (mostly because people often forget the array was initialized on the stack and accidentally return a pointer to it), but it's not an issue itself.

3 Linked List

Suppose we've defined a linked list **struct** as follows. Assume `*lst` points to the first element of the list, or is `NULL` if the list is empty.

```
struct ll_node {
    int first;
    struct ll_node* rest;
}
```

3.1 Implement `prepend`, which adds one new `value` to the front of the linked list. Hint: why use ll_node $**lst$ instead of ll_node$*lst$?

```
1  void prepend(struct ll_node** lst, int value) {
2      struct ll_node* item = (struct ll_node*) malloc(sizeof(struct ll_node));
3      item->first = value;
4      item->rest = *lst;
5      *lst = item;
6  }
```

3.2 Implement `free_ll`, which frees all the memory consumed by the linked list.

```
1  void free_ll(struct ll_node** lst) {
2      if (*lst) {
3          free_ll(&((*lst)->rest));
4          free(*lst);
5      }
6      *lst = NULL; // Make writes to **lst fail instead of writing to unusable memory.
7  }
```

4 Floating Point

The IEEE 754 standard defines a binary representation for floating point values using three fields.

- The *sign* determines the sign of the number (0 for positive, 1 for negative).
- The *exponent* is in **biased notation**. For instance, the bias is -127 which comes from -$(2^{8-1}-1)$ for single-precision floating point numbers.
- The *significand* or *mantissa* is akin to unsigned integers, but used to store a fraction instead of an integer.

The below table shows the bit breakdown for the single precision (32-bit) representation. The leftmost bit is the MSB and the rightmost bit is the LSB.

1	8	23
Sign	Exponent	Mantissa/Significand/Fraction

For normalized floats:
Value $= (-1)^{Sign} * 2^{Exp+Bias} * 1.\textbf{significand}_2$
For denormalized floats:
Value $= (-1)^{Sign} * 2^{Exp+Bias+1} * 0.\textbf{significand}_2$

Exponent	Significand	Meaning
0	Anything	Denorm
1-254	Anything	Normal
255	0	Infinity
255	Nonzero	NaN

Note that in the above table, our exponent has values from 0 to 255. When translating between binary and decimal floating point values, we must remember that there is a bias for the exponent.

4.1 Convert the following single-precision floating point numbers from binary to decimal or from decimal to binary. You may leave your answer as an expression.

- 0x00000000

  0

- 8.25

  0x41040000

- 0x00000F00

  $(2^{-12} + 2^{-13} + 2^{-14} + 2^{-15}) * 2^{-126}$

- 39.5625

  0x421E4000

- 0xFF94BEEF

  NaN

- -∞

  0xFF800000

- 1/3

  N/A — Impossible to actually represent, we can only approximate it

5 More Floating Point Representation

As we saw above, not every number can be represented perfectly using floating point. For this question, we will only look at positive numbers.

5.1 What is the next smallest number larger than 2 that can be represented completely?

For this question, you increment the number by the smallest amount possible. This is the same as incrementing the significand by 1 at the rightmost location.
$(1 + 2^{-23}) * 2 = 2 + 2^{-22}$

5.2 What is the next smallest number larger than 4 that can be represented completely?

For this question, you increment the number by the smallest amount possible. This is the same as incrementing the significand by 1 at the rightmost location.
$(1 + 2^{-23}) * 4 = 4 + 2^{-21}$

5.3 What is the largest odd number that we can represent? Hint: Try applying the step size technique covered in lecture.

To find the largest odd number we can represent, we want to find when odd numbers will stop appearing. This will be with step size of 2.

As a result, plugging into Part 4: $2 = 2^{x-150} \rightarrow x = 151$
This means the number before $2^{151-127}$ was a distance of 1 (it is the first value whose stepsize is 2) and no number after will be odd. Thus, the odd number is simply subtracting the previous step size of 1.This gives,
$2^{24} - 1$

CS 61C Fall 2022

RISC-V Assembly, Control

Discussion 4

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 Let `a0` point to the start of an array `x`. `lw s0, 4(a0)` will always load `x[1]` into `s0`.

1.2 Assuming integers are 4 bytes, adding the ASCII character `'d'` to the address of an integer array would get you the element at index 25 of that array (assuming the array is large enough).

1.3 Assuming no compiler or operating system protections, it is possible to have the code jump to data stored at `0(a0)` (offset 0 from the value in register `a0`) and execute instructions from there.

1.4 `jalr` is a shorthand expression for a `jal` that jumps to the specified label and does not store a return address anywhere.

1.5 Calling `j label` does the exact same thing as calling `jal label`.

2 Instructions

RISC-V is an assembly language, which is comprised of simple instructions that each do a single task such as addition or storing a chunk of data to memory.

For example, on the left is a snippet of C code and on the right is a chunk of RISC-V code that accomplishes the same thing.

```
int x = 5;
y[2];
y[0] = x;
y[1] = x * x;
```

```
// x -> s0, &y -> s1
addi s0, x0, 5
sw   s0, 0(s1)
mul  t0, s0, s0
sw   t0, 4(s1)
```

For your reference, here are some of the basic instructions for arithmetic operations and dealing with memory (Note: ARG1 is argument register 1, ARG2 is argument register 2, and DR is destination register):

[inst]	[destination register] [argument register 1] [argument register 2]
add	Adds the two argument registers and stores in destination register
xor	Exclusive or's the two argument registers and stores in destination register
mul	Multiplies the two argument registers and stores in destination register
sll	Logical left shifts ARG1 by ARG2 and stores in DR
srl	Logical right shifts ARG1 by ARG2 and stores in DR
sra	Arithmetic right shifts ARG1 by ARG2 and stores in DR
slt/u	If ARG1 < ARG2, stores 1 in DR, otherwise stores 0, u does unsigned comparison

[inst]	[register] [offset]([register containing base address])
sw	Stores the contents of the register to the address+offset in memory
lw	Takes the contents of address+offset in memory and stores in the register

[inst]	[argument register 1] [argument register 2] [label]
beq	If ARG1 == ARG2, moves to label
bne	If ARG1 != ARG2, moves to label

[inst]	[destination register] [label]
jal	Stores the next instruction's address into DR and moves to label

You may also see that there is an "i" at the end of certain instructions, such as addi, slli, etc. This means that ARG2 becomes an "immediate" or an integer instead of using a register. There are also immediates in some other instructions such as **sw** and **lw**. Note that the size (maximum number of bits) of an immediate in any given instruction depends on what type of instruction it is (more on this soon!).

2.1 Assume we have an array in memory that contains `int *arr = {1,2,3,4,5,6,0}`. Let register `s0` hold the address of the element at index 0 in arr. You may assume integers are four bytes and our values are word-aligned. What do the snippets of RISC-V code do? Assume that all the instructions are run one after the other in the same context.

```
a) lw   t0, 12(s0)        -->

b) sw   t0, 16(s0)        -->

c) slli t1, t0, 2
   add  t2, s0, t1
   lw   t3, 0(t2)         -->
   addi t3, t3, 1
   sw   t3, 0(t2)

d) lw   t0, 0(s0)
   xori t0, t0, 0xFFF     -->
   addi t0, t0, 1
```

2.2 Assume that `s0` and `s1` contain signed integers. Without any pseudoinstructions, how can we branch on the following conditions to jump to some LABEL?

`s0` $<$ `s1` `s0` $\neq$ `s1` `s0` $\leq$ `s1` `s0` $>$ `s1`

3 Lost in Translation

3.1 Translate between the C and RISC-V verbatim.

C	RISC-V

`// s0 -> a, s1 -> b`
`// s2 -> c, s3 -> z`
`int a = 4, b = 5, c = 6, z;`
`z = a + b + c + 10;`	

`// s0 -> int * p = intArr;`
`// s1 -> a;`
`*p = 0;`
`int a = 2;`
`p[1] = p[a] = a;`	

`// s0 -> a, s1 -> b`
`int a = 5, b = 10;`
`if(a + a == b) {`
`  a = 0;`
`} else {`
`  b = a - 1;`
`}`	

	``` addi s0, x0, 0 addi s1, x0, 1 addi t0, x0, 30 loop: beq s0, t0, exit add s1, s1, s1 addi s0, s0, 1 jal x0, loop exit: ```

``
// s0 -> n, s1 -> sum
// assume n > 0 to start
for(int sum = 0; n > 0; n--) {
  sum += n;
}
``	

4 Arrays in RISC-V

Comment what each code block does. Each block runs in isolation. Assume that there is an array, `int arr[6] = {3, 1, 4, 1, 5, 9}`, which starts at memory address `0xBFFFFF00`, and a linked list struct (as defined below), `struct ll* lst`, whose first element is located at address `0xABCD0000`. Let `s0` contain `arr`'s address `0xBFFFFF00`, and let `s1` contain `lst`'s address `0xABCD0000`. You may assume integers and pointers are 4 bytes and that structs are tightly packed. Assume that `lst`'s last node's `next` is a NULL pointer to memory address `0x00000000`.

```
struct ll {
    int val;
    struct ll* next;
}
```

4.1
```
lw  t0, 0(s0)
lw  t1, 8(s0)
add t2, t0, t1
sw  t2, 4(s0)
```

4.2
```
loop: beq  s1, x0, end
      lw   t0, 0(s1)
      addi t0, t0, 1
      sw   t0, 0(s1)
      lw   s1, 4(s1)
      jal  x0, loop
 end:
```

4.3
```
       add  t0, x0, x0
loop:  slti t1, t0, 6
       beq  t1, x0, end
       slli t2, t0, 2
       add  t3, s0, t2
       lw   t4, 0(t3)
       sub  t4, x0, t4
       sw   t4, 0(t3)
       addi t0, t0, 1
       jal  x0, loop
 end:
```

5 Memory Access

Using the given instructions and the sample memory arrays provided, what will happen when the RISC-V code is executed? For load instructions (`lw, lb, lh`), write out what each register will store. For store instructions (`sw, sh, sb`), update the memory array accordingly. Recall that RISC-V is little-endian and byte addressable.

5.1
```
li x5 0x00FF0000
lw x6 0(x5)
addi x5 x5 4
lh x7 2(x5)
lw x8 0(x6)
lb x9 3(x7)
```

What value does each register hold after the code is executed?

0xFFFFFFFF	
	...
0x00FF0004	0x000C561C
0x00FF0000	36
	...
0x00000036	0xFDFDFDFD
0x00000024	0xDEADB33F
	...
0x0000000C	0xC5161C00
	...
0x00000000	

5.2
```
li x5 0xABADCAFE
li x6 0xF9120504
li x7 0xBEEFCACE
sw x5 0(x6)
addi x6 x6 4
addi x5 x5 4
sh x6 2(x5)
sb x7 1(x7)
sb x7 3(x6)
sb x7 3(x5)
```

0xFFFFFFFF	
0xF9120504	
0xABADCAFE	
0x00000004	
0x00000000	0x00000000

Update the memory array with its new values after the code is executed. Some memory addresses may not have been labeled for you yet.

CS 61C Fall 2022 — RISC-V Assembly, Control

Discussion 4

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 Let `a0` point to the start of an array `x`. `lw s0, 4(a0)` will always load `x[1]` into `s0`.

False. This only holds for data types that are four bytes wide, like `int` or `float`. For data-types like `char` that are only one byte wide, `4(a0)` is too large of an offset to return the element at index 1, and will instead return a `char` further down the array (or some other data beyond the array, depending on the array length).

1.2 Assuming integers are 4 bytes, adding the ASCII character `'d'` to the address of an integer array would get you the element at index 25 of that array (assuming the array is large enough).

True. There is no fundamental difference between integers, strings, and memory addresses in RISC-V (they're all bags of bits), so it's possible to manipulate data in this way. (We don't recommend it, though).

1.3 Assuming no compiler or operating system protections, it is possible to have the code jump to data stored at `0(a0)` (offset 0 from the value in register `a0`) and execute instructions from there.

True. If your compiler/OS allows it (some do not, for security reasons), it is possible for your code to jump to and execute instructions passed into the program via an array. Conversely, it's also possible for your code to treat itself as normal data (search up self-modifying code if you want to see more details).

1.4 `jalr` is a shorthand expression for a `jal` that jumps to the specified label and does not store a return address anywhere.

False. `j label` is a pseudo-instruction for `jal x0, label`. `jalr` is used to return to the memory address specified in the second argument. Keep in mind that `jal` jumps to a label (which is translated into an immediate by the assembler), whereas `jalr` jumps to an address stored in a register, which is set at runtime.

1.5 Calling `j label` does the exact same thing as calling `jal label`.

False. As from the previous problem, `j label` is short for `jal x0, label` — since it's writing the return address to `x0`, it's effectively discarding it since we have no need to jump back to the original PC. `jal label` is short for `jal ra, label`.

2 Instructions

RISC-V is an assembly language, which is comprised of simple instructions that each do a single task such as addition or storing a chunk of data to memory.

For example, on the left is a snippet of C code and on the right is a chunk of RISC-V code that accomplishes the same thing.

```
int x = 5;
y[2];
y[0] = x;
y[1] = x * x;
```

```
// x -> s0, &y -> s1
addi s0, x0, 5
sw   s0, 0(s1)
mul  t0, s0, s0
sw   t0, 4(s1)
```

For your reference, here are some of the basic instructions for arithmetic operations and dealing with memory (Note: ARG1 is argument register 1, ARG2 is argument register 2, and DR is destination register):

[inst]	[destination register] [argument register 1] [argument register 2]
add	Adds the two argument registers and stores in destination register
xor	Exclusive or's the two argument registers and stores in destination register
mul	Multiplies the two argument registers and stores in destination register
sll	Logical left shifts ARG1 by ARG2 and stores in DR
srl	Logical right shifts ARG1 by ARG2 and stores in DR
sra	Arithmetic right shifts ARG1 by ARG2 and stores in DR
slt/u	If ARG1 < ARG2, stores 1 in DR, otherwise stores 0, u does unsigned comparison

[inst]	[register] [offset]([register containing base address])
sw	Stores the contents of the register to the address+offset in memory
lw	Takes the contents of address+offset in memory and stores in the register

[inst]	[argument register 1] [argument register 2] [label]
beq	If ARG1 == ARG2, moves to label
bne	If ARG1 != ARG2, moves to label

[inst]	[destination register] [label]
jal	Stores the next instruction's address into DR and moves to label

You may also see that there is an "i" at the end of certain instructions, such as addi, slli, etc. This means that ARG2 becomes an "immediate" or an integer instead of using a register. There are also immediates in some other instructions such as **sw** and **lw**. Note that the size (maximum number of bits) of an immediate in any given instruction depends on what type of instruction it is (more on this soon!).

2.1 Assume we have an array in memory that contains `int *arr = {1,2,3,4,5,6,0}`. Let register `s0` hold the address of the element at index 0 in arr. You may assume integers are four bytes and our values are word-aligned. What do the snippets of RISC-V code do? Assume that all the instructions are run one after the other in the same context.

```
a) lw   t0, 12(s0)        -->       Sets t0 equal to arr[3]

b) sw   t0, 16(s0)        -->       Stores t0 into arr[4]

c) slli t1, t0, 2
   add  t2, s0, t1
   lw   t3, 0(t2)         -->       Increments arr[t0] by 1
   addi t3, t3, 1
   sw   t3, 0(t2)

d) lw   t0, 0(s0)
   xori t0, t0, 0xFFF     -->       Sets t0 to -1 * arr[0]
   addi t0, t0, 1
```

2.2 Assume that `s0` and `s1` contain signed integers. Without any pseudoinstructions, how can we branch on the following conditions to jump to some LABEL?

`s0` $<$ `s1` `s0` $\neq$ `s1` `s0` $\leq$ `s1` `s0` $>$ `s1`

```
blt s0, s1, LABEL  bne s0, s1, LABEL  bge s1, s0, LABEL  blt s1, s0, LABEL
```

Note that RISC-V does not provide a bgt instruction because you can manipulate the blt instruction to get an equivalent result. Also note that the above solutions assume that `s0` and `s1` contained signed integers. If they are unsigned, then we would use the unsigned variants of the above commands (namely, `bltu, bgeu`).

3 Lost in Translation

3.1 Translate between the C and RISC-V verbatim.

C	RISC-V

`// s0 -> a, s1 -> b`
`// s2 -> c, s3 -> z`
`int a = 4, b = 5, c = 6, z;`
`z = a + b + c + 10;`
	`addi s0, x0, 4`
	`addi s1, x0, 5`
	`addi s2, x0, 6`
	`add  s3, s0, s1`
	`add  s3, s3, s2`
	`addi s3, s3, 10`

`// s0 -> int * p = intArr;`
`// s1 -> a;`
`*p = 0;`
`int a = 2;`
`p[1] = p[a] = a;`
	`sw   x0, 0(s0)`
	`addi s1, x0, 2`
	`sw   s1, 4(s0)`
	`slli t0, s1, 2`
	`add  t0, t0, s0`
	`sw   s1, 0(t0)`

`// s0 -> a, s1 -> b`
`int a = 5, b = 10;`
`if(a + a == b) {`
`  a = 0;`
`} else {`
`  b = a - 1;`
`}`
	`  addi s0, x0, 5`
	`  addi s1, x0, 10`
	`  add  t0, s0, s0`
	`  bne  t0, s1, else`
	`  xor  s0, x0, x0`
	`  jal  x0,  exit`
	`else:`
	`  addi s1, s0, -1`
	`exit:`

`// computes s1 = 2^30`
`// assume int s1, s0; was declared above`
`s1 = 1;`
`for(s0 = 0; s0 != 30; s0++) {`
`  s1 *= 2;`
`}`
	`  addi s0, x0, 0`
	`  addi s1, x0, 1`
	`  addi t0, x0, 30`
	`loop:`
	`  beq  s0, t0, exit`
	`  add  s1, s1, s1`
	`  addi s0, s0, 1`
	`  jal  x0, loop`
	`exit:`

`// s0 -> n, s1 -> sum`
`// assume n > 0 to start`
`for(int sum = 0; n > 0; n--) {`
`  sum += n;`
`}`
	`  addi s1, x0, 0`
	`loop:`
	`  beq  s0, x0, exit`
	`  add  s1, s1, s0`
	`  add  s0, s0, -1`
	`  jal  x0, loop`
	`exit:`

4 Arrays in RISC-V

Comment what each code block does. Each block runs in isolation. Assume that there is an array, `int arr[6] = {3, 1, 4, 1, 5, 9}`, which starts at memory address `0xBFFFFF00`, and a linked list struct (as defined below), `struct ll* lst`, whose first element is located at address `0xABCD0000`. Let `s0` contain `arr`'s address `0xBFFFFF00`, and let `s1` contain `lst`'s address `0xABCD0000`. You may assume integers and pointers are 4 bytes and that structs are tightly packed. Assume that `lst`'s last node's `next` is a NULL pointer to memory address `0x00000000`.

```
struct ll {
    int val;
    struct ll* next;
}
```

4.1 `lw  t0, 0(s0)`

```
lw  t1, 8(s0)
add t2, t0, t1
sw  t2, 4(s0)
```

Sets `arr[1]` to `arr[0]` + `arr[2]`.

4.2
```
loop: beq  s1, x0, end
      lw   t0, 0(s1)
      addi t0, t0, 1
      sw   t0, 0(s1)
      lw   s1, 4(s1)
      jal  x0, loop
 end:
```

Increments all values in the linked list by 1.

4.3
```
       add  t0, x0, x0
loop:  slti t1, t0, 6
       beq  t1, x0, end
       slli t2, t0, 2
       add  t3, s0, t2
       lw   t4, 0(t3)
       sub  t4, x0, t4
       sw   t4, 0(t3)
       addi t0, t0, 1
       jal  x0, loop
 end:
```

Negates all elements in `arr`.

5 Memory Access

Using the given instructions and the sample memory arrays provided, what will happen when the RISC-V code is executed? For load instructions (`lw, lb, lh`), write out what each register will store. For store instructions (`sw, sh, sb`), update the memory array accordingly. Recall that RISC-V is little-endian and byte addressable.

5.1
```
li x5 0x00FF0000
lw x6 0(x5)
addi x5 x5 4
lh x7 2(x5)
lw x8 0(x6)
lb x9 3(x7)
```

0xFFFFFFFF	
	...
0x00FF0004	0x000C561C
0x00FF0000	36
	...
0x00000036	0xFDFDFDFD
0x00000024	0xDEADB33F
	...
0x0000000C	0xC5161C00
	...
0x00000000	

What value does each register hold after the code is executed?

`x5` will hold `0x00FF0004`, after adding 4 to the initial address. `x6` will hold 36, loading the word from the address `0x00FF0000`. `x7` will hold `0xC`, loading the upper half of the address `0x00FF0004`. `x8` will hold the word at 36 = 0x24, so `0xDEADB33F`. Finally, `x9` will hold `0xFFFFFFC5`, taking the most significant byte and sign-extending it.

5.2

```
li x5 0xABADCAFE
li x6 0xF9120504
li x7 0xBEEFCACE
sw x5 0(x6)
addi x6 x6 4
addi x5 x5 4
sh x6 2(x5)
sb x7 1(x7)
sb x7 3(x6)
sb x7 3(x5)
```

0xFFFFFFFF	
0xF9120504	
0xABADCAFE	
0x00000004	
0x00000000	0x00000000

Update the memory array with its new values after the code is executed. Some memory addresses may not have been labeled for you yet.

0xFFFFFFFF	
0xF9120508	0xCE000000
0xF9120504	0xABADCAFE
0xBEEFCAD2	
0xBEEFCACE	0x0000CE00
0xABADCB02	0xCE080000
0xABADCAFE	
0x00000004	
0x00000000	0x00000000

CS 61C Fall 2022

CALL, RISC-V Procedures

Discussion 5

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 After calling a function and having that function return, the `t` registers may have been changed during the execution of the function, while `a` registers cannot.

1.2 Because `a0` and `a1` are the return values of a function, the other `a` registers will be unchanged after returning from a function.

1.3 The stack should only be manipulated at the beginning and end of functions, where the callee saved registers are temporarily saved.

1.4 The compiler may output pseudoinstructions.

1.5 The main job of the assembler is to generate optimized machine code.

1.6 The object files produced by the assembler are only moved, not edited, by the linker.

1.7 The destination of all jump instructions is completely determined after linking.

2 Calling Convention Practice

2.1 In a function called `myfunc`, we want to call two functions called `generate_random` and `reverse_and_multiply`.

`myfunc` takes in 3 arguments: `a0, a1, a2`

`generate_random` takes in no arguments and returns a random integer to `a0`.

`reverse_and_multiply` takes in 4 arguments: `a0, a1, a2, a3` and doesn't return anything.

```
myfunc:
    # Prologue (omitted)

    # assign registers to hold arguments to myfunc
    addi t0 a0 0
    addi s0 a1 0
    addi a7 a2 0

    jal generate_random

    # store and process return value
    addi t1 a0 0
    slli t5 t1 2

    # setup arguments for reverse
    add a0 t0 x0
    add a1 s0 x0
    add a2 t5 x0
    addi a3 t1 0

    jal reverse

    # additional computations
    add t0 s0 x0
    add t1 t1 a7
    add s9 s8 s7
    add s3 x0 t5

    # Epilogue (omitted)
    ret
```

2.1 Which registers, if any, need to be saved on the stack in the prologue?

2.2 If generate_random follows calling conventions, which registers do we need to save on the stack before calling generate_random?

2.3 If reverse follows calling conventions, which registers do we need to save on the stack before calling `reverse`?

2.4 Which registers need to be recovered in the epilogue before returning?

3 Translation

3.1 In this question, we will be translating between RISC-V code and binary/hexadecimal values.

Translate the following Risc-V instructions into binary and hexadecimal notations

```
addi  s1  x0  -24  =      0b_____________________________    =  0x___________
sh  s1  4(t1)      =      0b_____________________________    =  0x___________
```

3.2 In this question, we will be translating between RISC-V code and binary/hexadecimal values.

Translate the following hexadecimal values into the relevant RISC-V instruction.

```
0x234554B7 =  _____________________________________
0xFE050CE3 = _____________________________________
```

4 RISC-V Addressing

We have several *addressing modes* to access memory (immediate not listed):

1. Base displacement addressing adds an immediate to a register value to create a memory address (used for lw, lb, sw, sb).
2. PC-relative addressing uses the PC and adds the immediate value of the instruction (multiplied by 2) to create an address (used by branch and jump instructions).
3. Register Addressing uses the value in a register as a memory address. For instance, `jalr`, `jr`, and `ret`, where `jr` and `ret` are just pseudoinstructions that get converted to `jalr`.

4.1 What is the range of 32-bit instructions that can be reached from the current PC using a branch instruction? Recall that RISC-V supports 16b instructions via an extension.

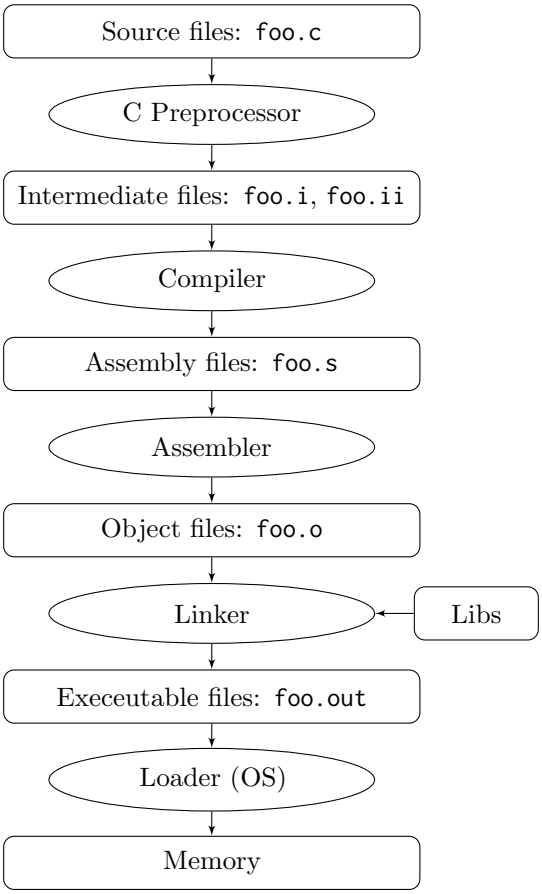
4.2 What is the maximum range of 32-bit instructions that can be reached from the current PC using a jump instruction?

4.3 Given the following RISC-V code (and instruction addresses), fill in the blank fields for the following instructions (you'll need your RISC-V reference sheet!). Each field refers to a different block of the instruction encoding.

```
0x002cff00: loop: add t1, t2, t0          |_______|_______|_______|_______|_______|__0x33__|
0x002cff04:       jal ra, foo             |_____________________________|_____________|__0x6F__|
0x002cff08:       bne t1, zero, loop      |_______|_______|_______|_______|_______|__0x63__|
...
0x002cff2c: foo:  jr ra                 ra = _____________________________
```

5 CALL

The following is a diagram of the CALL stack detailing how C programs are built and executed by machines.



5.1 How many passes through the code does the Assembler have to make? Why?

5.2 Which step in CALL resolves relative addressing? Absolute addressing?

5.3 Describe the six main parts of the object files outputted by the Assembler (Header, Text, Data, Relocation Table, Symbol Table, Debugging Information).

6 Assembling RISC-V

Let's say that we have a C program that has a single function `sum` that computes the sum of an array. We've compiled it to RISC-V, but we haven't assembled the RISC-V code yet.

```
1  .import print.s             # print.s is a different file
2  .data
3          array: .word 1 2 3 4 5
4  .text
5  .globl sum
6  sum:    la t0, array
7          li t1, 4
8          mv t2, x0
9  loop:   blt t1, x0, end
10         slli t3, t1, 2
11         add t3, t0, t3
12         lw t3, 0(t3)
13         add t2, t2, t3
14         addi t1, t1, -1
15         j loop
16 end:    mv a0, t2
17         jal ra, print_int   # Defined in print.s
```

6.1 Which lines contain pseudoinstructions that need to be converted to regular RISC-V instructions?

6.2 For the branch/jump instructions, which labels will be resolved in the first pass of the assembler? The second? Assume that the code is processed from top to bottom.

Let's assume that the code for this program starts at address `0x00061C00`. The code below is labelled with its address in memory (think: why is there a jump of 8 between the first and second lines?).

```
1  0x00061C00: sum:    la t0, array
2  0x00061C08:         li t1, 4
3  0x00061C0C:         mv t2, x0
4  0x00061C10: loop:   blt t1, x0, end
5  0x00061C14:         slli t3, t1, 2
6  0x00061C18:         add t3, t0, t3
7  0x00061C1C:         lw t3, 0(t3)
8  0x00061C20:         add t2, t2, t3
9  0x00061C24:         addi t1, t1, -1
10 0x00061C28:         j loop
11 0x00061C2C: end:    mv a0, t2
12 0x00061C30:         jal ra, print_int
```

6.3 What is in the symbol table after the assembler makes its passes?

6.4 What's contained in the relocation table?

CS 61C Fall 2022

CALL, RISC-V Procedures

Discussion 5

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 After calling a function and having that function return, the `t` registers may have been changed during the execution of the function, while `a` registers cannot.

False. `a0` and `a1` registers are often used to store the return value from a function, so the function can set their values to the its return values before returning.

1.2 Because `a0` and `a1` are the return values of a function, the other `a` registers will be unchanged after returning from a function.

False. While `a0` and `a1` are the return values, all `a` registers must be caller saved. The function may have manipulated the argument registers in its execution, so it is unknown that `a` registers will stay the same.

1.3 The stack should only be manipulated at the beginning and end of functions, where the callee saved registers are temporarily saved.

False. While it is a good idea to create a separate 'prologue' and 'epilogue' to save callee registers onto the stack, the stack is mutable anywhere in the function. A good example is if you want to preserve the current value of a temporary register, you can decrement the `sp` to save the register onto the stack right before a function call.

1.4 The compiler may output pseudoinstructions.

True. It is the job of the assembler to replace these pseudoinstructions.

1.5 The main job of the assembler is to generate optimized machine code.

False. That's the job of the compiler. The assembler is primarily responsible for replacing pseudoinstructions and resolving offsets.

1.6 The object files produced by the assembler are only moved, not edited, by the linker.

False. The linker needs to relocate all absolute address references.

1.7 The destination of all jump instructions is completely determined after linking.

False. Jumps relative to registers (i.e. from jalr instructions) are only known at run-time. Otherwise, you would not be able to call a function from different call sites.

2 Calling Convention Practice

2.1 In a function called `myfunc`, we want to call two functions called `generate_random` and `reverse_and_multiply`.

`myfunc` takes in 3 arguments: `a0, a1, a2`

`generate_random` takes in no arguments and returns a random integer to `a0`.

`reverse_and_multiply` takes in 4 arguments: `a0, a1, a2, a3` and doesn't return anything.

```
1  myfunc:
2      # Prologue (omitted)
3
4      # assign registers to hold arguments to myfunc
5      addi t0 a0 0
6      addi s0 a1 0
7      addi a7 a2 0
8
9      jal generate_random
10
11     # store and process return value
12     addi t1 a0 0
13     slli t5 t1 2
14
15     # setup arguments for reverse
16     add a0 t0 x0
17     add a1 s0 x0
18     add a2 t5 x0
19     addi a3 t1 0
20
21     jal reverse
22
23     # additional computations
24     add t0 s0 x0
25     add t1 t1 a7
26     add s9 s8 s7
27     add s3 x0 t5
28
29     # Epilogue (omitted)
30     ret
```

2.1 Which registers, if any, need to be saved on the stack in the prologue?

`s0`, `s3`, `s9`, `ra` We must save all s-registers we modify, and it is conventional to store ra in the prologue (rather than just before calling a function) when the function contains a function call.

2.2 If generate_random follows calling conventions, which registers do we need to save on the stack before calling generate_random?

`t0`, `a7`

Under calling conventions, all the t-registers and a-registers may be changed by generate_random, so we must store all of these which we need to know the value of after the call. t0 is used on line 16 and a7 is used on line 25. Note that while t1 and t5 are used later, we don't care about its value before calling generate_random (they are set after the call, on lines 12-13), so we don't need to store them.

2.3 If reverse follows calling conventions, which registers do we need to save on the stack before calling `reverse`?

`t1`, `t5`, `a7`

As before, we must save t-registers and a-registers we need to read later.

2.4 Which registers need to be recovered in the epilogue before returning?

`s0`, `s3`, `s9`, `ra`

This mirrors what we saved in the prologue.

3 Translation

3.1 In this question, we will be translating between RISC-V code and binary/hexadecimal values.

Translate the following Risc-V instructions into binary and hexadecimal notations

```
1 addi  s1  x0  -24  =      0b_____________________________    =  0x___________
2 sh  s1  4(t1)      =      0b_____________________________    =  0x___________
```

For this question, use the reference sheet to get information about the instructions and convert them to binary representation. One thing that helps is splitting the parsing into parts. For question 1:

```
1 addi  s1  x0  x4:
2 rd= s1 = 0b01001
3 rs1 = x0 = 0b00000
4 immediate = -24 = 0b1111 1110 1000
5 opcode = 001 0011
6 funct3 = 000
7 Bringing it together - 0b1111 1110 1000 0000 0000 0100 1001 0011 = 0xFE800493
```

For question 2, with a similar method we get the answer: 0b0000 0000 1001 0011 0001 0010 0010 0011 = 0x00931223

3.2 In this question, we will be translating between RISC-V code and binary/hexadecimal values.

Translate the following hexadecimal values into the relevant RISC-V instruction.

```
1 0x234554B7 =  _____________________________
2 0xFE050CE3 = _____________________________
```

For the reverse conversion, we want to first determine the instruction type. In order to do that, we first look at the opcode (and then func3/func7 if necessary). Let's start with the first one:

```
0x234554B7 = 0b0010 0011 0100 0101 0101 0100 1011 0111, the opcode is always the last 7 bits so
    opcode = 011 0111, which corresponds to the operation lui!
Looking at lui, we can see that the first 20 bits correspond to the immediate, and the next 5 ones
    are the register ones. So:
0b0010 0011 0100 0101 0101 = 0x23455 So, the immediate input was indeed 0x23455.
Looking at the next 5 bits, they must be the rd register values. So, we have
rd = 0b01001
That is equal to 9, which is the register x9 = s1. Thus, overall we have
lui s1 0x23455
```

For question 2, with a similar approach: beq a0, x0, -8

4 RISC-V Addressing

We have several *addressing modes* to access memory (immediate not listed):

1. Base displacement addressing adds an immediate to a register value to create a memory address (used for lw, lb, sw, sb).
2. PC-relative addressing uses the PC and adds the immediate value of the instruction (multiplied by 2) to create an address (used by branch and jump instructions).
3. Register Addressing uses the value in a register as a memory address. For instance, `jalr`, `jr`, and `ret`, where `jr` and `ret` are just pseudoinstructions that get converted to `jalr`.

4.1 What is the range of 32-bit instructions that can be reached from the current PC using a branch instruction? Recall that RISC-V supports 16b instructions via an extension.

The B-format instruction encoding has space for a 12 bit immediate field. Since RISC-V supports 16b instructions, the byte offset needed to branch to any instruction will always be divisible by 2. Thus, we can assume an implicit 0 at bit 0, allowing a 13 bit byte offset (that's why the reference sheet describes the immediate as `imm[12:1]`!). Since the byte offset is signed, the branch immediate can move the PC in the range of $[-2^{12}, 2^{12} - 1]$ bytes. As we are looking for the range of 32-bit instructions, we look for only offsets divisible by 4, leading to a total of $[-2^{10}, 2^{10} - 1]$ 32-bit instructions to branch to.

4.2 What is the maximum range of 32-bit instructions that can be reached from the current PC using a jump instruction?

The immediate field of the jal instruction is 20 bits, while that of the jalr instruction is only 12 bits, so jal can reach a wider range of instructions. Similar to above, this 20-bit immediate is multiplied by 2 and added to the PC to get the final address. Since the immediate is signed, we have a range of $[-2^{20}, 2^{20} - 2]$ bytes, or $[-2^{19}, 2^{19} - 1]$ 2-byte instructions. As we actually want the number of 4-byte

instructions, we can reference those within $[-2^{18}, 2^{18}-1]$ instructions of the current PC.

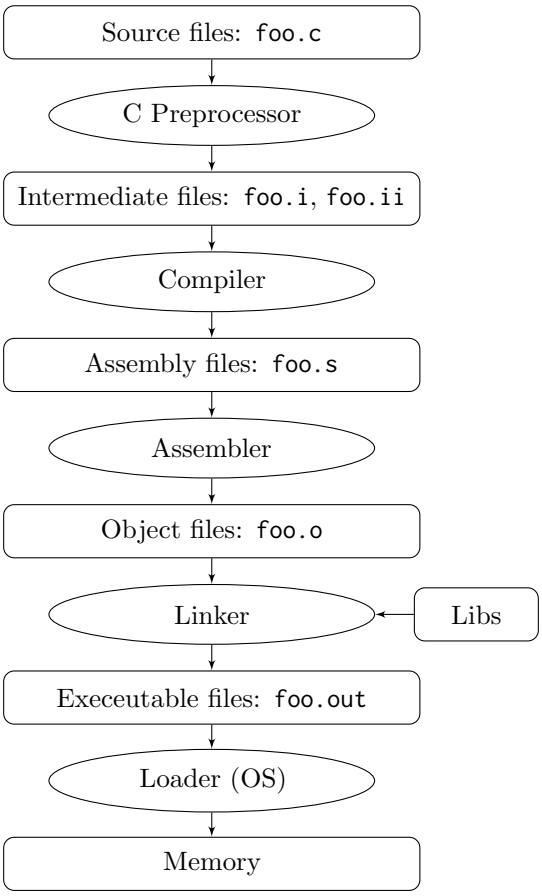
4.3 Given the following RISC-V code (and instruction addresses), fill in the blank fields for the following instructions (you'll need your RISC-V reference sheet!). Each field refers to a different block of the instruction encoding.

```
0x002cff00: loop: add t1, t2, t0          |________|________|________|________|________|__0x33__|
0x002cff04:       jal ra, foo             |_____________________________|__________________|__0x6F__|
0x002cff08:       bne t1, zero, loop      |________|________|________|________|________|__0x63__|
...
0x002cff2c: foo:  jr ra                 ra = ____________________________________
```

```
0x002cff00: loop: add t1, t2, t0       |   0   |   5   |   7    |  0  |  6  |  0x33  | → 0x00538333
0x002cff04:       jal ra, foo          |  0  |  0x14  |  0  |   0   |   1   |  0x6F  | → 0x028000ef
0x002cff08:       bne t1, zero, loop   | 1 |  0x3F  | 0 | 6 | 1 |  0xC  | 1 |  0x63  | → 0xfe031ce3
...
0x002cff2c: foo:  jr ra                ra = _________0x002cff08_________
```

5 CALL

The following is a diagram of the CALL stack detailing how C programs are built and executed by machines.



5.1 How many passes through the code does the Assembler have to make? Why?

Two: The first finds all the label addresses, and the second resolves forward references while using these label addresses.

5.2 Which step in CALL resolves relative addressing? Absolute addressing?

Assembler, Linker

5.3 Describe the six main parts of the object files outputted by the Assembler (Header, Text, Data, Relocation Table, Symbol Table, Debugging Information).

- Header: Sizes and positions of the other parts
- Text: The machine code
- Data: Binary representation of any data in the source file
- Relocation Table: Identifies lines of code that need to be "handled" by the Linker (jumps to external labels (e.g. lib files), references to static data)
- Symbol Table: List of file labels and data that can be referenced across files
- Debugging Information: Additional information for debuggers

6 Assembling RISC-V

Let's say that we have a C program that has a single function `sum` that computes the sum of an array. We've compiled it to RISC-V, but we haven't assembled the RISC-V code yet.

```
1  .import print.s             # print.s is a different file
2  .data
3          array: .word 1 2 3 4 5
4  .text
5  .globl sum
6  sum:    la t0, array
7          li t1, 4
8          mv t2, x0
9  loop:   blt t1, x0, end
10         slli t3, t1, 2
11         add t3, t0, t3
12         lw t3, 0(t3)
13         add t2, t2, t3
14         addi t1, t1, -1
15         j loop
16 end:    mv a0, t2
17         jal ra, print_int   # Defined in print.s
```

6.1 Which lines contain pseudoinstructions that need to be converted to regular RISC-V instructions?

6, 7, 8, 15, 16.

`la` becomes the `auipc` and `addi` instructions.

`li` becomes an `addi` instruction here (e.g. `li t0, 4` → `addi t0, x0, 4`).

`mv` becomes an `addi` instruction (i.e. `mv rd, rs` → `addi rd, rs, 0`).

`j` becomes a `jal` instruction (e.g. `j loop` → `jal x0, loop`).

6.2 For the branch/jump instructions, which labels will be resolved in the first pass of the assembler? The second? Assume that the code is processed from top to bottom.

`loop` (in `j loop`) will be resolved in the first pass since it's a backward reference. Since the assembler will have kept note of where `end` is in the first pass, it will resolve `end` in `blt t1, x0, end` in the second pass. (`print_int` in `jal ra, print_int` will be resolved by the Linker.)

Let's assume that the code for this program starts at address `0x00061C00`. The code below is labelled with its address in memory (think: why is there a jump of 8 between the first and second lines?).

There's a jump of 8 because `la` is a pseudoinstruction that gets translated to two regular RISC-V instructions!

```
1  0x00061C00: sum:    la t0, array
2  0x00061C08:         li t1, 4
3  0x00061C0C:         mv t2, x0
4  0x00061C10: loop:   blt t1, x0, end
5  0x00061C14:         slli t3, t1, 2
6  0x00061C18:         add t3, t0, t3
7  0x00061C1C:         lw t3, 0(t3)
8  0x00061C20:         add t2, t2, t3
9  0x00061C24:         addi t1, t1, -1
10 0x00061C28:         j loop
11 0x00061C2C: end:    mv a0, t2
12 0x00061C30:         jal ra, print_int
```

6.3 What is in the symbol table after the assembler makes its passes?

Label	**Address**
sum	`0x00061C00`

The only labels that are put in the symbol table are the ones which external programs can reference, declared using the `.globl` directive.

6.4 What's contained in the relocation table?

`array` and `print_int`.

Since `array` is defined in the static portion of memory, there's no way the assembler could know where it will be located (relative to the program counter) until the program actually executes. Recall that the static portion of memory is above the code portion of memory. Since we haven't linked other files with this one yet (that's done in the linker phase!), we don't know how much code we'll have, so we don't know where the static portion of memory will begin! Also, other files may declare items in static memory, and the assembler won't know how these are specifically ordered when the program is finally loaded.

Similarly, `print_int` is defined in a different file, so the assembler doesn't know where it will be in the final executable. That will be decided in the linking stage.

CS 61C Fall 2022

SDS and FSMs

Discussion 6

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 Simplifying boolean logic expressions will not affect the performance of the hardware implementation.

1.2 The fewer logic gates, the faster the circuit (assuming each gate has the same propagation delays).

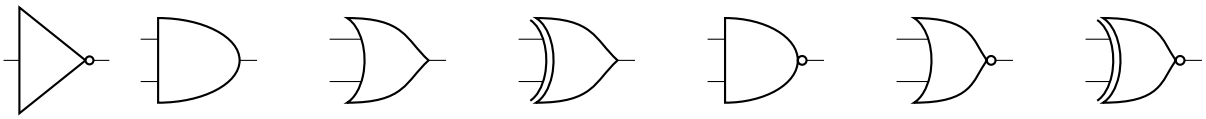
1.3 The time it takes for clock-to-q and register setup can be greater than one clock cycle.

1.4 Every possible combinational logic circuit can be expressed by some combination of NOR gates.

1.5 The shortest combinational logic path between two state elements is useful in determining circuit frequency and minimum clock cycle.

2 Logic Gates

2.1 Label the following logic gates:



2.2 Convert the following to simplified boolean expressions on input signals A and B. Remember that simplified boolean expressions should only have NOT, AND, and OR primitives ($\overline{A}$, $\times$, and $+$ respectively):

(a) NAND

(b) XOR

(c) XNOR

2.3 Create an AND gate using only NAND gates.

3 Boolean Logic

In digital electronics, it is often important to get certain outputs based on your inputs, as laid out by a truth table. Truth tables map directly to Boolean expressions, and Boolean expressions map directly to logic gates. However, in order to minimize the number of logic gates needed to implement a circuit, it is often useful to simplify long Boolean expressions.

We can simplify expressions using the nine key laws of Boolean algebra:

Name	AND Form	OR form
Commutative	$AB = BA$	$A + B = B + A$
Associative	$AB(C) = A(BC)$	$A + (B + C) = (A + B) + C$
Identity	$1A = A$	$0 + A = A$
Null	$0A = 0$	$1 + A = 1$
Absorption	$A(A + B) = A$	$A + AB = A$
Distributive	$(A + B)(A + C) = A + BC$	$A(B + C) = AB + AC$
Idempotent	$A(A) = A$	$A + A = A$
Inverse	$A(\overline{A}) = 0$	$A + \overline{A} = 1$
De Morgan's	$\overline{AB} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A}(\overline{B})$

3.1 Use multiple iterations of De Morgan's laws to prove the identity $\bar{A} + AB = \bar{A} + B$.

3.2 Prove that De Morgan's law can be generalized for the complement of any number of terms.

3.3 Simplify the following Boolean expressions:

(a) $(A + B)(A + \overline{B})C$

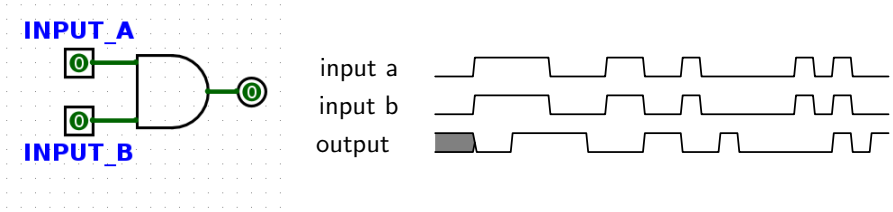
(b) $\bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + AB\bar{C} + A\bar{B}\bar{C} + ABC + A\bar{B}C$

(c) $\overline{A(\bar{B}\bar{C} + BC)}$

(d) $\overline{A}(A + B) + (B + AA)(A + \overline{B})$

4 State Intro

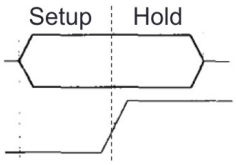
There are two basic types of circuits: combinational logic circuits and state elements. **Combinational logic** circuits simply change based on their inputs after whatever propagation delay is associated with them. For example, if an AND gate (pictured below) has an associated propagation delay of 2ps, its output will change based on its input as follows:



You should notice that the output of this AND gate *always* changes 2ps after its inputs change.

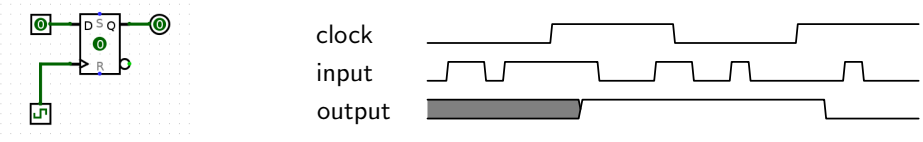
State elements, on the other hand, can *remember* their inputs even after the inputs change. State elements change value based on a clock signal. A rising edge-triggered register, for example, samples its input at the rising edge of the clock (when the clock signal goes from 0 to 1).

Like logic gates, registers also have a delay associated with them before their output will reflect the input that was sampled. This is called the **clk-to-q** delay. ("Q" often indicates output). This is the time between the rising edge of the clock signal and the time the register's output reflects the input change.



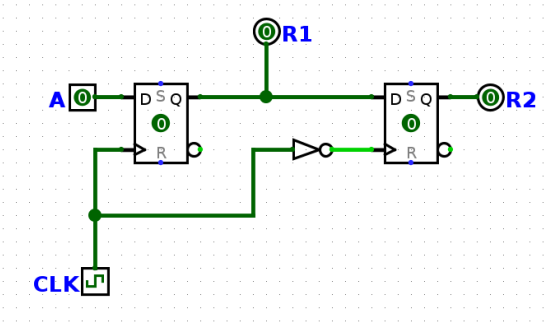
The input to the register samples has to be stable for a certain amount of time around the rising edge of the clock for the input to be sampled accurately. The amount of time before the rising edge the input must be stable is called the **setup** time, and the time after the rising edge the input must be stable is called the **hold** time. Hold time is generally included in clk-to-q delay, so clk-to-q time will usually be greater than or equal to hold time. Logically, the fact that clk-to-q $\geq$ hold time makes sense since it only takes clk-to-q seconds to copy the value over, so there's no need to have the value fed into the register for any longer.

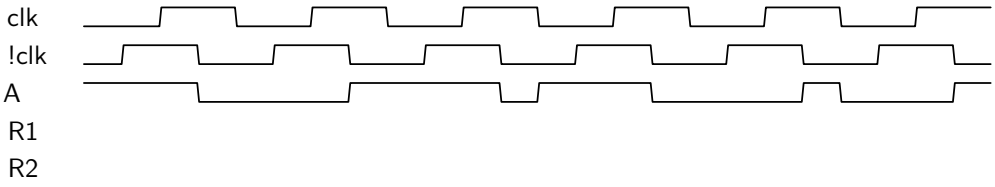
For the following register circuit, assume **setup** of 2.5ps, **hold** time of 1.5ps, and a **clk-to-q** time of 1.5ps. The clock signal has a period of 13ps.

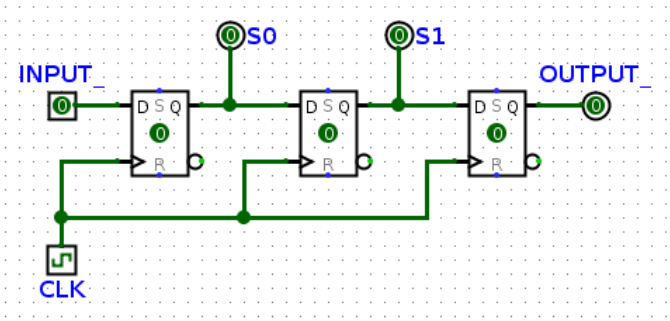


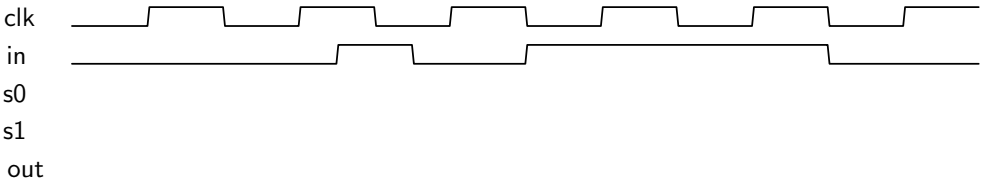
You'll notice that the value of the output in the diagram above doesn't change immediately after the rising edge of the clock. Until enough time has passed for the output to reflect the input, the value held by the output is garbage; this is represented by the shaded gray part of the output graph. Clock cycle time must be small enough that inputs to registers don't change within the hold time and large enough to account for clk-to-q times, setup times, and combinational logic delays.

4.1 For the following 2 circuits, fill out the timing diagram. The clock period (rising edge to rising edge) is 8ps. For every register, clk-to-q delay is 2ps, setup time is 4ps, and hold time is 2ps. NOT gates have a 2ps propagation delay, which is already accounted for in the !clk signal given.

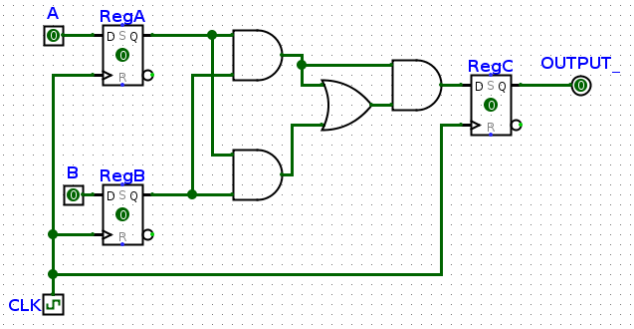








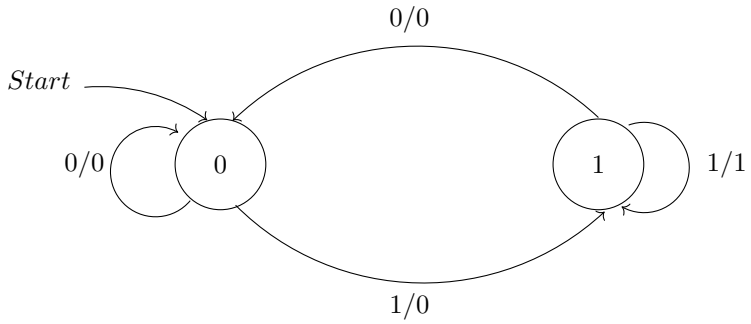
4.2 In the circuit below, RegA and RegB have setup, hold, and clk-to-q times of 4ns, all logic gates have a delay of 5ns, and RegC has a setup time of 6ns. What is the maximum allowable hold time for RegC? What is the minimum acceptable clock cycle time for this circuit, and clock frequency does it correspond to?



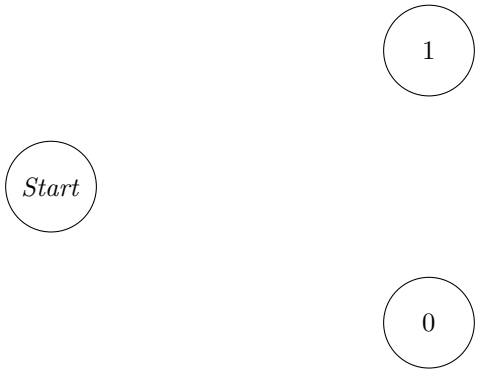
5 Finite State Machines

Automatons are machines that receive input and use various states to produce output. A finite state machine is a type of simple automaton where the next state and output depend only on the current state and input. Each state is represented by a circle, and every proper finite state machine has a starting state, signified either with the label "Start" or a single arrow leading into it. Each transition between states is labeled [input]/[output].

5.1 What pattern in a bitstring does the FSM below detect? What would it output for the input bitstring "011001001110"?



5.2 Fill in the following FSM for outputting a 1 whenever we have two repeating bits as the most recent bits, and a 0 otherwise. You may not need all states.



5.3 Draw an FSM that will output a 1 if it recognizes the regex pattern {10+1}. (That is, if the input forms a pattern of a 1, followed by one or more 0s, followed by a 1.)

CS 61C Fall 2022

SDS and FSMs

Discussion 6

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 Simplifying boolean logic expressions will not affect the performance of the hardware implementation.

False. Different gate arrangements that implement the same logic can have different propagation delays, which can affect the allowable clock speed.

1.2 The fewer logic gates, the faster the circuit (assuming each gate has the same propagation delays).

False. Propagation delays add to the allowable clock speed with the depth of the circuit, so a wide circuit with more gates in parallel can have less delay than just a few gates arranged in sequence.

1.3 The time it takes for clock-to-q and register setup can be greater than one clock cycle.

False. This can result in instability if registers are connected to each other, as register outputs may not have propagated properly before the next rising edge.

1.4 Every possible combinational logic circuit can be expressed by some combination of NOR gates.

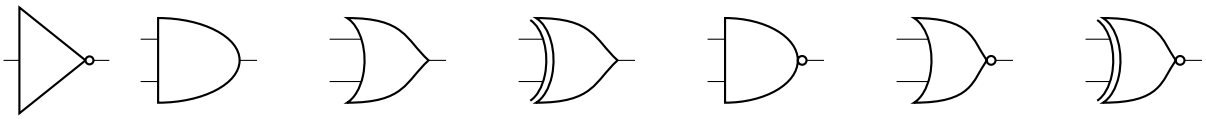
True. NOR can be used to express AND, OR, and NOT gates (You can try this yourself, starting from Q2.3!) Thus, NOR is 'functionally complete' and can be used to represent any possible Boolean expression, and thus any CL circuit.

1.5 The shortest combinational logic path between two state elements is useful in determining circuit frequency and minimum clock cycle.

False. The minimum clock cycle has to allow enough time for every CL delay to settle on an output, so the frequency is based off of the **longest** CL delay possible in any area between state elements.

2 Logic Gates

2.1 Label the following logic gates:



NOT, AND, OR, XOR, NAND, NOR, XNOR

2.2 Convert the following to simplified boolean expressions on input signals A and B. Remember that simplified boolean expressions should only have NOT, AND, and OR primitives ($\overline{A}$, $\times$, and $+$ respectively):

(a) NAND

Conceptually, NAND is the complement of AND, or $\overline{(AB)}$. We can use De Morgan's law to expand this to $\bar{A} + \bar{B}$.

Alternatively, using the canonical Sums of Products form results in $\bar{A}\bar{B} + \bar{A}B + A\bar{B}$, which simplifies to $\overline{A} + A\bar{B}$. Adding the $\bar{A}\bar{B}$ term back again (Consider: Why can we do this?), allows us to simplify to $\bar{A} + \bar{B}$.

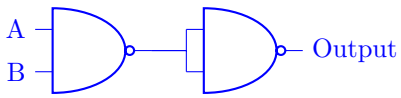
(b) XOR

$\bar{A}B + A\bar{B}$ (canonical form)

(c) XNOR

$\bar{A}\bar{B} + AB$ (canonical form)

2.3 Create an AND gate using only NAND gates.



3 Boolean Logic

In digital electronics, it is often important to get certain outputs based on your inputs, as laid out by a truth table. Truth tables map directly to Boolean expressions, and Boolean expressions map directly to logic gates. However, in order to minimize the number of logic gates needed to implement a circuit, it is often useful to simplify long Boolean expressions.

We can simplify expressions using the nine key laws of Boolean algebra:

Name	AND Form	OR form
Commutative	$AB = BA$	$A + B = B + A$
Associative	$AB(C) = A(BC)$	$A + (B + C) = (A + B) + C$
Identity	$1A = A$	$0 + A = A$
Null	$0A = 0$	$1 + A = 1$
Absorption	$A(A + B) = A$	$A + AB = A$
Distributive	$(A + B)(A + C) = A + BC$	$A(B + C) = AB + AC$
Idempotent	$A(A) = A$	$A + A = A$
Inverse	$A(\overline{A}) = 0$	$A + \overline{A} = 1$
De Morgan's	$\overline{AB} = \overline{A} + \overline{B}$	$\overline{A + B} = \overline{A}(\overline{B})$

3.1 Use multiple iterations of De Morgan's laws to prove the identity $\bar{A} + AB = \bar{A} + B$.

$$\begin{aligned} \bar{A} + AB &= \overline{\bar{A}\overline{AB}} \\ &= \overline{\bar{A}(\bar{A} + \bar{B})} \\ &= \overline{\bar{A}\bar{A} + A\bar{B}} \\ &= \overline{A\bar{B}} \\ &= \bar{A} + B \end{aligned}$$

3.2 Prove that De Morgan's law can be generalized for the complement of any number of terms.

We will prove $\overline{A + B + C + ...} = \overline{A}(\overline{B})(\overline{C})....$ Treat $B + C + ...$ as a single term, then apply De Morgan's on both terms, resulting in $\overline{A}(\overline{B + C + ...})$. Repeating this for all terms results in $\overline{A}(\overline{B})(\overline{C})....$

A similar approach works for the other identity, as both AND and OR are associative.

3.3 Simplify the following Boolean expressions:

(a) $(A + B)(A + \overline{B})C$

$$\begin{aligned} (A + B)(A + \bar{B})C &= (A + B\bar{B})C \\ &= AC \end{aligned}$$

(b) $\bar{A}\bar{B}\bar{C} + \bar{A}B\bar{C} + AB\bar{C} + A\bar{B}\bar{C} + ABC + A\bar{B}C$

$$\begin{aligned} \bar{A}\bar{C}(\bar{B} + B) + A\bar{C}(B + \bar{B}) + AC(B + \bar{B}) &= \bar{A}\bar{C} + A\bar{C} + AC \\ &= \bar{A}\bar{C} + A\bar{C} + A\bar{C} + AC \\ &= (\bar{A} + A)\bar{C} + A(\bar{C} + C) \\ &= \bar{C} + A \end{aligned}$$

Alternatively, using the identity from 2.1:

$$\begin{aligned}\bar{A}\bar{C}(\bar{B}+B)+A\bar{C}(B+\bar{B})+AC(B+\bar{B}) &= \bar{A}\bar{C}+A\bar{C}+AC\\ &= \bar{C}(\bar{A}+A)+AC\\ &= \bar{C}+AC\\ &= \bar{C}+A\end{aligned}$$

(c) $\overline{A(\bar{B}\bar{C}+BC)}$

$$\begin{aligned}\overline{A(\bar{B}\bar{C}+BC)} &= \bar{A}+\overline{\bar{B}\bar{C}+BC}\\ &= \bar{A}+\overline{\bar{B}\bar{C}}\,\overline{BC}\\ &= \bar{A}+(B+C)(\bar{B}+\bar{C})\\ &= \bar{A}+B\bar{C}+\bar{B}C\end{aligned}$$

(d) $\overline{A}(A+B)+(B+AA)(A+\overline{B})$

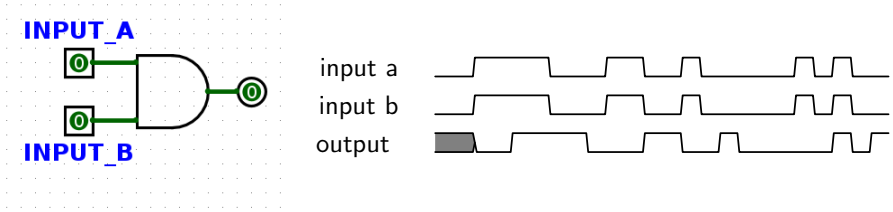
$$\begin{aligned}\overline{A}(A+B)+(B+AA)(A+\overline{B}) &= (\overline{A}A+\overline{A}B)+(B+AA)(A+\overline{B})\\ &= \overline{A}B+(B+A)(A+\overline{B})\\ &= \overline{A}B+(BA+AA+B\overline{B}+A\overline{B})\\ &= \overline{A}B+(BA+A+A\overline{B})\\ &= \overline{A}B+A\\ &= A+B\end{aligned}$$

Alternatively,

$$\begin{aligned}\overline{A}(A+B)+(B+AA)(A+\overline{B}) &= \overline{A}(A+B)+(A+B)(A+\overline{B})\\ &= (A+B)(\overline{A}+A+\overline{B})\\ &= A+B\end{aligned}$$

4 State Intro

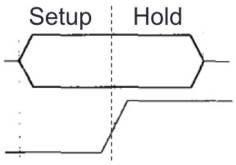
There are two basic types of circuits: combinational logic circuits and state elements. **Combinational logic** circuits simply change based on their inputs after whatever propagation delay is associated with them. For example, if an AND gate (pictured below) has an associated propagation delay of 2ps, its output will change based on its input as follows:



You should notice that the output of this AND gate *always* changes 2ps after its inputs change.

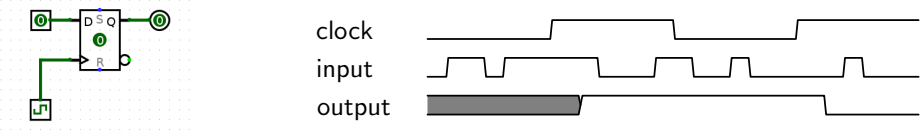
State elements, on the other hand, can *remember* their inputs even after the inputs change. State elements change value based on a clock signal. A rising edge-triggered register, for example, samples its input at the rising edge of the clock (when the clock signal goes from 0 to 1).

Like logic gates, registers also have a delay associated with them before their output will reflect the input that was sampled. This is called the **clk-to-q** delay. ("Q" often indicates output). This is the time between the rising edge of the clock signal and the time the register's output reflects the input change.



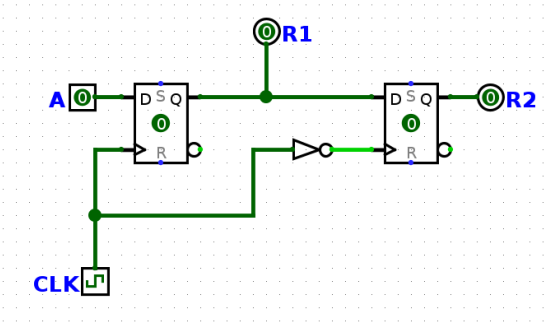
The input to the register samples has to be stable for a certain amount of time around the rising edge of the clock for the input to be sampled accurately. The amount of time before the rising edge the input must be stable is called the **setup** time, and the time after the rising edge the input must be stable is called the **hold** time. Hold time is generally included in clk-to-q delay, so clk-to-q time will usually be greater than or equal to hold time. Logically, the fact that clk-to-q $\geq$ hold time makes sense since it only takes clk-to-q seconds to copy the value over, so there's no need to have the value fed into the register for any longer.

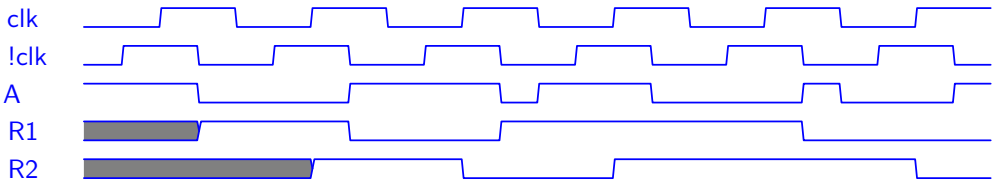
For the following register circuit, assume **setup** of 2.5ps, **hold** time of 1.5ps, and a **clk-to-q** time of 1.5ps. The clock signal has a period of 13ps.

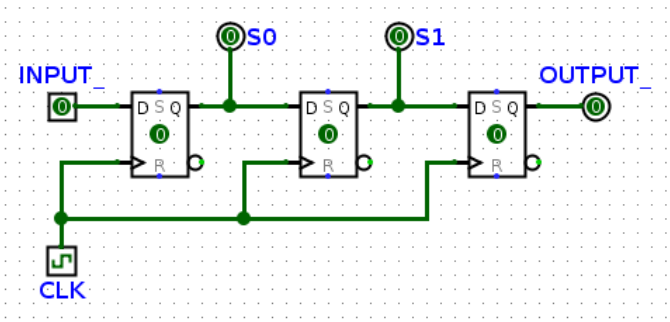


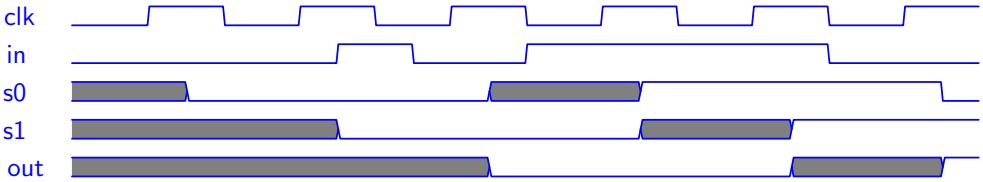
You'll notice that the value of the output in the diagram above doesn't change immediately after the rising edge of the clock. Until enough time has passed for the output to reflect the input, the value held by the output is garbage; this is represented by the shaded gray part of the output graph. Clock cycle time must be small enough that inputs to registers don't change within the hold time and large enough to account for clk-to-q times, setup times, and combinational logic delays.

4.1 For the following 2 circuits, fill out the timing diagram. The clock period (rising edge to rising edge) is 8ps. For every register, clk-to-q delay is 2ps, setup time is 4ps, and hold time is 2ps. NOT gates have a 2ps propagation delay, which is already accounted for in the !clk signal given.

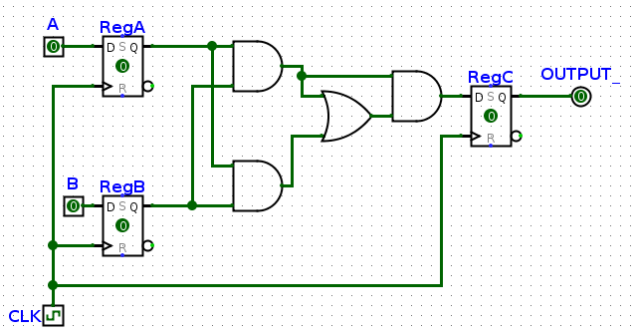








4.2 In the circuit below, RegA and RegB have setup, hold, and clk-to-q times of 4ns, all logic gates have a delay of 5ns, and RegC has a setup time of 6ns. What is the maximum allowable hold time for RegC? What is the minimum acceptable clock cycle time for this circuit, and clock frequency does it correspond to?



The maximum allowable hold time for RegC is how long it takes for RegC's input to change, so (clk-to-q of A or B) + shortest CL time = 4 + (5 + 5) = 14 ns.

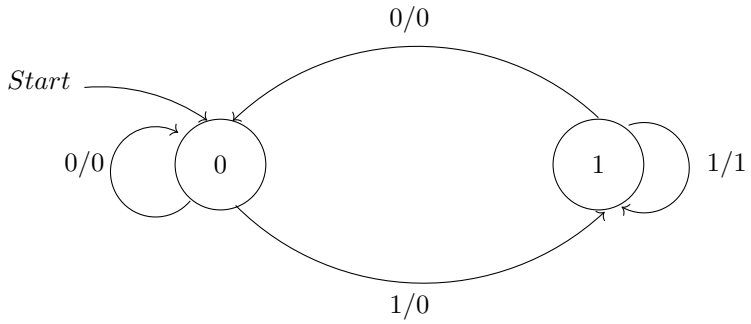
The minimum acceptable clock cycle time is clk-to-q + longest CL time + setup time = 4 + (5 + 5 + 5) + 6 = 25 ns.

25 ns corresponds to a clock frequency of $(1/(25 * 10^{-9}))s^{-1} = 40MHz$

5 Finite State Machines

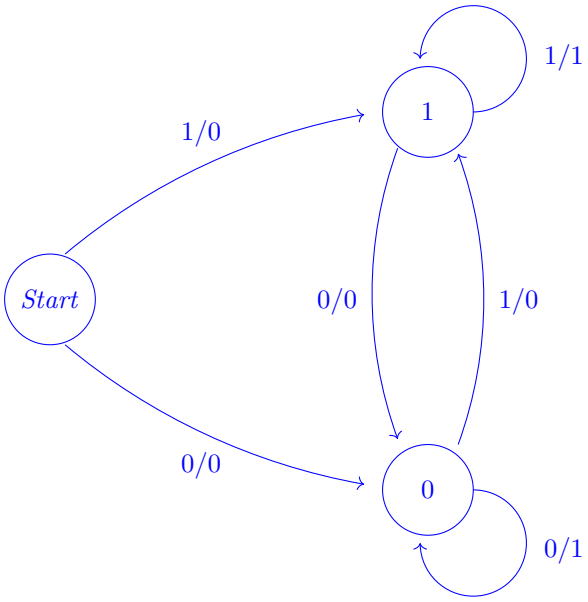
Automatons are machines that receive input and use various states to produce output. A finite state machine is a type of simple automaton where the next state and output depend only on the current state and input. Each state is represented by a circle, and every proper finite state machine has a starting state, signified either with the label "Start" or a single arrow leading into it. Each transition between states is labeled [input]/[output].

5.1 What pattern in a bitstring does the FSM below detect? What would it output for the input bitstring "011001001110"?

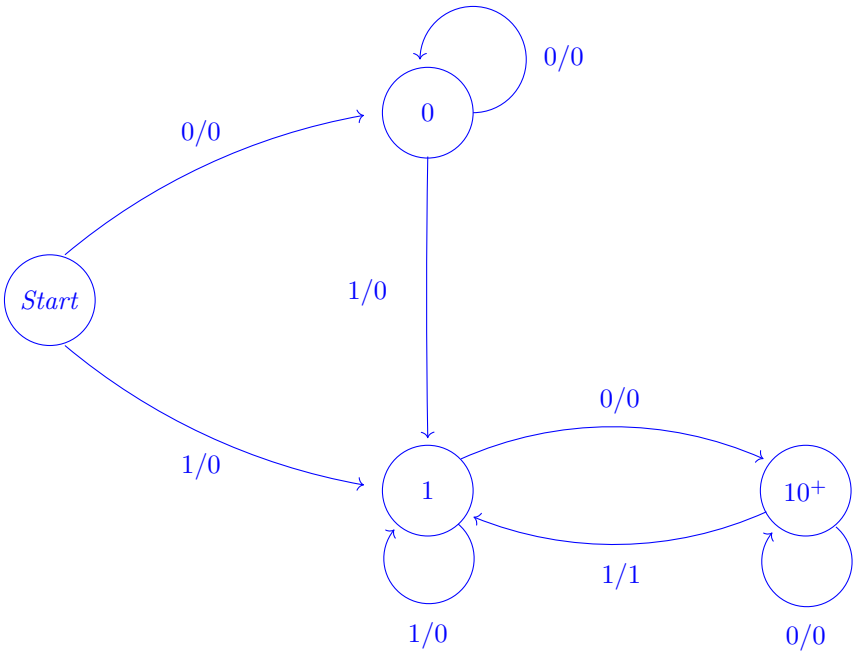


The FSM outputs a 1 if it detects the pattern "11".
The FSM would output "001000000110"

5.2 Fill in the following FSM for outputting a 1 whenever we have two repeating bits as the most recent bits, and a 0 otherwise. You may not need all states.



5.3 Draw an FSM that will output a 1 if it recognizes the regex pattern {10+1}. (That is, if the input forms a pattern of a 1, followed by one or more 0s, followed by a 1.)



CS 61C Fall 2022 — RISC-V Single Cycle Datapath — Discussion 7

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 The single cycle datapath makes use of all hardware units for each instruction.

1.2 It is possible to execute the stages of the single cycle datapath in parallel to speed up execution of a single instruction.

1.3 If the delay of reading from IMEM is reduced, then any (non-empty) program using the single cycle datapath will speed up.

1.4 The control signals used throughout all datapath stages to guide a correct 'execution line' all come from decoding an instruction's unique binary encoding only.

1.5 Storing instructions and loading instructions are the only instructions that require input/output from DMEM.

1.6 It is possible to use both the output of the immediate generator and the value in register rs2.

2 Single-Cycle CPU

2.1 For this worksheet, we will be working with the single-cycle CPU datapath provided the last page.

(a) Explain what happens in each datapath stage, and which hardware units in the datapath are used.

IF Instruction Fetch

ID Instruction Decode

EX Execute

MEM Memory

WB Writeback

(b) On the datapath, fill in each **round** box with the name of the datapath component, and each **square** box with the name of the control signal.

(c) List all possible values that each control signal may take on for the single cycle datapath, then briefly describe what each value means for each signal.

Signal Name	Values	Signal Name	Values
PCSel		RegWEn	
ImmSel		BrEq	
BrLt		ALUSel	
MemRW		WBSel	

2.2 Fill out the following table with the control signals for each instruction based on the datapath on the last page. If the value of the signal does not affect the execution of an instruction, use the * (don't care) symbol to indicate this. If the value of the signal **does** affect the execution, but can be different depending on the program, list all possible values (for example, for a signal with possible values of 0 and 1, write 0/1).

	BrEq	BrLT	PCSel	ImmSel	BrUn	ASel	BSel	ALUSel	MemRW	RegWEn	WBSel
add											
ori											
lw											
sw											
beq											
jal											
blt											

2.3 Clocking Methodology

- A **state element** is an element connected to the clock (denoted by a triangle at the bottom). The **input signal** to each state element must stabilize before each **rising edge**.
- The **critical path** is the longest delay path between state elements in the circuit. The circuit cannot be clocked faster than this, since anything faster would mean that the correct value is not guaranteed to reach the state element in the alloted time. If we place registers in the critical path, we can shorten the period by **reducing the amount of logic between registers**.

For this exercise, assume the delay for each stage in the datapath is as follows:

IF: 200 ps ID: 100 ps EX: 200 ps MEM: 200 ps WB: 100 ps

(a) Mark the stages of the datapath that the following instructions use and calculate the total time needed to execute the instruction.

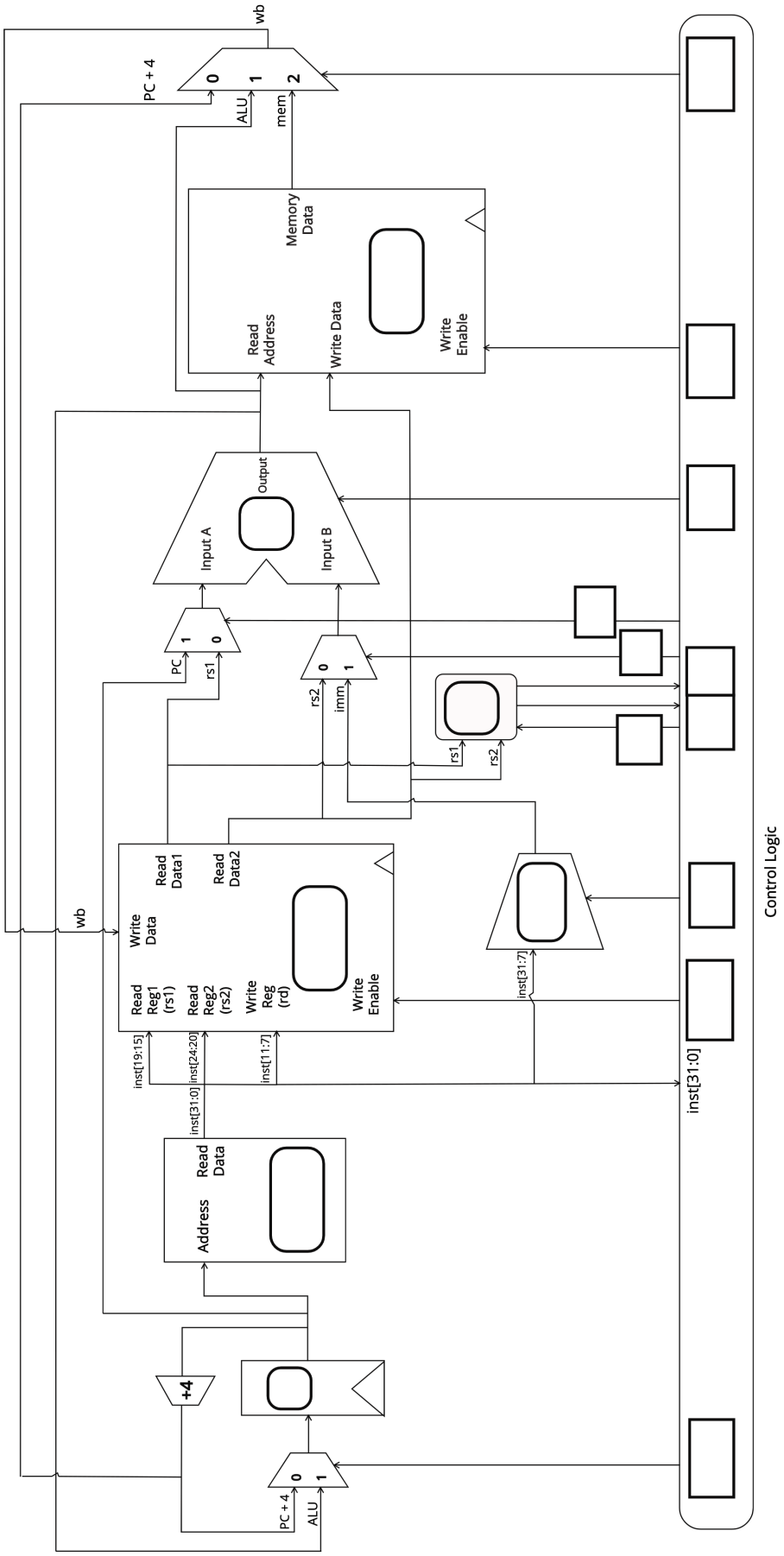
	IF	ID	EX	MEM	WB	Total Time
add						
ori						
lw						
sw						
beq						
jal						
bltu						

(b) Which instruction(s) exercise the critical path?

(c) What is the fastest you could clock this single cycle datapath?

(d) Why is the single cycle datapath inefficient?

(e) How can you improve its performance?


wb
PC + 4
ALU
mem
0
1
2
Memory Data
Read Address
Write Data
Write Enable
Output
Input A
Input B
PC
rs1
rs2
imm
Read Data1
Read Data2
Write Data
Read Reg1 (rs1)
Read Reg2 (rs2)
Write Reg (rd)
Write Enable
inst[19:15]
inst[24:20]
inst[11:7]
inst[31:0]
inst[31:7]
Address
Read Data
+4
Control Logic

CS 61C Fall 2022 — RISC-V Single Cycle Datapath — Discussion 7

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 The single cycle datapath makes use of all hardware units for each instruction.

False. All units are active in each cycle, but their output may be ignored (gated) by control signals.

1.2 It is possible to execute the stages of the single cycle datapath in parallel to speed up execution of a single instruction.

False. Each stage depends on the value produced by the stage before it (e.g., instruction decode depends on the instruction fetched).

1.3 If the delay of reading from IMEM is reduced, then any (non-empty) program using the single cycle datapath will speed up.

True. Since every instruction must read from IMEM during the instruction fetch stage, making the IMEM faster will speed up every single instruction.

1.4 The control signals used throughout all datapath stages to guide a correct 'execution line' all come from decoding an instruction's unique binary encoding only.

False. PCSel is used to determine what instruction will be executed next, which is not immediately known for a branch instruction. This is discovered in the Branch Comparator after the register values are retrieved from the RegFile. The comparator then produces the BrEq (branch equal) and BrLt (branch less than) flags, which are used in the control logic with the instruction encoding to produce the PCSel signal.

1.5 Storing instructions and loading instructions are the only instructions that require input/output from DMEM.

True. For all other instructions, we don't need to read the data that is read out from DMEM, and thus don't need to wait for the output of the MEM stage.

1.6 It is possible to use both the output of the immediate generator and the value in register rs2.

False. You may only use *either* the immediate generator or the value in register rs2. Notice in our datapath, there is a mux with a signal (BSel) that decides whether we use the output of the immediate generator or the value in rs2.

2 Single-Cycle CPU

2.1 For this worksheet, we will be working with the single-cycle CPU datapath provided the last page.

(a) Explain what happens in each datapath stage, and which hardware units in the datapath are used.

IF Instruction Fetch

Send address to the instruction memory (IMEM), and read IMEM at that address.
Hardware units: PC register, +4 adder, PCSel mux, IMEM

ID Instruction Decode

Generate control signals from the instruction bits, generate the immediate, and read registers from the RegFile.
Hardware units: RegFile, ImmGen

EX Execute

Perform ALU operations, and do branch comparison.
Hardware units: ASel mux, BSel mux, branch comparator, ALU

MEM Memory

Read from or write to the data memory (DMEM).
Hardware units: DMEM

WB Writeback

Write back either PC + 4, the result of the ALU operation, or data from memory to the RegFile.
Hardware units: WBSel mux, RegFile

(b) On the datapath, fill in each **round** box with the name of the datapath component, and each **square** box with the name of the control signal.

See last page.

(c) List all possible values that each control signal may take on for the single cycle datapath, then briefly describe what each value means for each signal.

Signal Name	Values	Signal Name	Values
PCSel		RegWEn	
ImmSel		BrEq	
BrLt		ALUSel	
MemRW		WBSel	

PCSel: 0: OldPC + 4 is next PC; 1: ALU value (for branches, jumps, etc...)
RegWEn: 0: WB value is not written to register; 1: WB is written
ImmSel: 0-4 used for I, B, S, J, U type immediates; 5-7 unused
BrEq: 0 when inputs not equal; 1 when inputs equal
BrLt: 0 when `rs1` is not less than `rs2`; 1 when it is less than
ALUSel: note, this is using the same design reference 61C does; may differ based on CPU design; you do **not** have to memorise all of these!

- 0: add (`rd = rs1 + rs2`)
- 1: sll (`rd = rs1 << rs2`)
- 2: slt (`rd = (rs1 < rs2 (signed)) ? 1 : 0`)
- 3: unused
- 4: xor (`rd = rs1 ^ rs2`)
- 5: srl (`rd = (unsigned) A >> B`)
- 6: or (`rd = rs1 | rs2`)
- 7: and (`rd = rs1 & rs2`)
- 8: mul (`rd = (signed) (rs1 * rs2)[31:0]`)
- 9: mulh (`rd = (signed) (rs1 * rs2) [63:32]`)
- 10: unused
- 11: mulhu (`rd = (unsigned) (rs1 * rs2) [63:32]`)
- 12: sub (`rd = rs1 - rs2`)
- 13: sra (`rd = (signed) rs1 >> rs2`)
- 14: unused
- 15: bsel (`rd = rs2`)

MemRW: 0 for all non-write-to-memory operations; 1 to enable writing to main memory
WBSel: 0 for PC + 4; 1 for ALU output; 2 for main memory read output; 3 unused

2.2 Fill out the following table with the control signals for each instruction based on the datapath on the last page. If the value of the signal does not affect the execution of an instruction, use the * (don't care) symbol to indicate this. If the value of the signal **does** affect the execution, but can be different depending on the program, list all possible values (for example, for a signal with possible values of 0 and 1, write 0/1).

	BrEq	BrLT	PCSel	ImmSel	BrUn	ASel	BSel	ALUSel	MemRW	RegWEn	WBSel
add	*	*	0 (PC + 4)	*	*	0 (Reg)	0 (Reg)	add	0	1	1 (ALU)
ori	*	*	0	I	*	0 (Reg)	1 (Imm)	or	0	1	1 (ALU)
lw	*	*	0	I	*	0 (Reg)	1 (Imm)	add	0	1	2 (MEM)
sw	*	*	0	S	*	0 (Reg)	1 (Imm)	add	1	0	*
beq	0/1	*	0/1	B	*	1 (PC)	1 (Imm)	add	0	0	*
jal	*	*	1 (ALU)	J	*	1 (PC)	1 (Imm)	add	0	1	0 (PC + 4)
blt	*	0/1	0/1	B	0	1 (PC)	1 (Imm)	add	0	0	*

2.3 Clocking Methodology

- A **state element** is an element connected to the clock (denoted by a triangle at the bottom). The **input signal** to each state element must stabilize before each **rising edge**.
- The **critical path** is the longest delay path between state elements in the circuit. The circuit cannot be clocked faster than this, since anything faster would mean that the correct value is not guaranteed to reach the state element in the alloted time. If we place registers in the critical path, we can shorten the period by **reducing the amount of logic between registers**.

For this exercise, assume the delay for each stage in the datapath is as follows:

IF: 200 ps ID: 100 ps EX: 200 ps MEM: 200 ps WB: 100 ps

(a) Mark the stages of the datapath that the following instructions use and calculate the total time needed to execute the instruction.

	IF	ID	EX	MEM	WB	Total Time
add	X	X	X		X	600 ps
ori	X	X	X		X	600 ps
lw	X	X	X	X	X	800 ps
sw	X	X	X	X		700 ps
beq	X	X	X			500 ps
jal	X	X	X		X	600 ps
bltu	X	X	X			500 ps

(b) Which instruction(s) exercise the critical path?

Load word (lw), which uses all 5 stages.

(c) What is the fastest you could clock this single cycle datapath?

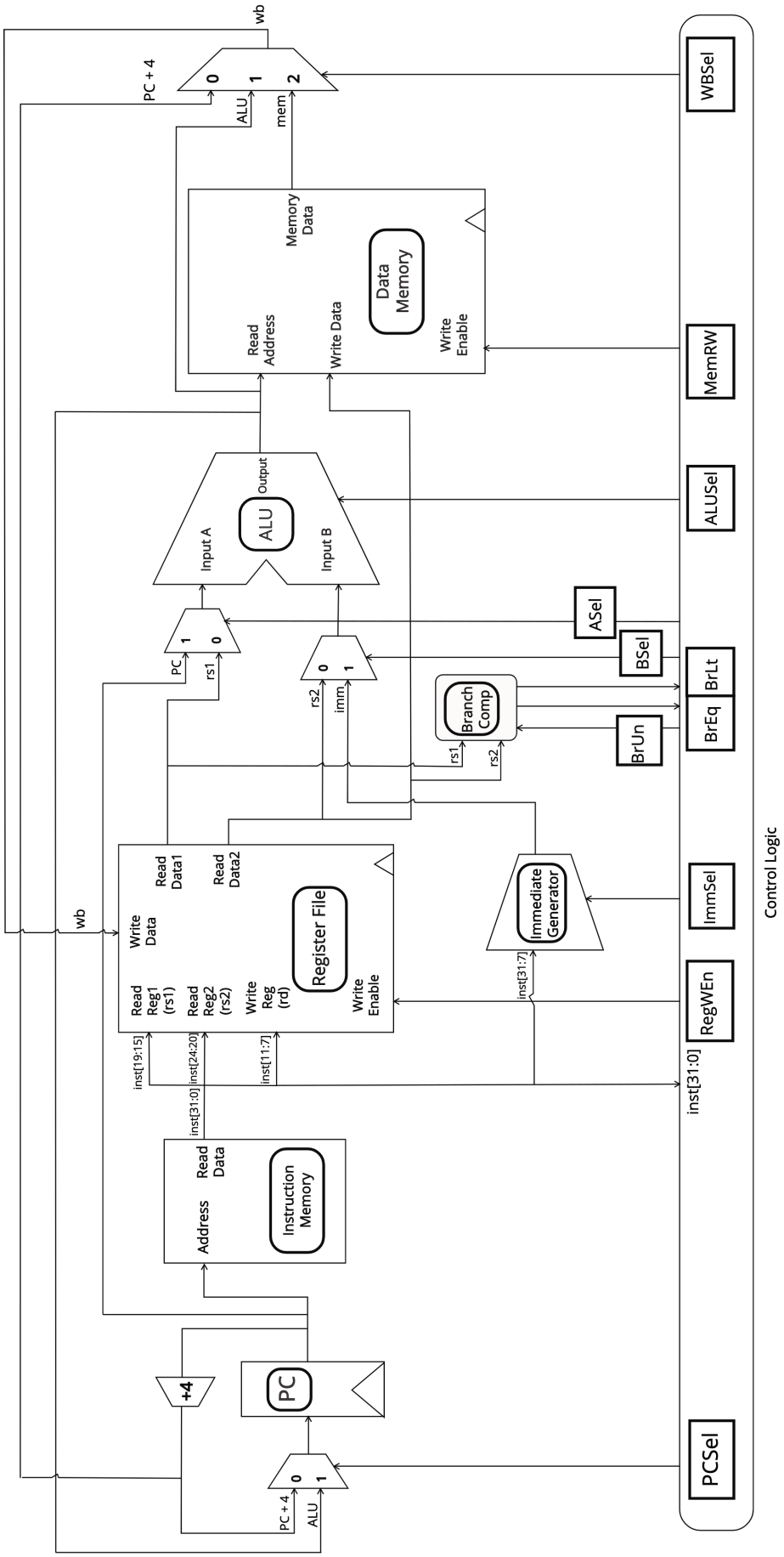
$$\frac{1}{800} \text{ picoseconds} = \frac{1}{800 * 10^{-12}} \text{ seconds} = 1,250,000,000 s^{-1} = 1.25 GHz$$

(d) Why is the single cycle datapath inefficient?

At any given time, most of the parts of the single cycle datapath are sitting unused. Also, even though not every instruction exercises the critical path, the datapath can only be clocked as fast as the slowest instruction.

(e) How can you improve its performance?

Performance can be improved with pipelining, or putting registers between stages so that the amount of combinational logic between registers is reduced, allowing for a faster clock time. We'll talk more about this next week!


wb
PC + 4
ALU
mem
0
1
2
Memory Data
Data Memory
Read Address
Write Data
Write Enable
Input A
ALU
Output
Input B
PC
rs1
rs2
imm
Branch Comp
rs1
rs2
Read Data1
Read Data2
Write Data
Register File
Read Reg1 (rs1)
Read Reg2 (rs2)
Write Reg (rd)
Write Enable
Immediate Generator
inst[31:7]
inst[19:15]
inst[24:20]
inst[11:7]
inst[31:0]
Address
Read Data
Instruction Memory
+4
PC
PC + 4
ALU
WBSel
MemRW
ALUSel
ASel
BSel
BrLt
BrEq
BrUn
ImmSel
RegWEn
PCSel
inst[31:0]
Control Logic

CS 61C Fall 2022 — RISC-V Pipelining and Hazards

Discussion 8

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 By pipelining the CPU datapath, each single instruction will execute faster (latency is reduced), resulting in a speed-up in performance.

1.2 A pipelined CPU datapath results in instructions being executed with higher throughput (than the single-cycle CPU).

1.3 Through adding additional hardware, we can implement two 'read' ports as well as a 'write' port to the RegFile (where registers can be accessed). This solves the hazard of two instructions reading and writing to the same register simultaneously.

1.4 All data hazards can be resolved with forwarding.

1.5 As stalling reduces performance significantly, we generally prefer other solutions to fixing pipeline hazards, even at the cost of complexity or hardware. In a modern-day pipelined CPU, are there still use-cases for stalling to resolve potential hazards? If so, describe a program that would.

2 Pipelining Registers

In order to pipeline, we separate the datapath into 5 discrete stages, each completing a different function and accessing different resources on the way to executing an entire instruction.

In the **IF** stage, we use the Program Counter to access our instruction as it is stored in IMEM. Then, we separate the distinct parts we need from the instruction bits in the **ID** stage and generate our immediate, the register values from the RegFile, and other control signals. Afterwards, using these values and signals, we complete the necessary ALU operations in the **EX** stage. Next, anything we do in regards with DMEM (not to be confused with RegFile or IMEM) is done in the **MEM** stage, before we hit the **WB** stage, where we write the computed value that we want back into the return register in the RegFile.

These 5 stages, divided by registers as shown in the figure, allow the datapath to provide a pipeline for multiple instructions to operate at the same time, each accessing different resources. A pipelined datapath is provided for you on the last page. Use it to answer the following questions.

2.1 What is the purpose of the new registers?

2.2 Looking at the way PC is passed through the datapath, there are two places where +4 is added to the PC, once in the **IF** and **MEM** stage. Why do we add +4 to the PC again in the memory stage?

2.3 Why do we need to save the instruction in a register multiple times?

3 Performance Analysis

Register clk-to-q 30 ps	**Branch comp.** 75 ps	**Memory write** 200 ps
Register setup 20 ps	**ALU** 200 ps	**RegFile read** 100 ps
Register hold 10 ps	**Imm. Gen.** 15 ps	
Mux 25 ps	**Memory read** 250 ps	**RegFile setup** 20 ps

Given above are sample delays for each of the datapath components and register timings. In the questions below, use these in conjunction with the pipelined datapath on the last page to answer them.

3.1 What would be the fastest possible clock time for a single cycle datapath? Recall from last week's discussion that one instruction which exercises the critical path is `lw`.
(HINT: $t_{\text{clk-cycle}} \geq t_{\text{clk-to-q}} + t_{\text{longest-combinational-path}} + t_{\text{setup}}$)

3.2 What is the fastest possible clock time for a pipelined datapath?

3.3 What is the speedup from the single cycle datapath to the pipelined datapath? Why is the speedup less than 5×?

4 Hazards

One of the costs of pipelining is that it introduces pipeline hazards. Hazards, generally, are issues with something in the CPU's instruction pipeline that could cause the next instruction to execute incorrectly.

The 5-stage pipelined CPU introduces three types: structural hazards (hardware not sufficient), data hazards (using wrong values in computation), and control hazards (executing the wrong instruction).

Structural Hazards

Structural hazards occur when more than one instruction needs to use the same datapath resource at the same time. In the standard 5-stage pipeline, **there aren't structural hazards**, unless there are active changes to the pipeline. The structural hazards that could exist are prevented by RV32I's hardware requirements.

There are two main causes of structural hazards:

- **Register File:** The register file is accessed both during ID, when it is read to decode the instruction, and the corresponding register values; and during WB, when it is written to in the `rd` register. If the RegFile only had one port, then it wouldn't work since we have one instruction being decoded and another writing back.
  - We resolve this by having separate read and write ports. However, this only works if the read/written registers are different.
  - To account for reads and writes to the same register, processors usually write to the register during the first half of the clock cycle, and read from it during in the second half. This is an implementation of the idea of **double pumping**, which is when data is transferred along data buses at double the rate, by utilising both the rising and falling clock edges in a clock cycle.
- **Main Memory:** Main memory is accessed for both instructions and data. If memory could only support one read/write at a time, then instruction A going through IF and attempting to fetch an instruction from memory cannot happen at the same time as instruction B attempting to read (or write) to data portions of memory.
  - Having a separate instruction memory (abbreviated IMEM) and data memory (abbreviated DMEM) solves this hazard.

Something to remember about structural hazards is that they can always be resolved by adding more hardware.

Data Hazards

Data hazards are caused by data dependencies between instructions. In CS 61C, where we always assume that instructions go through the processor in order, we see data hazards when an instruction **reads** a register before a previous instruction has finished **writing** to that register.

There are two types of data hazards:

- **EX-ID:** this hazard exists because the output from the execute stage is not written back to the RegFile until the writeback stage, yet can be requested by the subsequent instruction in the decode stage.
- **MEM-ID:** this hazard exists because the output from the memory access stage is not written back to the RegFile until the writeback stage, but can be requested from the decode stage, just as in EX-ID.

Solving Data Hazards

For all questions, assume **no branch prediction or double-pumping**.

Forwarding

Most data hazards can be resolved by forwarding, which is when the result of the EX or MEM stage is sent to the EX stage for a following instruction to use.

4.1 Look for data hazards in the code below, and figure out how forwarding could be used to solve them.

Instruction	C1	C2	C3	C4	C5	C6	C7
1. `addi t0, a0, -1`	IF	ID	EX	MEM	WB		
2. `and s2, t0, a0`		IF	ID	EX	MEM	WB	
3. `sltiu a0, t0, 5`			IF	ID	EX	MEM	WB

4.2 Imagine you are a hardware designer working on a CPU's forwarding control logic. How many instructions after the addi instruction could be affected by data hazards created by this addi instruction?

Stalls

4.3 Look for data hazards in the code below. One of them cannot be solved with forwarding—why? What can we do to solve this hazard?

Instruction	C1	C2	C3	C4	C5	C6	C7	C8
1. `addi s0, s0, 1`	IF	ID	EX	MEM	WB			
2. `addi t0, t0, 4`		IF	ID	EX	MEM	WB		
3. `lw t1, 0(t0)`			IF	ID	EX	MEM	WB	
4. `add t2, t1, x0`				IF	ID	EX	MEM	WB

4.4 Say you are the compiler and can re-order instructions to minimize data hazards while guaranteeing the same output. How can you fix the code above?

Detecting Data Hazards

Say we have the $rs1$, $rs2$, *RegWEn*, and rd signals for two instructions (instruction n and instruction $n+1$) and we wish to determine if a data hazard exists across the instructions. We can simply check to see if the rd for instruction n matches either $rs1$ or $rs2$ of instruction $n+1$, indicating that such a hazard exists (why does this make sense?).

We could then use our hazard detection to determine which forwarding paths/number of stalls (if any) are necessary to take to ensure proper instruction execution. In pseudo-code, part of this could look something like the following:

```
if (rs1(n + 1) == rd(n) && RegWen(n) == 1) {
    set ASel for (n + 1) to forward ALU output from n
}
if (rs2(n + 1) == rd(n) && RegWen(n) == 1) {
    set BSel for (n + 1) to forward ALU output from n
}
```

Control Hazards

Control hazards are caused by **jump and branch instructions**, because for all jumps and some branches, the next PC is not PC + 4, but the result of the ALU available after the EX stage. We could stall the pipeline for control hazards, but this decreases performance.

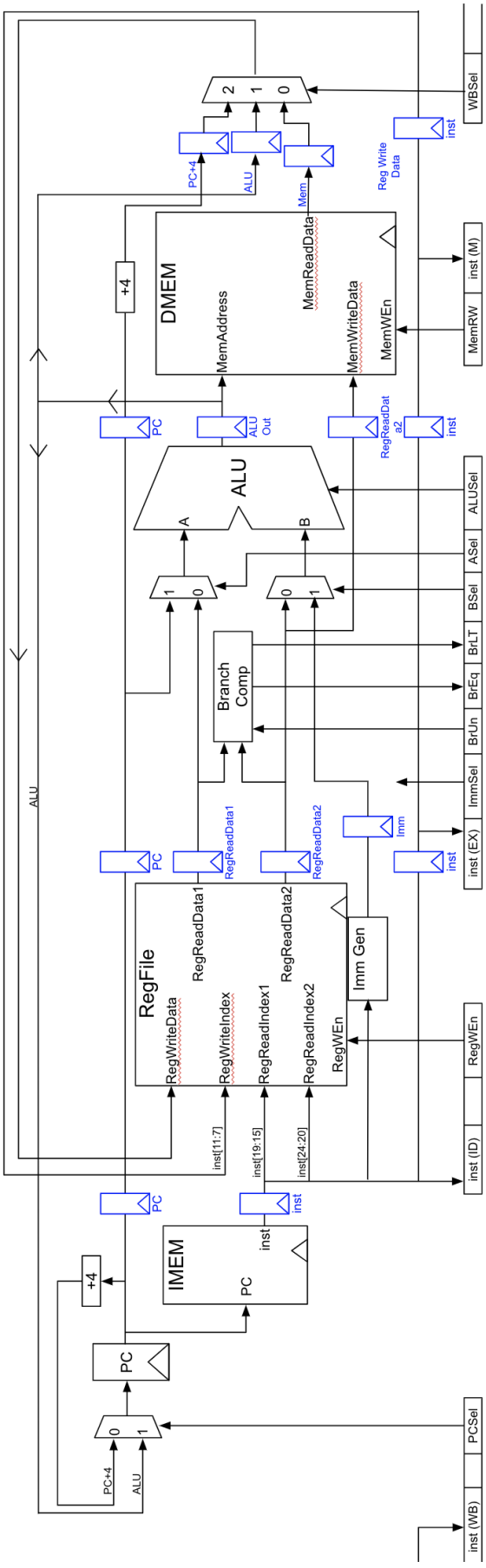
4.5 Besides stalling, what can we do to resolve control hazards?

Extra for Experience

4.6 Given the RISC-V code above and a pipelined CPU with no forwarding, how many hazards would there be? What types are each hazard? Consider all possible hazards between all instructions.

How many stalls would there need to be in order to fix the data hazard(s), if we use double-pumping? What about the control hazard(s), if we use branch prediction?

Instruction	C1	C2	C3	C4	C5	C6	C7	C8	C9
1. `sub t1, s0, s1`	IF	ID	EX	MEM	WB				
2. `or s0, t0, t1`		IF	ID	EX	MEM	WB			
3. `sw s1, 100(s0)`			IF	ID	EX	MEM	WB		
4. `bgeu s0, s2, loop`				IF	ID	EX	MEM	WB	
5. `add t2, x0, x0`					IF	ID	EX	MEM	WB


PCSel
inst (WB)
0
1
PC+4
ALU
PC
+4
IMEM
inst
PC
PC
inst
inst[11:7]
inst[19:15]
inst[24:20]
RegFile
RegWriteData
RegWriteIndex
RegReadIndex1
RegReadIndex2
RegWEn
RegReadData1
RegReadData2
Imm Gen
RegWEn
inst (ID)
PC
PC
RegReadData1
RegReadData2
Imm
inst
inst (EX)
ALU
Branch Comp
ImmSel
BrUn
BrEq
BrLT
BSel
ASel
ALUSel
A
B
ALU
PC
ALU Out
RegReadDat a2
inst
DMEM
MemAddress
MemReadData
MemWriteData
MemWEn
MemRW
inst (M)
+4
PC+4
ALU
Mem
Reg Write Data
inst
WBSel
2
1
0

CS 61C Fall 2022 — RISC-V Pipelining and Hazards

Discussion 8

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 By pipelining the CPU datapath, each single instruction will execute faster (latency is reduced), resulting in a speed-up in performance.

False. Because we implement registers between each stage of the datapath, the time it takes for an instruction to finish execution through the 5 stages will be longer than the single-cycle datapath we were first introduced with. A single instruction will take multiple clock cycles to get through all the stages, with the clock cycle based on the stage with the longest timing.

1.2 A pipelined CPU datapath results in instructions being executed with higher throughput (than the single-cycle CPU).

True. Recall that throughput is the number of instructions processed per unit time. Pipelining results in a higher throughput because more instructions are run at once, which utilizes more parts of the datapath simultaneously.

1.3 Through adding additional hardware, we can implement two 'read' ports as well as a 'write' port to the RegFile (where registers can be accessed). This solves the hazard of two instructions reading and writing to the same register simultaneously.

False. The addition of independent ports to the RegFile allows for multiple instructions to access the RegFile at the same time (such as one instruction reading values of two operands, while another instruction is writing to a return register). However, this does not work if both instructions are reading and writing to the same register. Some solutions to this data hazard could be to stall the latter instruction by 1 cycle or to forward the read value from a previous instruction, bypassing the RegFile completely.

1.4 All data hazards can be resolved with forwarding.

False. Hazards following `lw` cannot be fully resolved with forwarding because the output is not known until after the MEM stage, making a stall necessary.

1.5 As stalling reduces performance significantly, we generally prefer other solutions to fixing pipeline hazards, even at the cost of complexity or hardware. In a modern-day pipelined CPU, are there still use-cases for stalling to resolve potential hazards? If so, describe a program that would.

True. Say we have the RISC-V program where a0 is a pointer to an array of integers, and we want to load a1 with the first element * 2:

```
lw t1 0(a0)
add t2 t1 t1
mv a1 t2
```

In this program, there are no other instructions to move into the load delay slot, so we are forced to `nop` the next instruction and repeat it afterwards, essentially stalling for one cycle. While we do have many tools and alternative solutions to lessen possible performance loss, in some cases it is unavoidable.

2 Pipelining Registers

In order to pipeline, we separate the datapath into 5 discrete stages, each completing a different function and accessing different resources on the way to executing an entire instruction.

In the **IF** stage, we use the Program Counter to access our instruction as it is stored in IMEM. Then, we separate the distinct parts we need from the instruction bits in the **ID** stage and generate our immediate, the register values from the RegFile, and other control signals. Afterwards, using these values and signals, we complete the necessary ALU operations in the **EX** stage. Next, anything we do in regards with DMEM (not to be confused with RegFile or IMEM) is done in the **MEM** stage, before we hit the **WB** stage, where we write the computed value that we want back into the return register in the RegFile.

These 5 stages, divided by registers as shown in the figure, allow the datapath to provide a pipeline for multiple instructions to operate at the same time, each accessing different resources. A pipelined datapath is provided for you on the last page. Use it to answer the following questions.

2.1 What is the purpose of the new registers?

When we pipeline the datapath, the values from each stage need to be passed on at each clock cycle. Each stage in the pipeline only operates on a small set of values, but those values need to be correct with respect to the instruction that is currently being processed. Say we use load word (lw) as an example: if it is in the EX stage, then the EX stage should look like a snapshot of the single-cycle datapath. The values on the rs1, rs2, immediate, and PC values should be as if lw was the only instruction in the entire path. This also includes the control logic: the instruction is passed in at each stage, the appropriate control signals are generated for the stage of interest, and that stage can execute properly.

2.2 Looking at the way PC is passed through the datapath, there are two places where +4 is added to the PC, once in the **IF** and **MEM** stage. Why do we add +4 to the PC again in the memory stage?

We add +4 to the PC again in the memory stage so we don't need to pass both PC and PC+4 along the whole pipeline. This would use more registers, adding unnecessary hardware. We also can't just pass only PC+4 through the pipeline, as we need the original PC value in operands like `auipc`.

2.3 Why do we need to save the instruction in a register multiple times?

We need to save the instruction in a register multiple times because each pipeline stage needs to receive the right control signals for the instruction currently in that stage.

3 Performance Analysis

Register clk-to-q 30 ps	**Branch comp.** 75 ps	**Memory write** 200 ps
Register setup 20 ps	**ALU** 200 ps	**RegFile read** 100 ps
Register hold 10 ps	**Imm. Gen.** 15 ps	
Mux 25 ps	**Memory read** 250 ps	**RegFile setup** 20 ps

Given above are sample delays for each of the datapath components and register timings. In the questions below, use these in conjunction with the pipelined datapath on the last page to answer them.

3.1 What would be the fastest possible clock time for a single cycle datapath? Recall from last week's discussion that one instruction which exercises the critical path is `lw`.
(HINT: $t_{\text{clk-cycle}} \geq t_{\text{clk-to-q}} + t_{\text{longest-combinational-path}} + t_{\text{setup}}$)

$$\begin{aligned} t_{\text{clk}} &\geq t_{\text{PC clk-to-q}} + t_{\text{IMEM read}} + t_{\text{RF read}} + t_{\text{mux}} + t_{\text{ALU}} + t_{\text{DMEM read}} + t_{\text{mux}} + t_{\text{RF setup}} \\ &\geq 30 + 250 + 100 + 25 + 200 + 250 + 25 + 20 \\ &\geq 900 \text{ ps} \end{aligned}$$

Note that the delay in the immediate generator as well as the branch comparator are omitted because the immediate generator and branch comparison is done in parallel with the RegFile read and ALU computation respectively, the latter two taking much longer time.

3.2 What is the fastest possible clock time for a pipelined datapath?

$$\begin{aligned} \textbf{IF}: &\ t_{\text{PC clk-to-q}} + t_{\text{IMEM read}} + t_{\text{Reg setup}} = 30 + 250 + 20 = 300 \text{ ps} \\ \textbf{ID}: &\ t_{\text{Reg clk-to-q}} + t_{\text{RF read}} + t_{\text{Reg setup}} = 30 + 100 + 20 = 150 \text{ ps} \\ \textbf{EX}: &\ t_{\text{Reg clk-to-q}} + t_{\text{mux}} + t_{\text{ALU}} + t_{\text{Reg setup}} = 30 + 25 + 200 + 20 = 275 \text{ ps} \\ \textbf{MEM}: &\ t_{\text{Reg clk-to-q}} + t_{\text{DMEM read}} + t_{\text{Reg setup}} = 30 + 250 + 20 = 300 \text{ ps} \\ \textbf{WB}: &\ t_{\text{Reg clk-to-q}} + t_{\text{mux}} + t_{\text{RF setup}} = 30 + 25 + 20 = 75 \text{ ps} \end{aligned}$$

$$t_{\text{clk}} \geq \max(\textbf{IF}, \textbf{ID}, \textbf{EX}, \textbf{MEM}, \textbf{WB}) = 300 \text{ ps}$$

Again, the immediate generator and branch comparator delays are overshadowed by the longer delays of RegFile read and ALU.

3.3 What is the speedup from the single cycle datapath to the pipelined datapath? Why is the speedup less than 5×?

$\frac{900\ ps}{300\ ps}$, or a 3 times speedup. The speedup is less than 5 because of (1) the necessity of adding pipeline registers, which have clk-to-q and setup times, and (2) the need to set the clock to the maximum of the five stages, which take different amounts of time.

Note: Due to hazards, which require additional logic to resolve, the actual speedup would likely be even less than 3 times.

4 Hazards

One of the costs of pipelining is that it introduces pipeline hazards. Hazards, generally, are issues with something in the CPU's instruction pipeline that could cause the next instruction to execute incorrectly.

The 5-stage pipelined CPU introduces three types: structural hazards (hardware not sufficient), data hazards (using wrong values in computation), and control hazards (executing the wrong instruction).

Structural Hazards

Structural hazards occur when more than one instruction needs to use the same datapath resource at the same time. In the standard 5-stage pipeline, **there aren't structural hazards**, unless there are active changes to the pipeline. The structural hazards that could exist are prevented by RV32I's hardware requirements.

There are two main causes of structural hazards:

- **Register File:** The register file is accessed both during ID, when it is read to decode the instruction, and the corresponding register values; and during WB, when it is written to in the `rd` register. If the RegFile only had one port, then it wouldn't work since we have one instruction being decoded and another writing back.
  - We resolve this by having separate read and write ports. However, this only works if the read/written registers are different.
  - To account for reads and writes to the same register, processors usually write to the register during the first half of the clock cycle, and read from it during in the second half. This is an implementation of the idea of **double pumping**, which is when data is transferred along data buses at double the rate, by utilising both the rising and falling clock edges in a clock cycle.
- **Main Memory:** Main memory is accessed for both instructions and data. If memory could only support one read/write at a time, then instruction A going through IF and attempting to fetch an instruction from memory cannot happen at the same time as instruction B attempting to read (or write) to data portions of memory.

– Having a separate instruction memory (abbreviated IMEM) and data memory (abbreviated DMEM) solves this hazard.

Something to remember about structural hazards is that they can always be resolved by adding more hardware.

Data Hazards

Data hazards are caused by data dependencies between instructions. In CS 61C, where we always assume that instructions go through the processor in order, we see data hazards when an instruction **reads** a register before a previous instruction has finished **writing** to that register.

There are two types of data hazards:

- **EX-ID:** this hazard exists because the output from the execute stage is not written back to the RegFile until the writeback stage, yet can be requested by the subsequent instruction in the decode stage.
- **MEM-ID:** this hazard exists because the output from the memory access stage is not written back to the RegFile until the writeback stage, but can be requested from the decode stage, just as in EX-ID.

Solving Data Hazards

For all questions, assume **no branch prediction or double-pumping**.

Forwarding

Most data hazards can be resolved by forwarding, which is when the result of the EX or MEM stage is sent to the EX stage for a following instruction to use.

4.1 Look for data hazards in the code below, and figure out how forwarding could be used to solve them.

Instruction	C1	C2	C3	C4	C5	C6	C7
1. `addi t0, a0, -1`	IF	ID	EX	MEM	WB		
2. `and s2, t0, a0`		IF	ID	EX	MEM	WB	
3. `sltiu a0, t0, 5`			IF	ID	EX	MEM	WB

There are two data hazards, between instructions 1 and 2, and between instructions 1 and 3. The first could be resolved by forwarding the result of the EX stage in C3 to the beginning of the EX stage in C4, and the second could be resolved by forwarding the result of the MEM stage in C4 to the beginning of the EX stage in C5.

4.2 Imagine you are a hardware designer working on a CPU's forwarding control logic. How many instructions after the addi instruction could be affected by data hazards created by this addi instruction?

Three instructions. For example, with the addi instruction, any instruction that uses t0 that has its ID stage in C3, C4, or C5 will not have the result of addi's writeback in C5. If, however, we are allowed to assume double-pumping (write-then-read to registers), then it would only affect two instructions since the ID stage of instruction

4 would be allowed to line up with the WB stage of intruction 1. (Side note: how is this implemented in hardware? We add 2 wires: one from the beginning of the MEM stage for the output of the ALU and one from the beginning of the WB stage. Both of these wires will connect to the A/B muxes in the EX stage.)

Stalls

4.3 Look for data hazards in the code below. One of them cannot be solved with forwarding—why? What can we do to solve this hazard?

Instruction	C1	C2	C3	C4	C5	C6	C7	C8
1. `addi s0, s0, 1`	IF	ID	EX	MEM	WB			
2. `addi t0, t0, 4`		IF	ID	EX	MEM	WB		
3. `lw t1, 0(t0)`			IF	ID	EX	MEM	WB	
4. `add t2, t1, x0`				IF	ID	EX	MEM	WB

There are two data hazards in the code. The first hazard is between instructions 2 and 3, from t0, and the second is between instructions 3 and 4, from t1. The hazard between instructions 2 and 3 can be resolved with forwarding, but the hazard between instructions 3 and 4 cannot be resolved with forwarding. This is because even with forwarding, instruction 4 needs the result of instruction 3 at the beginning of C6, and it won't be ready until the end of C6.

We can fix this by stalling: insert a nop (no-operation) between instructions 3 and 4.

4.4 Say you are the compiler and can re-order instructions to minimize data hazards while guaranteeing the same output. How can you fix the code above?

Reorder the instructions 2-3-1-4, because instruction 1 has no dependencies.

Detecting Data Hazards

Say we have the $rs1$, $rs2$, $RegWEn$, and rd signals for two instructions (instruction n and instruction $n+1$) and we wish to determine if a data hazard exists across the instructions. We can simply check to see if the rd for instruction n matches either $rs1$ or $rs2$ of instruction $n+1$, indicating that such a hazard exists (why does this make sense?).

We could then use our hazard detection to determine which forwarding paths/number of stalls (if any) are necessary to take to ensure proper instruction execution. In pseudo-code, part of this could look something like the following:

```
if (rs1(n + 1) == rd(n) && RegWen(n) == 1) {
    set ASel for (n + 1) to forward ALU output from n
}
if (rs2(n + 1) == rd(n) && RegWen(n) == 1) {
    set BSel for (n + 1) to forward ALU output from n
}
```

Control Hazards

Control hazards are caused by **jump and branch instructions**, because for all jumps and some branches, the next PC is not PC + 4, but the result of the ALU available after the EX stage. We could stall the pipeline for control hazards, but this decreases performance.

4.5 Besides stalling, what can we do to resolve control hazards?

We can predict which way branches will go, and when this prediction is incorrect, flush the pipeline and continue with the correct instruction. (The most naive prediction method is to simply predict that branches are always not taken).

Extra for Experience

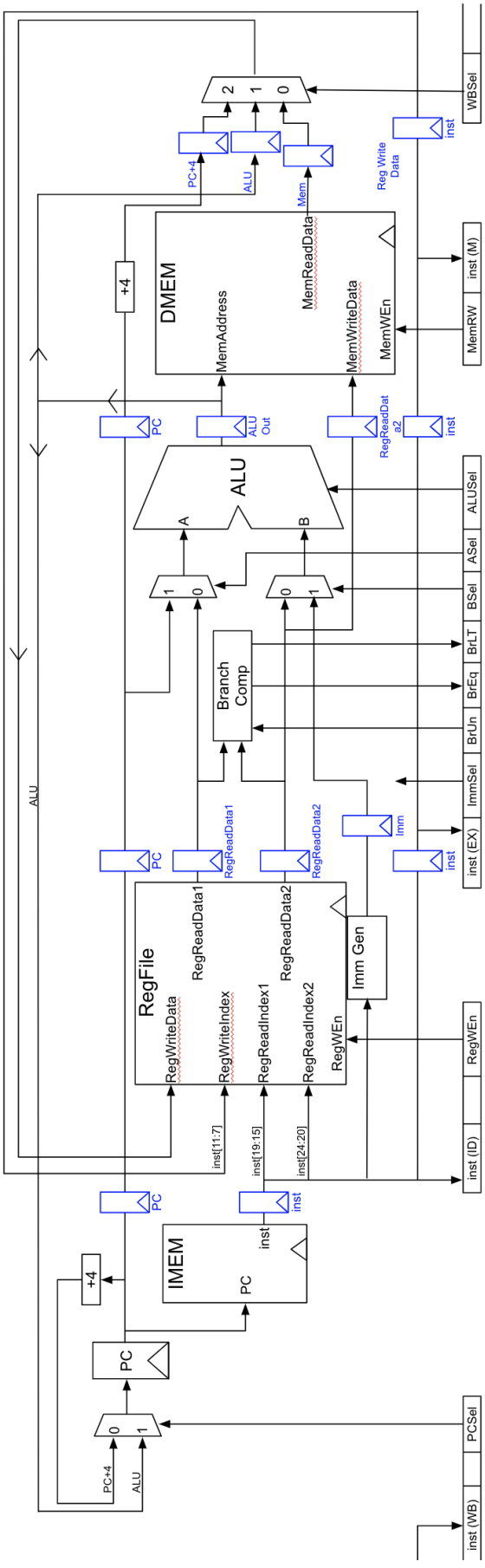
4.6 Given the RISC-V code above and a pipelined CPU with no forwarding, how many hazards would there be? What types are each hazard? Consider all possible hazards between all instructions.

How many stalls would there need to be in order to fix the data hazard(s), if we use double-pumping? What about the control hazard(s), if we use branch prediction?

Instruction	C1	C2	C3	C4	C5	C6	C7	C8	C9
1. `sub t1, s0, s1`	IF	ID	EX	MEM	WB				
2. `or s0, t0, t1`		IF	ID	EX	MEM	WB			
3. `sw s1, 100(s0)`			IF	ID	EX	MEM	WB		
4. `bgeu s0, s2, loop`				IF	ID	EX	MEM	WB	
5. `add t2, x0, x0`					IF	ID	EX	MEM	WB

There are four hazards: between instructions 1 and 2 (data hazard from t1), instructions 2 and 3 (data hazard from s0), instructions 2 and 4 (from s0), and instructions 4 and 5 (a control hazard).

Assuming that we can read and write to the RegFile on the same cycle, two stalls are needed between instructions 1 and 2 (WB→ID), and two stalls are needed between instructions 2 and 3 (WB→ID). For the control hazard, if we predicted correctly, then no stalls are needed, but if we predicted incorrectly, then we need 3 stalls while flushing the pipeline (MEM→1 cycle before IF). We don't need to stall for the hazard between 2 and 4 because stalling for instruction 3 already handles that.


PCSel
WBSel
inst (WB)
inst (ID)
inst (EX)
inst (M)
RegWEn
ImmSel
BrUn
BrEq
BrLT
BSel
ASel
ALUSel
MemRW
PC+4
ALU
PC
inst
IMEM
RegFile
RegWriteData
RegWriteIndex
RegReadIndex1
RegReadIndex2
RegWEn
RegReadData1
RegReadData2
Imm Gen
Branch Comp
DMEM
MemAddress
MemReadData
MemWriteData
MemWEn
Mem
Reg Write Data
RegReadData1
RegReadData2
Imm
ALU Out
RegReadDat a2
A
B
inst[11:7]
inst[19:15]
inst[24:20]
+4

CS 61C Fall 2022

Caches

Discussion 9

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 We cannot use a 1KB cache in a 32-bit system because it's too small and cannot contain all possible addresses.

1.2 If a piece of data is both in the cache and in memory, reading it from cache is faster than reading from memory.

1.3 Caches see an immediate improvement in memory access time at program execution.

1.4 Any cache miss that occurs when the cache is full is a capacity miss.

1.5 Increasing cache size by adding more blocks always improves (increases) hit rate for all programs.

1.6 Decreasing block size for a cache to increase the number of blocks held by the cache improves the program speed for all programs.

2 Hazards Review: CPI

Consider the following C program, which is compiled into RISC-V code on the right:

```
int func(int* arr) {
    int sum = 0;
    for(int i = 0; i < 10; i++) {
        sum += arr[i];
    }
    arr[0] = sum / 8;
    return sum;
}
```

```
    # a0: arr
    1. add t0, x0, x0  # t0: sum
    2. add t1, x0, x0  # t1: i
    3. addi t3, x0, 10
loop:
    4. bge t1, t3, end
    5. slli t2, t1, 2
    6. addi t2, t2, a0
    7. lw t2, 0(t2)
    8. add t0, t0, t2
    9. addi t1, t1, 1
    10.jal x0, loop
end:
    11.srai t2, t0, 3
    12.sw t2, 0(a0)
    13.add a0, t0, x0
    14.jalr x0, ra, 0
```

Assume that we are using the 5-stage pipeline from before with no optimizations (no forwarding, no double-pumping, and no branch prediction), but we'll always fetch the correct next instruction for jump instructions.

2.1 How many stalls do we need for a data hazard between two adjacent instructions? How many penalty cycles do we need to stall for a control hazard?

2.2 How many (RISC-V) instructions are executed through the execution of the compiled program?

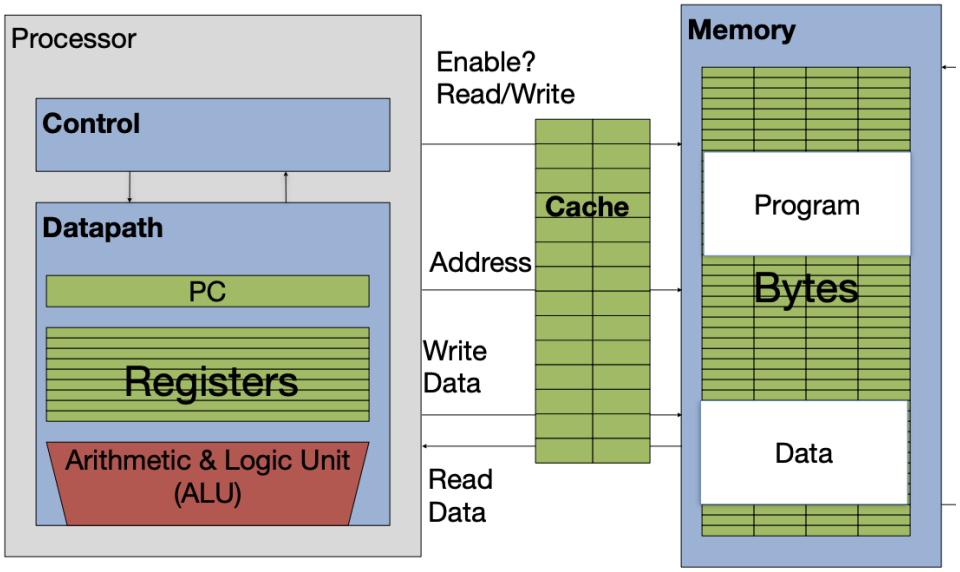
2.3 How many cycles does it take to execute the compiled program with our 5-stage pipelined CPU? Ignore the first 4 cycles where the pipeline isn't completely filled yet for now (i.e. start counting from when the first instruction is in its WB stage).

2.4 Based on the previous two parts, what is the CPI (cycles per instruction) of our CPU on this compiled program?

For additional review, you may attempt 2.3 and 2.4 with forwarding.

3 Understanding T/I/O

We use caches to make our access to data faster. When working with main memory (RAM), the main problem faced is the fact that access to the main memory is very slow. In fact, modern processors take about 100 instructions cycles or more to access the main memory, meaning memory accesses become the bottleneck of our programs. Caches help fix this problem for us - they hold a portion of the data in main memory, that we might access again later on. They are closer to the processor in the memory hierarchy, and thus accessing a cache is much faster than accessing the main memory.



As seen above, the access to cache is the middle step between the CPU asking for a memory bit, and the actual main memory access - if the data is not found in the cache, only then is main memory accessed. This way unnecessary trips to main memory are avoided. One important detail is that caches are much smaller in size than main memory - this is why we have to be efficient in what we hold in caches. When we are saving data in caches, we need to be as efficient as possible. In order to do this, we make use of locality. We have two different kinds of locality to consider.

Temporal Locality: If we have accessed a piece of information recently, it is possible that we will access it again. So, we hold this data in the cache.
Spatial Locality: If we have accessed a memory location recently, it is probable that we will access the neighbouring addresses as well. So, we also keep the neighbouring addresses within the cache. An example is array accesses - if we access the 0th element of an array, it is probable we will also access the 1st one.

Note that caches hold the data in blocks that have a size equal to the block size of the cache.

When working with caches, we have to be able to break down the memory addresses we work with to understand where they fit into our caches. There are three fields:

Tag - Used to distinguish different blocks that use the same index. Number of bits: (# of bits in memory address) - Index Bits - Offset Bits
Index - The set that this piece of memory will be placed in. Number of bits: $\log_2$(# of indices)
Offset - The location of the byte in the block. Number of bits: $\log_2$(size of block)

Given these definitions, the following is true:

$$\log_2(\text{memory size}) = \text{address bit-width} = \#\text{ tag bits} + \#\text{ index bits} + \#\text{ offset bits}$$

Another useful equality to remember is:

$$\text{cache size} = \text{block size} * \text{num blocks}$$

3.1 Assume we have a direct-mapped byte-addressed cache with capacity 32B and block size of 8B. Of the 32 bits in each address, which bits do we use to find the index of the cache to use?

3.2 Which bits are our tag bits? What about our offset?

3.3 Classify each of the following byte memory accesses as a cache hit (H), cache miss (M), or cache miss with replacement (R). Tip: Drawing out the cache can help you see the replacements more clearly.

Address	T/I/O	Hit, Miss, Replace
`0x00000004`		
`0x00000005`		
`0x00000068`		
`0x000000C8`		
`0x00000068`		
`0x000000DD`		
`0x00000045`		
`0x00000004`		
`0x000000C8`		

4 The 3 C's of Cache Misses

4.1 Go back to question 3 and classify each M and R as one of the 3 types of misses described below:

1. Compulsory: First time you ask the cache for a certain block. A miss that must occur when you first bring in a block. Reduce compulsory misses by having longer cache lines (bigger blocks), which bring in the surrounding addresses along with our requested data. Can also pre-fetch blocks beforehand using a hardware prefetcher (a special circuit that tries to guess the next few blocks that you will want).
2. Conflict: Occurs if, hypothetically, you went through the **entire** string of accesses with a fully associative cache and wouldn't have missed for that specific access. Increasing the associativity or improving the replacement policy would remove the miss. Note that Conflict misses tend to occur with inefficient cache usage, compared to Capacity misses.
3. Capacity: Capacity misses are independent of the associativity of your cache. If you hypothetically ran the **entire** string of memory accesses with a fully associative cache of the same size as your cache, and it was a miss for that specific access, then this miss is a capacity miss. The only way to remove the miss is to increase the cache capacity.

Note: The test you can use to see if a miss is a conflict miss is the same as the test you can use to see if a miss is a capacity miss.

Note: There are many different ways of fixing misses. The name of the miss doesn't necessarily tell us the best way to reduce the number of misses.

5 Code Analysis

Given the follow chunk of code, analyze the hit rate given that we have a byte-addressed computer with a total memory of **1 MiB**. It also features a **16 KiB** Direct-Mapped cache with **1 KiB** blocks. Assume that your cache begins cold.

```
#define NUM_INTS 8192   // 2^13
int A[NUM_INTS];        // A lives at 0x10000
int i, total = 0;
for (i = 0; i < NUM_INTS; i += 128) {
    A[i] = i;           // Line 1
}
for (i = 0; i < NUM_INTS; i += 128) {
    total += A[i];      // Line 2
}
```

5.1 How many bits make up a memory address on this computer?

5.2 What is the T:I:O breakdown?

5.3 Calculate the cache hit rate for the line marked Line 1:

5.4 Calculate the cache hit rate for the line marked Line 2:

CS 61C Fall 2022

Caches

Discussion 9

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 We cannot use a 1KB cache in a 32-bit system because it's too small and cannot contain all possible addresses.

False. The purpose of the cache is not to hold every possible piece of memory at the same time, but rather to hold some parts of it only, so a 1KB cache is not "too small".

1.2 If a piece of data is both in the cache and in memory, reading it from cache is faster than reading from memory.

True. The cache is smaller and faster than memory.

1.3 Caches see an immediate improvement in memory access time at program execution.

False. A cache starts off 'cold', and required loading in values in blocks at first directly from memory, forcing compulsory misses. This can be somewhat alleviated by the use of a hardware prefetcher, that uses the current pattern of misses to predict and prefetch data that may be accessed later on.

1.4 Any cache miss that occurs when the cache is full is a capacity miss.

False. When the cache is full, you can still get compulsory misses (when a block of data is put in the cache for the first time).

1.5 Increasing cache size by adding more blocks always improves (increases) hit rate for all programs.

False. Whether this improves the hit rate for a given program depends on the characteristics of the program. As an example, it is possible for a program that only consists of a loop that runs through an array once to have each access be separated by more than one block (say, the block size is 8B, but we have an integer array and accessing every fourth element, so our access are separated by 16B). This makes every miss a compulsory miss, and there is no way for us to reduce the number of compulsory misses just by adding more blocks to our cache.

1.6 Decreasing block size for a cache to increase the number of blocks held by the cache improves the program speed for all programs.

False. This question is similar to the one above, in that the answer to it depends on the program that is running. If we have a program with a for loop that loops through continuous memory (like an array), having a bigger blocks size and fewer

blocks might be helpful, as the single blocks will holds more continuous data. For example, lets say cache A has 10 lines and a block size of 8 bytes, while cache B has 20 lines with a a block size of 4 bytes and the array we loop through has 80 characters. Cache A in this case will have 10 cache misses and 70 hits, while Cache B will have 20 misses and 60 hits.

2 Hazards Review: CPI

Consider the following C program, which is compiled into RISC-V code on the right:

```
int func(int* arr) {
    int sum = 0;
    for(int i = 0; i < 10; i++) {
        sum += arr[i];
    }
    arr[0] = sum / 8;
    return sum;
}
```

```
    # a0: arr
    1. add t0, x0, x0  # t0: sum
    2. add t1, x0, x0  # t1: i
    3. addi t3, x0, 10
loop:
    4. bge t1, t3, end
    5. slli t2, t1, 2
    6. addi t2, t2, a0
    7. lw t2, 0(t2)
    8. add t0, t0, t2
    9. addi t1, t1, 1
    10.jal x0, loop
end:
    11.srai t2, t0, 3
    12.sw t2, 0(a0)
    13.add a0, t0, x0
    14.jalr x0, ra, 0
```

Assume that we are using the 5-stage pipeline from before with no optimizations (no forwarding, no double-pumping, and no branch prediction), but we'll always fetch the correct next instruction for jump instructions.

2.1 How many stalls do we need for a data hazard between two adjacent instructions? How many penalty cycles do we need to stall for a control hazard?

For data hazards, the second instruction's ID stage needs to wait until after the first instruction's WB stage to execute, resulting in 3 stalled cycles. For control hazards, if the branch is taken, then the second instruction's IF stage needs to wait until after the first instruction's MEM stage to execute, resulting in 3 stalled cycles.

2.2 How many (RISC-V) instructions are executed through the execution of the compiled program?

We start with the three instructions before the loop, then execute the instructions between loop and end 10 times, then execute the bge instruction once more before finishing with the four instructions at the end, resulting in a total of $3+10\times7+1+4 = 78$ instructions.

2.3 How many cycles does it take to execute the compiled program with our 5-stage pipelined CPU? Ignore the first 4 cycles where the pipeline isn't completely filled yet for now (i.e. start counting from when the first instruction is in its WB stage).

Once the pipeline is completely filled, each instruction will take only one cycle, except for stalls, so let's analyze where we need to stall in the program.

3→4: data hazard on t3, 3-cycle stall executed once. (There is also a data hazard on t1 between 2→4, but we don't need to stall extra for it since we're already stalling for 3→4).

5→6: data hazard on t2, 3-cycle stall executed 10 times.

6→7: data hazard on t2, 3-cycle stall executed 10 times.

7→8: data hazard on t2, 3-cycle stall executed 10 times.

9→4: data hazard on t1, 2-cycle stall executed 10 times. The execution here will always be 9→10→4, where t1 is used two instructions after being set, so we only need to stall two cycles each time.

4→11: control hazard! Whenever the branch is taken, we need to stall for 3 cycles, and this happens only once.

11→12: data hazard on t2, 3-cycle stall executed once.

Therefore, the total stall cycles we need are $3 + (3 + 3 + 3 + 2) \times 10 + 3 + 3 = 119$ cycles. Add one cycle per instruction for executing the instructions normally to get $119 + 78 = 197$ total cycles.

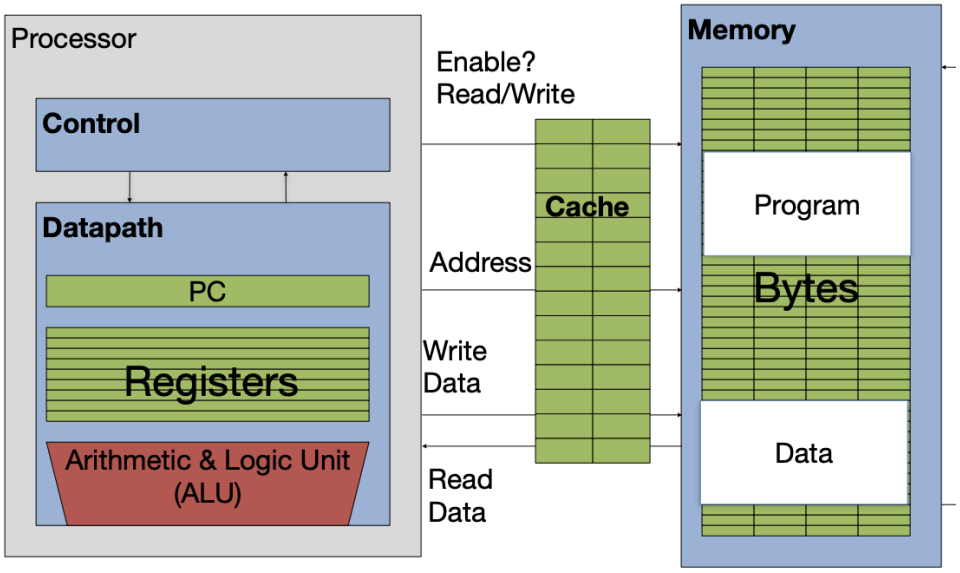
2.4 Based on the previous two parts, what is the CPI (cycles per instruction) of our CPU on this compiled program?

Add the first 4 cycles where the pipeline is being filled up to our answer in 2.3: $\frac{197+4}{78} = 2.58$

For additional review, you may attempt 2.3 and 2.4 with forwarding.

3 Understanding T/I/O

We use caches to make our access to data faster. When working with main memory (RAM), the main problem faced is the fact that access to the main memory is very slow. In fact, modern processors take about 100 instructions cycles or more to access the main memory, meaning memory accesses become the bottleneck of our programs. Caches help fix this problem for us - they hold a portion of the data in main memory, that we might access again later on. They are closer to the processor in the memory hierarchy, and thus accessing a cache is much faster than accessing the main memory.



As seen above, the access to cache is the middle step between the CPU asking for a memory bit, and the actual main memory access - if the data is not found in the cache, only then is main memory accessed. This way unnecessary trips to main memory are avoided. One important detail is that caches are much smaller in size than main memory - this is why we have to be efficient in what we hold in caches. When we are saving data in caches, we need to be as efficient as possible. In order to do this, we make use of locality. We have two different kinds of locality to consider.

> **Temporal Locality:** If we have accessed a piece of information recently, it is possible that we will access it again. So, we hold this data in the cache.
> **Spatial Locality:** If we have accessed a memory location recently, it is probable that we will access the neighbouring addresses as well. So, we also keep the neighbouring addresses within the cache. An example is array accesses - if we access the 0th element of an array, it is probable we will also access the 1st one.

Note that caches hold the data in blocks that have a size equal to the block size of the cache.

When working with caches, we have to be able to break down the memory addresses we work with to understand where they fit into our caches. There are three fields:

Tag - Used to distinguish different blocks that use the same index. Number of bits: (# of bits in memory address) - Index Bits - Offset Bits
Index - The set that this piece of memory will be placed in. Number of bits: $\log_2$(# of indices)
Offset - The location of the byte in the block. Number of bits: $\log_2$(size of block)

Given these definitions, the following is true:

$\log_2$(memory size) = address bit-width = # tag bits + # index bits + # offset bits

Another useful equality to remember is:

$$\text{cache size} = \text{block size} * \text{num blocks}$$

3.1 Assume we have a direct-mapped byte-addressed cache with capacity 32B and block size of 8B. Of the 32 bits in each address, which bits do we use to find the index of the cache to use?

We can determine the number of index bits we need from the number of sets our cache has. Since our cache is direct-mapped, the number of sets is the same as the number of blocks, so we just need to figure out how many blocks our cache has. Using the equality from above, we see that num blocks = cache size/block size, so our cache has $32/8 = 4$ blocks. We need $\log_2(4) = 2$ bits to differentiate the 4 blocks, so we have 2 index bits.
In order to determine where exactly the index bits are, we need to calculate the number of offset bits and tag bits we have. The number of offset bits is just dependent on the block size, so since our blocks are size 8B, we need $\log_2(8) = 3$ bits to differentiate the 8 bytes in the block, so we have 3 offset bits.
our offset bits take up the least significant bits, with the index bits being the set of next most significant bits. Denoting the most significant bit (MSB, on the left) as 31 and the least significant bit (LSB, on the right) as 0, having 3 offset bits means our index bits start at bit 3, and thus we use bits 3 and 4 as the index bits.

3.2 Which bits are our tag bits? What about our offset?

The offset (in this case) is the 3 least significant bits, so reusing the convention from the previous quesiton, the offset bits are bits 0, 1, and 2. Our tag is the remaining high-order bits, so our tag bits are bits 5-31.

3.3 Classify each of the following byte memory accesses as a cache hit (H), cache miss (M), or cache miss with replacement (R). Tip: Drawing out the cache can help you see the replacements more clearly.

Address	T/I/O	Hit, Miss, Replace
0x00000004		
0x00000005		
0x00000068		
0x000000C8		
0x00000068		
0x000000DD		
0x00000045		
0x00000004		
0x000000C8		

Ignore miss types (compulsory/conflict/capacity) until Q4.

```
0x00000004    Tag 0, Index 0, Offset 4: M, Compulsory
0x00000005    Tag 0, Index 0, Offset 5: H
0x00000068    Tag 3, Index 1, Offset 0: M, Compulsory
0x000000C8    Tag 6, Index 1, Offset 0: R, Compulsory
0x00000068    Tag 3, Index 1, Offset 0: R, Conflict
0x000000DD    Tag 6, Index 3, Offset 5: M, Compulsory
0x00000045    Tag 2, Index 0, Offset 5: R, Compulsory
0x00000004    Tag 0, Index 0, Offset 4: R, Capacity
0x000000C8    Tag 6, Index 1, Offset 0: R, Capacity
```

Note that the M and R distinction here is for student understanding, and that the cache doesn't behave differently for these cases.

4 The 3 C's of Cache Misses

4.1 Go back to question 3 and classify each M and R as one of the 3 types of misses described below:

1. Compulsory: First time you ask the cache for a certain block. A miss that must occur when you first bring in a block. Reduce compulsory misses by having longer cache lines (bigger blocks), which bring in the surrounding addresses along with our requested data. Can also pre-fetch blocks beforehand using a hardware prefetcher (a special circuit that tries to guess the next few blocks that you will want).
2. Conflict: Occurs if, hypothetically, you went through the **entire** string of accesses with a fully associative cache and wouldn't have missed for that specific access. Increasing the associativity or improving the replacement policy would remove the miss. Note that Conflict misses tend to occur with inefficient cache usage, compared to Capacity misses.
3. Capacity: Capacity misses are independent of the associativity of your cache. If you hypothetically ran the **entire** string of memory accesses with a fully

associative cache of the same size as your cache, and it was a miss for that specific access, then this miss is a capacity miss. The only way to remove the miss is to increase the cache capacity.

Note: The test you can use to see if a miss is a conflict miss is the same as the test you can use to see if a miss is a capacity miss.

See solutions for Q3.

5 Code Analysis

Given the follow chunk of code, analyze the hit rate given that we have a byte-addressed computer with a total memory of **1 MiB**. It also features a **16 KiB** Direct-Mapped cache with **1 KiB** blocks. Assume that your cache begins cold.

```
#define NUM_INTS 8192   // 2^13
int A[NUM_INTS];        // A lives at 0x10000
int i, total = 0;
for (i = 0; i < NUM_INTS; i += 128) {
    A[i] = i;           // Line 1
}
for (i = 0; i < NUM_INTS; i += 128) {
    total += A[i];      // Line 2
}
```

5.1 How many bits make up a memory address on this computer?

We take $\log_2(1 \text{ MiB}) = \log_2(2^{20}) = 20$.

5.2 What is the T:I:O breakdown?

Offset $= \log_2(1 \text{ KiB} = \log_2(2^{10}) = 10$
Index $= \log_2(\frac{16 \text{ KiB}}{1 \text{ KiB}}) = \log_2(16) = 4$
Tag $= 20 - 4 - 10 = 6$

5.3 Calculate the cache hit rate for the line marked Line 1:

The integer accesses are $4 * 128 = 512$ bytes apart, which means there are 2 accesses per block. The first accesses in each block is a compulsory cache miss, but the second is a hit because `A[i]` and A[i+128] are in the same cache block. Thus, we end up with a hit rate of **50%**.

5.4 Calculate the cache hit rate for the line marked Line 2:

The size of A is $8192 * 4 = 2^{15}$ bytes. This is exactly twice the size of our cache. At the end of Line 1, we have the second half of A inside our cache, but Line 2 starts with the first half of A. Thus, we cannot reuse any of the cache data brought in from Line 1 and must start from the beginning. Thus our hit rate is the same as Line 1 since we access memory in the same exact way as Line 1. We don't have to consider cache hits for total, as the compiler will most likely store it in a register. Thus, we end up with a hit rate of **50%**.

CS 61C Fall 2022

Cache Performance

Discussion 10

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 In a write-back cache, writes are generally slower than a write-through cache.

1.2 In a fully associative cache, the number of index bits is always zero.

1.3 For the same cache size and block size, a 4-way set associative cache will have fewer index bits than a direct-mapped cache.

1.4 Using N-way set associative caches can help balance between hardware complexity and cache performance.

1.5 Increasing cache size (while keeping other parameters constant) for a fully associative cache with LRU replacement will never increase miss rate (for the same memory access pattern).

1.6 Adding another level of caches will never increase AMAT.

1.7 The miss penalty for the last level of caches is equal to the access time of main memory.

2 Cache Associativity

In the previous discussion, we primarily focuses on Direct-Mapped caches, in which blocks map to specifically one slot in our cache. This is good for quick replacement and finding out block, but not good for spatial efficiency!

This is where we bring associativity into the matter. We define associativity as the number of slots a block can potentially map to in our cache. Thus, a Fully-Associative cache has the most associativity, meaning every block can go anywhere in the cache. Our Direct-Mapped cache, on the other hand, has the least, being only 1-way set associative.

For an N-way associative cache, the following are true:

$$\text{N} * \#\text{ sets} = \#\text{ blocks}, \quad \text{Index bits} = \log_2(\#\text{ sets})$$

2.1 Here's some practice involving a 2-way set associative cache. This time we have an 8-bit address space, 8 B blocks, and a cache size of 32 B. Classify each of the following accesses as a cache hit (H), cache miss (M) or cache miss with replacement (R). For any misses, list out which type of miss it is (Compulsory, Conflict, or Capacity). Assume that we have an LRU replacement policy (in general, this is not the case).

Address	T/I/O	Hit, Miss, Replace
`0b0000 0100`		
`0b0000 0101`		
`0b0110 1000`		
`0b1100 1000`		
`0b0110 1000`		
`0b1101 1101`		
`0b0100 0101`		
`0b0000 0100`		
`0b1100 1000`		

2.2 What is the hit rate of our above accesses?

3 Writes

When it comes to writing data to cache memory, there are multiple write policies to consider that offer different options when building our system. Some of them you might encounter are:

1. **Write-through**: In this policy, when we have a write we write to both the cache and the memory. This is the case for every write, so the main memory always has the updated data. This is simple to implement, but writing to main memory every single time is slow.
2. **Write-back**: On a write, the data is only updated/written in the cache. The main memory only receives the data upon eviction. This means the cache has more up to date data most of the time. While this is faster as there is less

accesses to main memory, it is harder to implement as we have to include more overhead, such as dirty bits and so on.

3. **Write-around**: Data is only written to main memory, and whenever we do so we automatically invalidate the old data in the cache.

Another thing to consider is what we do when we want to write to memory that is not in the cache, or a write miss. For that, we have 2 possible policies:

1. **Write-allocate**: On a write miss, we pull the block you missed on into the cache
2. **No write-allocate**: On a write miss, you do not pull the block you missed on into the cache. Only memory is updated. On a read cache miss, we still pull the data into the cache.

3.1 Considering the above information, lets consider a direct mapped, no write-allocate write-through cache with a capacity of 8B and a block size of 4B. Lets also assume that the memory addresses are 8 bits each. Assuming the cache is completely empty in the beginning, we make memory accesses to the following locations:

- 0x6A, Write
- 0x85, Read
- 0x6B, Read
- 0x87, Read
- 0x68, Write

With the above memory access pattern and the given cache configuration, how many times do we access the main memory?

3.2 Lets say for the same cache size, memory accesses but the only difference is that we have a no write-allocate write-back cache instead. How many memory accesses to the main memory do we have in this case?

3.3 For one last optimization, we decide to use a write-allocate cache instead. So, now we have a write-allocate, write-back cache. How many times do we access the memory now?

3.4 We discovered in our previous subparts that one set of write policies leads to less main memory accesses than the others. Out of the policies covered in class, is this

the most optimal set of write policies for a cache? If so, explain why. If they are not, what trade-offs come as a result of these policies?

4 AMAT

Recall that AMAT stands for Average Memory Access Time. The main formula for it is:

$$\text{AMAT} = \text{Hit Time} + \text{Miss Rate} * \text{Miss Penalty}$$

In a multi-level cache structure, we can separate miss rates into two types that we consider for each level.

- **Global:** Calculated as the number of accesses that missed at that level divided by the total number of accesses *to the cache system*.
- **Local:** Calculated as the number of accesses that missed at that level divided by the total number of accesses *to that cache level*.

4.1 In a 2-level cache system, after 100 total accesses to the cache system, we find that the L2$ (L2 cache) ended up missing 20 times. What is the global miss rate of L2$?

4.2 Given the system from the previous subpart, if L1$ had a local miss rate of 50%, what is the local miss rate of L2$?

Suppose your system consists of:

1. An L1$ that has a hit time of 2 cycles and has a local miss rate of 20%
2. An L2$ that has a hit time of 15 cycles and has a global miss rate of 5%
3. Main memory where accesses take 100 cycles

4.3 What is the local miss rate of L2$?

4.4 What is the AMAT of the system?

4.5 Suppose we want to reduce the AMAT of the system to 8 cycles or lower by adding in a L3$. If the L3$ has a local miss rate of 30%, what is the largest hit time that the L3$ can have?

CS 61C Fall 2022

Cache Performance

Discussion 10

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 In a write-back cache, writes are generally slower than a write-through cache.

False. Writes are generally faster in a write-back cache since they only write to cache, whereas writes in a write-through cache write to both cache and main memory.

1.2 In a fully associative cache, the number of index bits is always zero.

True. Since any block can go anywhere in a fully associative cache, there is no need for an index field that determines where each block could go to.

1.3 For the same cache size and block size, a 4-way set associative cache will have fewer index bits than a direct-mapped cache.

True. A direct mapped cache needs to index every line of the cache, whereas a 4-way set associative cache needs to index every set of 4 lines. The 4-way set associative cache will have 2 fewer index bits than the direct-mapped cache.

1.4 Using N-way set associative caches can help balance between hardware complexity and cache performance.

True. As N goes up, there will generally be less conflict misses, but more intricate comparator circuits are needed for the tag. Since the value of N is adjustable, N-way set associative caches helps balance these two aspects.

1.5 Increasing cache size (while keeping other parameters constant) for a fully associative cache with LRU replacement will never increase miss rate (for the same memory access pattern).

True. Since we always keep the most recently used blocks in cache, the misses on a cache with larger size is a subset of the misses on the same cache with smaller size.

Interestingly, this is not true if the replacement policy was FIFO! This is called Bélády's anomaly and is out of scope for this class. CS 162 will talk more about this...

1.6 Adding another level of caches will never increase AMAT.

False. In the extreme case of a miss rate of 100%, adding a new level of caches will increase AMAT by the hit time of those caches.

1.7 The miss penalty for the last level of caches is equal to the access time of main memory.

True. If we miss on the last level of caches, we must go to main memory to find the data we need.

2 Cache Associativity

In the previous discussion, we primarily focuses on Direct-Mapped caches, in which blocks map to specifically one slot in our cache. This is good for quick replacement and finding out block, but not good for spatial efficiency!

This is where we bring associativity into the matter. We define associativity as the number of slots a block can potentially map to in our cache. Thus, a Fully-Associative cache has the most associativity, meaning every block can go anywhere in the cache. Our Direct-Mapped cache, on the other hand, has the least, being only 1-way set associative.

For an N-way associative cache, the following are true:

$$\text{N} * \#\text{ sets} = \#\text{ blocks}, \quad \text{Index bits} = \log_2(\#\text{ sets})$$

2.1 Here's some practice involving a 2-way set associative cache. This time we have an 8-bit address space, 8 B blocks, and a cache size of 32 B. Classify each of the following accesses as a cache hit (H), cache miss (M) or cache miss with replacement (R). For any misses, list out which type of miss it is (Compulsory, Conflict, or Capacity). Assume that we have an LRU replacement policy (in general, this is not the case).

Address	T/I/O	Hit, Miss, Replace
`0b0000 0100`		
`0b0000 0101`		
`0b0110 1000`		
`0b1100 1000`		
`0b0110 1000`		
`0b1101 1101`		
`0b0100 0101`		
`0b0000 0100`		
`0b1100 1000`		

Since our cache is 2-way set associative, there are 2 blocks in a set. Given the cache size and the block size, we have 32 / 8 = 4 blocks. Thus, there are 4 / 2 = 2 sets in our cache. We need $\log_2(2) = 1$ bit to differentiate the 2 sets, so we have 1 index bit. Our block size of 8 B means we have $\log_2(8) = 3$ offset bits, and that the rest of our bits are our tag bits. Therefore, our TIO breakdown means bits 0, 1, and 2 are our offset bits, the only index bit is bit 3, and bits 4-7 being the tag bits.

```
0b0000 0100     Tag 0000, Index 0, Offset 100 - M, Compulsory
0b0000 0101     Tag 0000, Index 0, Offset 101 - H
0b0110 1000     Tag 0110, Index 1, Offset 000 - M, Compulsory
0b1100 1000     Tag 1100, Index 1, Offset 000 - M, Compulsory
0b0110 1000     Tag 0110, Index 1, Offset 000 - H
0b1101 1101     Tag 1101, Index 1, Offset 101 - R, Compulsory
0b0100 0101     Tag 0100, Index 0, Offset 101 - M, Compulsory
0b0000 0100     Tag 0000, Index 0, Offset 100 - H
0b1100 1000     Tag 1100, Index 1, Offset 000 - R, Capacity
```

2.2 What is the hit rate of our above accesses?

$\frac{3 \text{ hits}}{9 \text{ accesses}} = \frac{1}{3}$ hit rate

3 Writes

When it comes to writing data to cache memory, there are multiple write policies to consider that offer different options when building our system. Some of them you might encounter are:

1. **Write-through**: In this policy, when we have a write we write to both the cache and the memory. This is the case for every write, so the main memory always has the updated data. This is simple to implement, but writing to main memory every single time is slow.
2. **Write-back**: On a write, the data is only updated/written in the cache. The main memory only receives the data upon eviction. This means the cache has more up to date data most of the time. While this is faster as there is less accesses to main memory, it is harder to implement as we have to include more overhead, such as dirty bits and so on.
3. **Write-around**: Data is only written to main memory, and whenever we do so we automatically invalidate the old data in the cache.

Another thing to consider is what we do when we want to write to memory that is not in the cache, or a write miss. For that, we have 2 possible policies:

1. **Write-allocate**: On a write miss, we pull the block you missed on into the cache
2. **No write-allocate**: On a write miss, you do not pull the block you missed on into the cache. Only memory is updated. On a read cache miss, we still pull the data into the cache.

3.1 Considering the above information, lets consider a direct mapped, no write-allocate write-through cache with a capacity of 8B and a block size of 4B. Lets also assume that the memory addresses are 8 bits each. Assuming the cache is completely empty in the beginning, we make memory accesses to the following locations:

- 0x6A, Write
- 0x85, Read
- 0x6B, Read

- 0x87, Read
- 0x68, Write

With the above memory access pattern and the given cache configuration, how many times do we access the main memory?

Short Answer: 4
Long Answer:

- The first cache access to the location 0x6A is a miss, as the cache is initially empty. As the cache is no write allocate, on this cache miss we just write to the main memory only, so this is our first main memory access, and the cache is still empty.
- Then on the second cache access, we have a read miss - for this one we go to main memory, and actually bring the line in the cache, which occupies the cache line with index 1. 2nd main memory access.
- Third one is again a read miss, so the same happens and the line with the index 0 gets filled out with the memory address this time. Third main memory access.
- This one is actually a cache hit - tag is 0b1 0000 (which was put in the cache in step 2) so no main memory access, read hit.
- This one is a write hit, but because our cache is write through we actually write in the main memory as well, so 4th main memory access.

3.2 Lets say for the same cache size, memory accesses but the only difference is that we have a no write-allocate write-back cache instead. How many memory accesses to the main memory do we have in this case?

Short Answer: 3
Long Answer:

- The first cache access it the same as above, 1st main memory access.
- Again, same as above, 2nd main memory access.
- Same as above, 3rd main memory access.
- Same as above, read hit (no main memory access)
- This one is a write hit, but this time as we have a write back cache, we do not go to main memory - we write the new data on the cache, and turn the dirty bit on the relevant line to 1.

3.3 For one last optimization, we decide to use a write-allocate cache instead. So, now we have a write-allocate, write-back cache. How many times do we access the memory now?

Short Answer: 2
Long Answer:

- This is again a write miss, but because our cache is write-allocate now, we actually bring the data in from the main memory into the cache, and the line

with index 0 gets filled. 1st main memory access.

- This one is the same as the above 2 examples, so cache line with index 1 gets filled and we have a read miss. 2nd main memory access.
- Because the first write actually filled the 0th index line with the relevant data, this 3rd memory access actually becomes a read hit, as there is data on the cache now. No main memory access!
- Same as above, read hit (no main memory access).
- Same as 2.2, as the cache is write-back, this write hit is taken on the cache itself, and the line on the cache gets changed - no main memory access.

3.4 We discovered in our previous subparts that one set of write policies leads to less main memory accesses than the others. Out of the policies covered in class, is this the most optimal set of write policies for a cache? If so, explain why. If they are not, what trade-offs come as a result of these policies?

Our write-allocate, write-back cache seems to have the best performance here. In practice, write-back policies have been used much than write-through, though some architectures still employ specialized write-through caches. However, some trade-offs to a write-back cache include having to deal with overhead especially when dealing with multiple local caches as a result of multiple CPUs (this is different from multi-level cache structures). In a write-back cache, where the caches may hold updated data while main memory doesn't, 'cache coherency' must be maintained between caches so that each write is propagated across the system.

4 AMAT

Recall that AMAT stands for Average Memory Access Time. The main formula for it is:

$$\text{AMAT} = \text{Hit Time} + \text{Miss Rate} * \text{Miss Penalty}$$

In a multi-level cache structure, we can separate miss rates into two types that we consider for each level.

- **Global:** Calculated as the number of accesses that missed at that level divided by the total number of accesses *to the cache system*.
- **Local:** Calculated as the number of accesses that missed at that level divided by the total number of accesses *to that cache level*.

4.1 In a 2-level cache system, after 100 total accesses to the cache system, we find that the L2$ (L2 cache) ended up missing 20 times. What is the global miss rate of L2$?

$\frac{20}{100} = 20\%$

4.2 Given the system from the previous subpart, if L1$ had a local miss rate of 50%, what is the local miss rate of L2$?

$\frac{20}{50\% * 100} = \frac{20}{50} = 40\%$. We know that L2$ is accessed when L1$ misses, so if L1$ misses 50% of the time, that means we access L2$ 50 times, of which we ended up having 20 misses in L2$.

Suppose your system consists of:

1. An L1$ that has a hit time of 2 cycles and has a local miss rate of 20%
2. An L2$ that has a hit time of 15 cycles and has a global miss rate of 5%
3. Main memory where accesses take 100 cycles

4.3 What is the local miss rate of L2$?

The number of accesses to the L2$ is the number of misses in L1$, so we divide the global miss rate of L2$ with the miss rate of L1$.

L2$ Local miss rate = $\frac{\text{Misses In L2\$}}{\text{Accesses in L2\$}} = \frac{\text{Misses in L2\$}}{\text{Total Accesses}} / \frac{\text{Misses in L1\$}}{\text{Total Accesses}} =$

$\frac{\text{Global Miss Rate}}{\text{L1\$ Miss Rate}} = \frac{5\%}{20\%} = 0.25 = 25\%$

4.4 What is the AMAT of the system?

AMAT = 2 + 20% x (15 + 25% x 100) = 10 cycles, as the Miss Penalty of the L1$ is the 'local' AMAT of the L2$.

Using global rates of each level, alternatively, AMAT = 2 + 20% x 15 + 5% x 100 = 10 cycles (using global miss rates)

4.5 Suppose we want to reduce the AMAT of the system to 8 cycles or lower by adding in a L3$. If the L3$ has a local miss rate of 30%, what is the largest hit time that the L3$ can have?

Let H = hit time of the cache. Extending the AMAT equation so that the Miss Penalty of the L2$ is the 'local' AMAT of the L3$, we can write:
$2 + 20\% * (15 + 25\% * (H + 30\% * 100)) \leq 8$
Solving for H, we find that $H \leq 30$. So the largest hit time is 30 cycles.

CS 61C Fall 2022

Parallelism

Discussion 11

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 SIMD is a form of instruction-level parallelism.

1.2 SIMD is ideal for flow-control heavy tasks (i.e. tasks with many branches/if statements).

1.3 Intel's SIMD intrinsic instructions invoke large registers available on the architecture in order to perform one operation on multiple values at once.

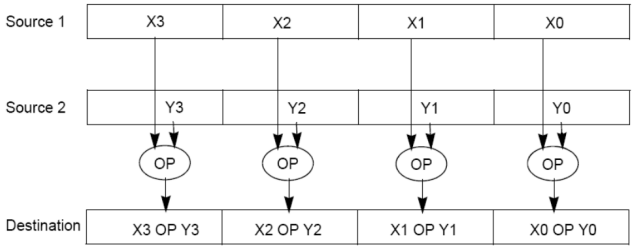
1.4 Each hardware thread in the CPU uses a shared cache.

1.5 The number of hardware threads available can be more than the number of processor cores on the computer.

1.6 In thread-level parallelism, the amount of speedup is directly proportional to the increase in number of hardware threads.

2 Data-Level Parallelism

The idea central to data level parallelism is vectorized calculation: applying operations to multiple items (which are part of a single vector) at the same time.



Some machines with x86 architectures have special, wider registers, that can hold 128, 256, or even 512 bits. Intel intrinsics (Intel proprietary technology) allow us to use these wider registers to harness the power of DLP in C code.

Below is a small selection of the available Intel intrinsic instructions. All of them perform operations using 128-bit registers. The type __m128i is used when these registers hold 4 ints, 8 shorts or 16 chars; __m128d is used for 2 double precision floats, and __m128 is used for 4 single precision floats. Where you see "epiXX", epi stands for **e**xtended **p**acked **i**nteger, and XX is the number of bits in the integer. "epi32" for example indicates that we are treating the 128-bit register as a pack of 4 32-bit integers.

- `__m128i _mm_set1_epi32(int i)`:
  Set the four signed 32-bit integers within the vector to `i`.
- `__m128i _mm_loadu_si128( __m128i *p)`:
  Load the 4 successive ints pointed to by `p` into a 128-bit vector.
- `__m128i _mm_mullo_epi32(__m128i a, __m128i b)`:
  Return vector $(a_0 \cdot b_0, a_1 \cdot b_1, a_2 \cdot b_2, a_3 \cdot b_3)$.
- `__m128i _mm_add_epi32(__m128i a, __m128i b)`:
  Return vector $(a_0 + b_0, a_1 + b_1, a_2 + b_2, a_3 + b_3)$
- `void _mm_storeu_si128( __m128i *p, __m128i a)`:
  Store 128-bit vector `a` at pointer `p`.
- `__m128i _mm_and_si128(__m128i a, __m128i b)`:
  Perform a bitwise AND of 128 bits in a and b, and return the result.
- `__m128i _mm_cmpeq_epi32(__m128i a, __m128i b)`:
  The ith element of the return vector will be set to 0xFFFFFFFF if the ith elements of a and b are equal, otherwise it'll be set to 0.

2.1 SIMD-ize the following function, which returns the product of all of the elements in an array.

```
static int product_naive(int n, int *a) {
    int product = 1;
    for (int i = 0; i < n; i++) {
        product *= a[i];
    }
    return product;
}
```

Things to think about: When iterating through a loop and grabbing elements 4 at a time, how should we update our index for the next iteration? What if our array has a length that isn't a multiple of 4? Can we always SIMD-ize an entire array? What can we do to handle this tail case?

```
static int product_vectorized(int n, int *a) {
    int result[4];
    __m128i prod_v = ________________________________________;
    for (int i = 0; i < _____; i += ___) { // Vectorized loop
        prod_v = ____________________________________________________________________________;
    }
    __mm_storeu_si128(_____________________________, _____________________________);
    for (int i = _____________; i < ____________; i++) { // Handle tail case
        result[0] *= ________________________;
    }
    return ____________________________________________________;
}
```

3 Thread-Level Parallelism

OpenMP provides an easy interface for using multithreading within C programs. Some examples of OpenMP directives:

- The `parallel` directive indicates that each thread should run a copy of the code within the block. If a for loop is put within the block, **every** thread will run every iteration of the for loop.

  ```
  #pragma omp parallel
  {
      ...
  }
  NOTE: The opening curly brace needs to be on a newline or else there
      will be a compile-time error!
  ```

- The `parallel for` directive will split up iterations of a for loop over various threads. Every thread will run **different** iterations of the for loop. The following two code snippets are equivalent.

  ```
  #pragma omp parallel for
  for (int i = 0; i < n; i++) {
      ...
  }
  ```

  ```
  #pragma omp parallel
  {
  #pragma omp for
      for (int i =0; i < n; i++) { ... }
  }
  ```

There are two functions you can call that may be useful to you:

- `int omp_get_thread_num()` will return the number of the thread executing the code
- `int omp_get_num_threads()` will return the number of total hardware threads executing the code

3.1 For each question below, state and justify whether the program is **sometimes incorrect**, **always incorrect**, **slower than serial**, **faster than serial**, or **none of the above**. Assume the default number of threads is greater than 1. Assume no thread will complete before another thread starts executing. Assume `arr` is an `int[]` of length `n`.

(a)
```
// Set element i of arr to i
#pragma omp parallel
{
    for (int i = 0; i < n; i++)
        arr[i] = i;
}
```

(b)
```
// Set arr to be an array of Fibonacci numbers.
arr[0] = 0;
arr[1] = 1;
#pragma omp parallel for
for (int i = 2; i < n; i++)
    arr[i] = arr[i-1] + arr[i - 2];
```

(c)
```
// Set all elements in arr to 0;
int i;
#pragma omp parallel for
for (i = 0; i < n; i++)
    arr[i] = 0;
```

(d)
```
// Set element i of arr to i;
int i;
#pragma omp parallel for
for (i = 0; i < n; i++)
    *arr = i;
    arr++;
```

3.2 What potential issue can arise from this code?

```
1 // Decrements element i of arr. n is a multiple of omp_get_num_threads()
2 #pragma omp parallel
3 {
4     int threadCount = omp_get_num_threads();
5     int myThread = omp_get_thread_num();
6     for (int i = 0; i < n; i++) {
7         if (i % threadCount == myThread) arr[i] -= 1;
8     }
9 }
```

4 Concurrency

The benefits of multi-threading programming come only after you understand concurrency. Here are two of the most common concurrency issues:

1. **Cache-incoherence**: each hardware thread has its own cache, hence data modified in one thread may not be immediately reflected in the other. This can be solved by bypassing the cache and writing directly to memory, i.e. using volatile keywords in many languages, or by using a coherency protocol such as MOESI.
2. **Read-modify-write**: Read-modify-write is a very common pattern in programming. In the context of multi-threading programming, the **interleaving** of R, M, W stages often produces a lot of issues.

4.1 MOESI Protocol

Parallel processing allows individual cores of a CPU to operate as independent units with their own caches. However, for this to be the case, the machine must be able to coordinate the information flow of all cores and all caches so that this information is reliable to some degree. Therefore, we impose **cache states**, composing of the **valid**, **dirty** and **shared** bits, to denote status of the cache data at a specific cache block. These cache states are used when there is a cache **miss** or **write** to a certain core's cache so that if the information is modified in one place, the other caches are informed. In summary, we don't want two caches with different data both saying that they have the most up-to-date data, because that simply can't be true. In other words, from the perspective of the **host processor**, their cache line states may be updated due to actions taken by **proxy processor** execution.

Consider this visual representation of the addressing of a cache block and the updated construction of the block itself:

Address		
Tag	Index	Offset

$\longrightarrow$

Contents			
State			
Valid	Dirty	Shared	Data

Each state describes a specific set of conditions, on a **single cache block**, in respect to the overall memory system(all caches and main memory).

4.1 Match all conditions below with their corresponding state(s).

Note: Some conditions can apply to multiple states!

(a) data in host cache up-to-date

(b) data in main memory is outdated

(c) data in main memory up-to-date

(d) if evicted, host cache must write this line's data back to main memory

(e) no copies exist in other (proxy) caches

(f) copies may exist in other (proxy) caches

(g) access from processor will result in a miss

1. **Modified**(M)
2. **Owned**(O)
3. **Exclusive**(E)
4. **Shared**(S)
5. **Invalid**(I)

4.2 Atomic Instructions

In order to solve the problems created by Read-modify-write, we have to rely on the idea of uninterrupted execution, also known as **atomic** execution.

In RISC-V, we have two categories of atomic instructions:

1. **Amoswap**: allows for uninterrupted memory operations within a single instruction
2. **Load-reserve, store-conditional**: allows us to have uninterrupted execution across multiple instructions

Both of these can be used to achieve atomic primitives. Here we'll focus on the former with this example:

`Test-and-set`

```
Start:  addi         t0 x0 1 # Locked = 1
        amoswap.w.aq t1 t0 (a0)
        bne          t1 x0 Start
# If the lock is not free, retry

        ... # Critical section

        amoswap.w.rl x0 x0 (a0) # Release lock
```

amoswap rd, rs2, (rs1): Atomically, loads the word starting at address **rs1** into **rd** and puts **rs2** into memory at address **rs1**. Data races are avoided using the *aq* and *rl* flags, which *acquire* a lock that forces multiple threads to wait their turn until the lock is *released*.

Test-and-set: We have a lock stored at the address specified by `a0`. We utilize `amoswap` to put in 1 and get the old value. If the old value was a 1, we would not have changed the value of the lock and we will realize that someone currently has the lock. If the old value was a 0, we will have just "locked" the lock and can continue with the critical section. When we are done, we put a 0 back into the lock to "unlock" it.

4.2 We've experimented with data synchronization across threads in C, but now let's take a look at how to parallelize and avoid data races in RISC-V!

We want to parallelize a program that finds the sum of the integers in an array pointed to by `a0` (array length = `a2`) and places it in memory at address `a1`. There is a free word of memory initialized to zero (i.e. result of `calloc(4, 1)`) pointed to by `a3`. For the sake of simplicity, assume there is a function `get_thread_num` that returns the current thread's number and a function `get_num_threads` that returns the total number of threads.

Here is some skeleton code to parallelize this operation. Note the use of `amoswap`. Before filling out the skeleton code, answer questions 4.3 and 4.4 first.

```
    #Prologue
    ...
    mv s0 a0        #s0 points to the array
    mv s1 a1        #s1 points to the global sum
    mv s2 a2        #s2 has the length of array
    jal get_num_threads
    mv s3 a0        #s3 has the total number of threads
    jal get_thread_num
    mv s4 a0        #s4 has the current thread number
    li t0 0
Loop:
    bge ____________
    slli t1 s4 2
    add t1 s0 t1    #index into array
    lw t2 0(t1)
    add t0 t0 t2    #add to local sum
    add ____________
    j Loop
Exit:
    ________________
Try:
    lw t1 0(a3)     #Check if work is being done in another thread
    bnez t1 Try
    amoswap.w.aq ____________
    ________________
    ________________
    add t2 t2 t0    #add local sum to total
    ________________

    amoswap.w.rl ____________
    #Epilogue
    ...
```

4.3 Why do we want to use an atomic instruction in our parallelized implementation?

4.4 Between which lines in the program above should threads start to run in parallel on separate copies of code? (Equivalent to where we put **`#pragma`** `omp parallel` in C)

CS 61C Fall 2022

Parallelism

Discussion 11

1 Pre-Check

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 SIMD is a form of instruction-level parallelism.

False. Instruction-level parallelism deals with performing multiple instructions in parallel, i.e. pipelining. SIMD is a form of data parallelism with a single instruction performing operation on multiple streams of data.

1.2 SIMD is ideal for flow-control heavy tasks (i.e. tasks with many branches/if statements).

False. Data-level parallelism really shines through when we need to repeatedly perform the same operation on a large amount of data. Flow control statements disrupt the continuous flow of computation, which makes programs with them hard to take advantage of SIMD.

1.3 Intel's SIMD intrinsic instructions invoke large registers available on the architecture in order to perform one operation on multiple values at once.

True. For example, we can pack four 32-bit integers in a single 128-bit register and perform the same arithmetic operation on all four integers in one go, using an instruction such as `__m128i _mm_add_epi32(__m128i a, __m128i b)`.

1.4 Each hardware thread in the CPU uses a shared cache.

False, each thread has its own cache, which can lead to cache-incoherence.

1.5 The number of hardware threads available can be more than the number of processor cores on the computer.

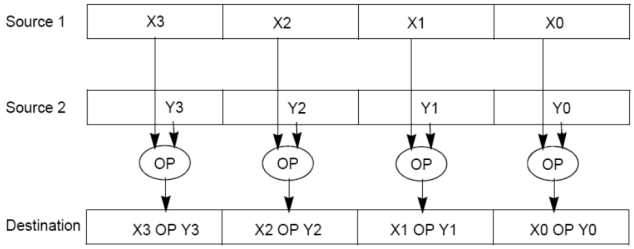
True. Modern computers can offer more hardware threads than processors using hyper-threading.

1.6 In thread-level parallelism, the amount of speedup is directly proportional to the increase in number of hardware threads.

False, usually there is some overhead in paralleling an operation. Amdahl's Law shows that true speedup is affected not only by the number of threads but also by the amount of code that cannot be sped up.

2 Data-Level Parallelism

The idea central to data level parallelism is vectorized calculation: applying operations to multiple items (which are part of a single vector) at the same time.



Some machines with x86 architectures have special, wider registers, that can hold 128, 256, or even 512 bits. Intel intrinsics (Intel proprietary technology) allow us to use these wider registers to harness the power of DLP in C code.

Below is a small selection of the available Intel intrinsic instructions. All of them perform operations using 128-bit registers. The type __m128i is used when these registers hold 4 ints, 8 shorts or 16 chars; __m128d is used for 2 double precision floats, and __m128 is used for 4 single precision floats. Where you see "epiXX", epi stands for **e**xtended **p**acked **i**nteger, and XX is the number of bits in the integer. "epi32" for example indicates that we are treating the 128-bit register as a pack of 4 32-bit integers.

- `__m128i _mm_set1_epi32(int i)`:
  Set the four signed 32-bit integers within the vector to `i`.
- `__m128i _mm_loadu_si128( __m128i *p)`:
  Load the 4 successive ints pointed to by `p` into a 128-bit vector.
- `__m128i _mm_mullo_epi32(__m128i a, __m128i b)`:
  Return vector $(a_0 \cdot b_0, a_1 \cdot b_1, a_2 \cdot b_2, a_3 \cdot b_3)$.
- `__m128i _mm_add_epi32(__m128i a, __m128i b)`:
  Return vector $(a_0 + b_0, a_1 + b_1, a_2 + b_2, a_3 + b_3)$
- `void _mm_storeu_si128( __m128i *p, __m128i a)`:
  Store 128-bit vector `a` at pointer `p`.
- `__m128i _mm_and_si128(__m128i a, __m128i b)`:
  Perform a bitwise AND of 128 bits in a and b, and return the result.
- `__m128i _mm_cmpeq_epi32(__m128i a, __m128i b)`:
  The ith element of the return vector will be set to 0xFFFFFFFF if the ith elements of a and b are equal, otherwise it'll be set to 0.

2.1 SIMD-ize the following function, which returns the product of all of the elements in an array.

```
static int product_naive(int n, int *a) {
    int product = 1;
    for (int i = 0; i < n; i++) {
        product *= a[i];
    }
    return product;
}
```

Things to think about: When iterating through a loop and grabbing elements 4 at a time, how should we update our index for the next iteration? What if our array has a length that isn't a multiple of 4? Can we always SIMD-ize an entire array? What can we do to handle this tail case?

```
static int product_vectorized(int n, int *a) {
    int result[4];
    __m128i prod_v = __mm_set1_epi32(1);
    for (int i = 0; i < n/4 * 4; i += 4) { // Vectorized loop
        prod_v = __mm_mullo_epi32(prod_v, __mm_loadu_si128((__m128i *) (a + i)));
    }
    _mm_storeu_si128((__m128i *) result, prod_v);
    for (int i = n/4 * 4; i < n; i++) { // Handle tail case
        result[0] *= a[i];
    }
    return result[0] * result[1] * result[2] * result[3];
}
```

3 Thread-Level Parallelism

OpenMP provides an easy interface for using multithreading within C programs. Some examples of OpenMP directives:

- The `parallel` directive indicates that each thread should run a copy of the code within the block. If a for loop is put within the block, **every** thread will run every iteration of the for loop.

  ```
  #pragma omp parallel
  {
      ...
  }
  NOTE: The opening curly brace needs to be on a newline or else there
      will be a compile-time error!
  ```

- The `parallel for` directive will split up iterations of a for loop over various threads. Every thread will run **different** iterations of the for loop. The following two code snippets are equivalent.

  ```
  #pragma omp parallel for
  for (int i = 0; i < n; i++) {
      ...
  }
  ```

  ```
  #pragma omp parallel
  {
  #pragma omp for
      for (int i =0; i < n; i++) { ... }
  }
  ```

There are two functions you can call that may be useful to you:

- `int omp_get_thread_num()` will return the number of the thread executing the code
- `int omp_get_num_threads()` will return the number of total hardware threads executing the code

3.1 For each question below, state and justify whether the program is **sometimes incorrect**, **always incorrect**, **slower than serial**, **faster than serial**, or **none of the above**. Assume the default number of threads is greater than 1. Assume no thread will complete before another thread starts executing. Assume `arr` is an `int[]` of length `n`.

(a)
```
// Set element i of arr to i
#pragma omp parallel
{
    for (int i = 0; i < n; i++)
        arr[i] = i;
}
```

Slower than serial: There is no **for** directive, so every thread executes this loop in its entirety. `n` threads running `n` loops at the same time will actually execute in the same time as 1 thread running 1 loop. Despite the possibility of false sharing, the values should all be correct at the end of the loop. Furthermore, the existence of parallel overhead due to the extra number of threads will slow down the execution time.

(b)
```
// Set arr to be an array of Fibonacci numbers.
arr[0] = 0;
arr[1] = 1;
#pragma omp parallel for
for (int i = 2; i < n; i++)
    arr[i] = arr[i-1] + arr[i - 2];
```

Always incorrect (when $n > 4$): Loop has data dependencies, so the calculation of all threads but the first one will depend on data from the previous thread. Because we said "assume no thread will complete before another thread starts executing," this code will always read incorrect values.

(c)
```
// Set all elements in arr to 0;
int i;
#pragma omp parallel for
for (i = 0; i < n; i++)
    arr[i] = 0;
```

Faster than serial: The **for** directive actually automatically makes loop variables (such as the index) private, so this will work properly. The **for** directive splits up the iterations of the loop into continuous chunks for each thread, so there will be no data dependencies or false sharing.

(d)
```
// Set element i of arr to i;
int i;
#pragma omp parallel for
for (i = 0; i < n; i++)
    *arr = i;
    arr++;
```

Sometimes incorrect: Because we are not indexing into the array, there is a data race to increment the array pointer. If multiple threads are executed such that they all execute the first line, *arr = i; before the second line, arr++;, they will clobber each other's outputs by overwriting what the other threads wrote in the same position.

3.2 What potential issue can arise from this code?

```
// Decrements element i of arr. n is a multiple of omp_get_num_threads()
#pragma omp parallel
{
    int threadCount = omp_get_num_threads();
    int myThread = omp_get_thread_num();
    for (int i = 0; i < n; i++) {
        if (i % threadCount == myThread) arr[i] -= 1;
    }
}
```

False sharing arises because different threads can modify elements located in the same memory block simultaneously. This is a problem because some threads may have incorrect values in their cache block when they modify the value `arr[i]`, invalidating the cache block. This will slow down the execution of this program due to frequent coherency misses.

4 Concurrency

The benefits of multi-threading programming come only after you understand concurrency. Here are two of the most common concurrency issues:

1. **Cache-incoherence**: each hardware thread has its own cache, hence data modified in one thread may not be immediately reflected in the other. This can be solved by bypassing the cache and writing directly to memory, i.e. using volatile keywords in many languages, or by using a coherency protocol such as MOESI.
2. **Read-modify-write**: Read-modify-write is a very common pattern in programming. In the context of multi-threading programming, the **interleaving** of R, M, W stages often produces a lot of issues.

4.1 MOESI Protocol

Parallel processing allows individual cores of a CPU to operate as independent units with their own caches. However, for this to be the case, the machine must be able to coordinate the information flow of all cores and all caches so that this information is reliable to some degree. Therefore, we impose **cache states**, composing of the **valid**, **dirty** and **shared** bits, to denote status of the cache data at a specific cache block. These cache states are used when there is a cache **miss** or **write** to a certain core's cache so that if the information is modified in one place, the other caches are informed. In summary, we don't want two caches with different data both saying that they have the most up-to-date data, because that simply can't be true. In other words, from the perspective of the **host processor**, their cache line states may be

updated due to actions taken by **proxy processor** execution.

Consider this visual representation of the addressing of a cache block and the updated construction of the block itself:

Address		
Tag	Index	Offset

$\longrightarrow$

Contents			
State			
Valid	Dirty	Shared	Data

Each state describes a specific set of conditions, on a **single cache block**, in respect to the overall memory system(all caches and main memory).

4.1 Match all conditions below with their corresponding state(s).

Note: Some conditions can apply to multiple states!

(a) data in host cache up-to-date

(b) data in main memory is outdated

(c) data in main memory up-to-date

(d) if evicted, host cache must write this line's data back to main memory

(e) no copies exist in other (proxy) caches

(f) copies may exist in other (proxy) caches

(g) access from processor will result in a miss

1. **Modified**(M)

   a, b, d, e

2. **Owned**(O)

   a, b, d, f

3. **Exclusive**(E)

   a, c, e

4. **Shared**(S)

   a, f

   In the shared state, we don't know if main memory is up-to-date or not - both are possible.

5. **Invalid**(I)

   f, g

4.2 Atomic Instructions

In order to solve the problems created by Read-modify-write, we have to rely on the idea of uninterrupted execution, also known as **atomic** execution.

In RISC-V, we have two categories of atomic instructions:

1. **Amoswap**: allows for uninterrupted memory operations within a single instruction
2. **Load-reserve, store-conditional**: allows us to have uninterrupted execution across multiple instructions

Both of these can be used to achieve atomic primitives. Here we'll focus on the former with this example:

`Test-and-set`

```
Start:  addi         t0 x0 1 # Locked = 1
        amoswap.w.aq t1 t0 (a0)
        bne          t1 x0 Start
# If the lock is not free, retry

        ... # Critical section

        amoswap.w.rl x0 x0 (a0) # Release lock
```

amoswap rd, rs2, (rs1): Atomically, loads the word starting at address **rs1** into **rd** and puts **rs2** into memory at address **rs1**. Data races are avoided using the *aq* and *rl* flags, which *acquire* a lock that forces multiple threads to wait their turn until the lock is *released*.

Test-and-set: We have a lock stored at the address specified by `a0`. We utilize `amoswap` to put in 1 and get the old value. If the old value was a 1, we would not have changed the value of the lock and we will realize that someone currently has the lock. If the old value was a 0, we will have just "locked" the lock and can continue with the critical section. When we are done, we put a 0 back into the lock to "unlock" it.

4.2 We've experimented with data synchronization across threads in C, but now let's take a look at how to parallelize and avoid data races in RISC-V!

We want to parallelize a program that finds the sum of the integers in an array pointed to by `a0` (array length = `a2`) and places it in memory at address `a1`. There is a free word of memory initialized to zero (i.e. result of `calloc(4, 1)`) pointed to by `a3`. For the sake of simplicity, assume there is a function `get_thread_num` that returns the current thread's number and a function `get_num_threads` that returns the total number of threads.

Here is some skeleton code to parallelize this operation. Note the use of `amoswap`. Before filling out the skeleton code, answer questions 4.3 and 4.4 first.

```
1       #Prologue
2       ...
3       mv s0 a0        #s0 points to the array
4       mv s1 a1        #s1 points to the memory address
5       mv s2 a2        #s2 has the length of array
6       jal get_num_threads
7       mv s3 a0        #s3 has the total number of threads
8       jal get_thread_num
9       mv s4 a0        #s4 has the current thread number
10      li t0 0
11  Loop:
12      bge s4 s2 Exit
13      slli t1 s4 2
14      add t1 s0 t1    #index into array
15      lw t2 0(t1)
16      add t0 t0 t2    #add to local sum
17      add s4 s4 s3    #process indices which are equal to s4, modulo s3
18      j Loop
19  Exit:
20      li t0, 1        #try to swap a nonzero value into the lock
21  Try:
22      lw t1 0(a3)     #check if lock is held by other thread
23      bnez t1 Try
24      amoswap.w.aq t1 t0 (a3)
25      bnez t1 Try     #must try again if we fail to acquire lock
26      lw t2 0(s1)
27      add t2 t2 t0
28      sw t2 0(s1)     #add to the global sum in critical section to avoid data races
29
30      amoswap.w.rl x0 x0 (a3) # release lock
31      #Epilogue
32      ...
```

4.3 Why do we want to use an atomic instruction in our parallelized implementation?

Without using some sort of atomic instruction, we encounter a data race when multiple threads could write to the global sum at `s1`. This results in non-deterministic behavior in `s1`.

4.4 Between which lines in the program above should threads start to run in parallel on separate copies of code? (Equivalent to where we put **`#pragma`** `omp parallel` in C)

Between lines 5 and 6, after we store all arguments but before we find the total number of threads we are running. This is because we want to store the arguments only once for efficiency, but we don't know the number of threads until we spawn them.

CS 61C Fall 2022

OS, Virtual Memory, & I/O

Discussion 13

(In Fall 2022, Discussion 12 was canceled and merged with Discussion 13.)

1 Precheck

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 Responsibilities of the OS include loading programs, handling services, combining programs together for efficiency.

1.2 An operating system uses context switches to allow for multiple processes to run simultaneously across multiple CPUs.

1.3 Having virtual memory helps protect a system.

1.4 The virtual address space is limited by the amount of memory in the system.

1.5 The virtual and physical page number must be the same size.

1.6 If a page table entry can not be found in the TLB, then a page fault has occurred.

1.7 Polling and interrupts are only relevant concepts for low level programming.

1.8 Memory-mapped IO only works with polling.

2 Addressing

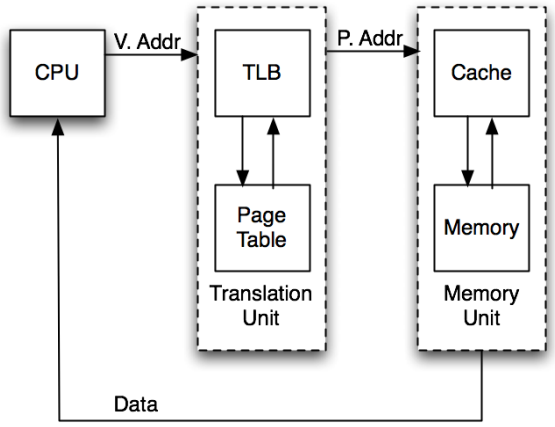
Virtual Address (VA) What your program uses

Virtual Page Number (VPN)	Page Offset

Physical Address (PA) What actually determines where in memory to go

Physical Page Number (PPN)	Page Offset

For example, with 4 KiB pages and byte addresses, there are 12 page offset bits since 4 KiB = 2^{12} B = 4096 B.



Pages

A chunk of memory or disk with a set size. Addresses in the same virtual page map to addresses in the same physical page. The page table determines the mapping.

Valid	Dirty	Permission Bits	PPN
— *Page entry (VPN: 0)* —			
— *Page entry (VPN: 1)* —			

Each stored row of the page table is called a **page table entry**. There are $2^{\text{VPN bits}}$ such entries in a page table. Say you have a VPN of 5 and you want to use the page table to find what physical page it maps to; you'll check the 5th (0-indexed) page table entry. If the valid bit is 1, then that means that the entry is valid (in other words, the physical page corresponding to that virtual page is in main memory as opposed to being only on disk) and therefore you can get the PPN from the entry and access that physical page in main memory. The page table is stored in memory: the OS sets a register (the Page Table Base Register) telling the hardware the address of the first entry of the page table. If you write to a page in memory, the processor updates the "dirty" bit in the page table entry corresponding to that page, which lets the OS know that updating that page on disk is necessary (remember: main memory contains a subset of what's on disk). This is a similar concept as having a dirty bit for each cache block in a write-back cache, which we covered in lecture and in Lab 9. Each process gets its own illusion of full memory to work with, and therefore its own page table.

Protection Fault The page table entry for a virtual page has permission bits that prohibit the requested operation. This is how a segmentation fault occurs.

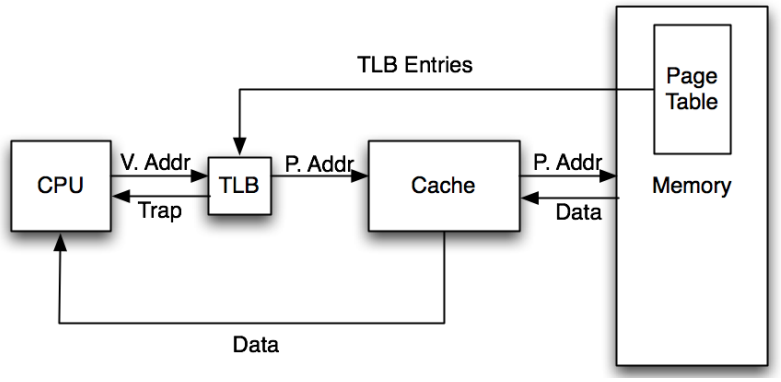
Page Fault The page table entry for a virtual page has its valid bit set to false.

This means that the entry is not in memory. For simplicity, we will assume the address causing the page fault is a valid request, and maps to a page that was swapped from memory to disk. Since the requested address is valid, the operating system checks if the page exists on disk. If so, we transfer the page to memory (evicting another page if necessary), and add the mapping to the page table *and the TLB*.

Translation Lookaside Buffer

A cache for the page table. Each block is a single page table entry. If an entry is not in the TLB, it's a TLB miss. Assuming fully associative:

TLB Valid	Tag (VPN)	Page Table Entry		
		Page Dirty	Permission Bits	PPN
— *TLB entry* —				
— *TLB entry* —				



To access some memory location, we get the virtual page number (VPN) from the virtual address (VA) and first try to translate the VPN to a physical page number (PPN) using the translation lookaside buffer (TLB). If the TLB doesn't contain the desired VPN, we check if the page table contains it (remember: the TLB is a subset of the page table!). If the page table doesn't contain an entry for the VPN, then this is a page fault; memory doesn't contain the corresponding physical page! This means we need to fetch the physical page from disk and put it into memory, update the page table entry, and load the entry into the TLB, Then, we use the physical page and the offset of the physical address in the page to access memory as the program intended.

2.1 What are three specific benefits of using virtual memory?

2.2 What should happen to the TLB when a new value is loaded into the page table address register?

3 VM Access Patterns

3.1 A processor has 16-bit addresses, 256 byte pages, and an 8-entry fully associative TLB with LRU replacement (the LRU field is 3 bits and encodes the order in which pages were accessed, 0 being the most recent). At some time instant, the TLB for the current process is the initial state given in the table below. Assume that all current page table entries are in the initial TLB. Assume also that all pages can be read from and written to. Fill in the final state of the TLB according to the access pattern below.

Free Physical Pages `0x17`, `0x18`, `0x19`

Access Pattern

1. `0x11f0` (**Read**)
2. `0x1301` (**Write**)
3. `0x20ae` (**Write**)
4. `0x2332` (**Write**)
5. `0x20ff` (**Read**)
6. `0x3415` (**Write**)

Initial TLB

VPN	PPN	Valid	Dirty	LRU
`0x01`	`0x11`	1	1	0
`0x00`	`0x00`	0	0	7
`0x10`	`0x13`	1	1	1
`0x20`	`0x12`	1	0	5
`0x00`	`0x00`	0	0	7
`0x11`	`0x14`	1	0	4
`0xac`	`0x15`	1	1	2
`0xff`	`0xff`	1	0	3

Final TLB

VPN	PPN	Valid	Dirty	LRU

4 Polling & Interrupts

4.1 Fill out this table that compares polling and interrupts.

Operation	**Definition**	**Pro/Good for**	**Con**
Polling			
Interrupts			

5 Memory Mapped I/O

5.1 For this question, the following addresses correspond to registers in some I/O devices and not regular user memory.

- `0xFFFF0000`—Receiver Control: LSB is the ready bit, there may be other bits set that we don't need right now.
- `0xFFFF0004`—Receiver Data: Received data stored at lowest byte.
- `0xFFFF0008`—Transmitter Control: LSB is the ready bit, there may be other bit set that we don't need right now.
- `0xFFFF000C`—Transmitter Data: Transmitted data stored at lowest byte.

Recall that receiver will only have data for us when the corresponding ready bit is 1, and that we can only write data to the transmitter when its ready bit is 1. Write RISC-V code that reads byte from the receiver and writes that byte to the transmitter (busy-waiting if necessary).

CS 61C Fall 2022 — OS, Virtual Memory, & I/O — Discussion 13

(In Fall 2022, Discussion 12 was canceled and merged with Discussion 13.)

1 Precheck

This section is designed as a conceptual check for you to determine if you conceptually understand and have any misconceptions about this topic. Please answer true/false to the following questions, and include an explanation:

1.1 Responsibilities of the OS include loading programs, handling services, combining programs together for efficiency.

False. While the OS is responsible for loading programs, handling services (such as the network stack and the file system), it is actually responsible for isolating programs from each other so that a given program doesn't interfere with another program's memory or execution, such as by accessing the same memory address.

1.2 An operating system uses context switches to allow for multiple processes to run simultaneously across multiple CPUs.

False. Context switches are used to switch between tasks on *one* CPU, so that another task can continue while one task is waiting on something else. This illusion of simultaneous execution happening in multiprogramming is different from the true parallel execution done on multiple CPUs, which is multiprocessing.

1.3 Having virtual memory helps protect a system.

True. By dedicating specific pages to a program, the OS can ensure that a program does not access pages it's not been given access to, providing isolation between programs.

1.4 The virtual address space is limited by the amount of memory in the system.

False. The physical address space is limited by the amount of physical memory in the system, the size of the virtual address space is set by the OS.

1.5 The virtual and physical page number must be the same size.

False. There could be fewer physical pages than virtual pages. However, the page size does need to be the same.

1.6 If a page table entry can not be found in the TLB, then a page fault has occurred.

False, the TLB acts as a cache for the page table, so an item can be valid in page table but not stored in TLB. A page fault occurs either when a page cannot be found in the page table or it has an invalid bit.

1.7 Polling and interrupts are only relevant concepts for low level programming.

False. Similar concepts apply to almost all types of applications, including web apps, mobile apps, distributed systems, and so on.

1.8 Memory-mapped IO only works with polling.

False. The implementation backing the memory mapping can use interrupt-driven IO (for example, reading files).

2 Addressing

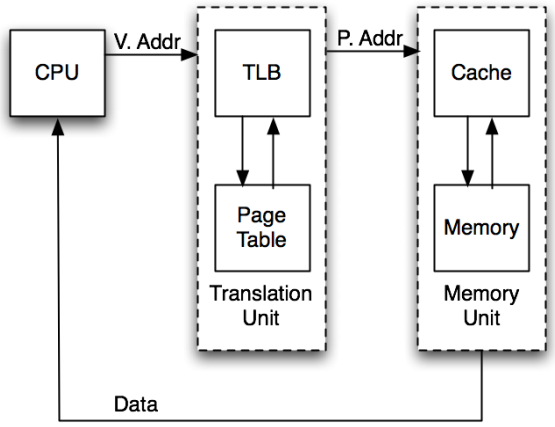
Virtual Address (VA) What your program uses

Virtual Page Number (VPN)	Page Offset

Physical Address (PA) What actually determines where in memory to go

Physical Page Number (PPN)	Page Offset

For example, with 4 KiB pages and byte addresses, there are 12 page offset bits since 4 KiB = 2^{12} B = 4096 B.



Pages

A chunk of memory or disk with a set size. Addresses in the same virtual page map to addresses in the same physical page. The page table determines the mapping.

Valid	Dirty	Permission Bits	PPN
— *Page entry (VPN: 0)* —			
— *Page entry (VPN: 1)* —			

Each stored row of the page table is called a **page table entry**. There are $2^{\text{VPN bits}}$ such entries in a page table. Say you have a VPN of 5 and you want to use the page table to find what physical page it maps to; you'll check the 5th (0-indexed) page table entry. If the valid bit is 1, then that means that the entry is valid (in other words, the physical page corresponding to that virtual page is in main memory as opposed to being only on disk) and therefore you can get the PPN from the entry and access that physical page in main memory. The page table is stored in memory: the OS sets a register (the Page Table Base Register) telling the hardware the address of the first entry of the page table. If you write to a page in memory, the processor updates the "dirty" bit in the page table entry corresponding to that page, which lets the OS know that updating that page on disk is necessary (remember: main memory contains a subset of what's on disk). This is a similar concept as having a dirty bit for each cache block in a write-back cache, which we covered in lecture and in Lab 9. Each process gets its own illusion of full memory to work with, and therefore its own page table.

Protection Fault The page table entry for a virtual page has permission bits that prohibit the requested operation. This is how a segmentation fault occurs.

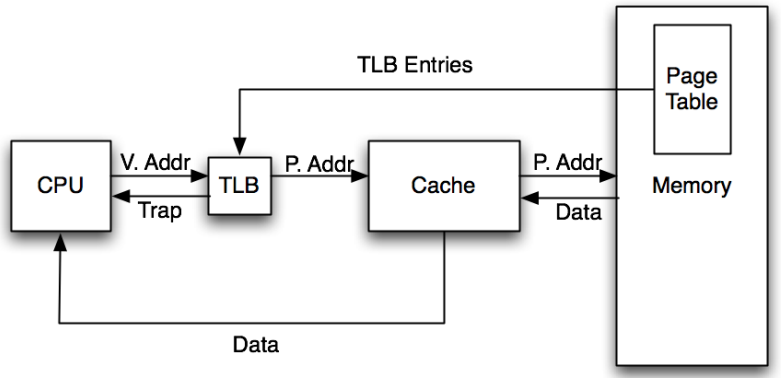
Page Fault The page table entry for a virtual page has its valid bit set to false.

This means that the entry is not in memory. For simplicity, we will assume the address causing the page fault is a valid request, and maps to a page that was swapped from memory to disk. Since the requested address is valid, the operating system checks if the page exists on disk. If so, we transfer the page to memory (evicting another page if necessary), and add the mapping to the page table *and the TLB*.

Translation Lookaside Buffer

A cache for the page table. Each block is a single page table entry. If an entry is not in the TLB, it's a TLB miss. Assuming fully associative:

TLB Valid	Tag (VPN)	Page Table Entry		
		Page Dirty	Permission Bits	PPN
— TLB entry —				
— TLB entry —				



To access some memory location, we get the virtual page number (VPN) from the virtual address (VA) and first try to translate the VPN to a physical page number (PPN) using the translation lookaside buffer (TLB). If the TLB doesn't contain the desired VPN, we check if the page table contains it (remember: the TLB is a subset of the page table!). If the page table doesn't contain an entry for the VPN, then this is a page fault; memory doesn't contain the corresponding physical page! This means we need to fetch the physical page from disk and put it into memory, update the page table entry, and load the entry into the TLB, Then, we use the physical page and the offset of the physical address in the page to access memory as the program intended.

2.1 What are three specific benefits of using virtual memory?

- Illusion of infinite memory (bridges memory and disk in memory hierarchy).
- Simulates full address space for each process so that the linker/loader don't need to know about other programs.
- Enforces protection between processes and even within a process (e.g. read-only pages set up by the OS).

2.2 What should happen to the TLB when a new value is loaded into the page table address register?

The valid bits of the TLB should all be set to 0. The page table entries in the TLB corresponded to the old process/page table, so none of them are valid once the page table address register points to a different page table

3 VM Access Patterns

3.1 A processor has 16-bit addresses, 256 byte pages, and an 8-entry fully associative TLB with LRU replacement (the LRU field is 3 bits and encodes the order in which pages were accessed, 0 being the most recent). At some time instant, the TLB for the current process is the initial state given in the table below. Assume that all current page table entries are in the initial TLB. Assume also that all pages can be read from and written to. Fill in the final state of the TLB according to the access pattern below.

Free Physical Pages `0x17`, `0x18`, `0x19`

Access Pattern

1. `0x11f0` (**Read**)
2. `0x1301` (**Write**)
3. `0x20ae` (**Write**)
4. `0x2332` (**Write**)
5. `0x20ff` (**Read**)
6. `0x3415` (**Write**)

Initial TLB

VPN	PPN	Valid	Dirty	LRU
`0x01`	`0x11`	1	1	0
`0x00`	`0x00`	0	0	7
`0x10`	`0x13`	1	1	1
`0x20`	`0x12`	1	0	5
`0x00`	`0x00`	0	0	7
`0x11`	`0x14`	1	0	4
`0xac`	`0x15`	1	1	2
`0xff`	`0xff`	1	0	3

Final TLB

VPN	PPN	Valid	Dirty	LRU
`0x01`	`0x11`	1	1	5
`0x13`	`0x17`	1	1	3
`0x10`	`0x13`	1	1	6
`0x20`	`0x12`	1	1	1
`0x23`	`0x18`	1	1	2
`0x11`	`0x14`	1	0	4
`0xac`	`0x15`	1	1	7
`0x34`	`0x19`	1	1	0

1. `0x11f0` (**Read**): hit, LRUs: 1, 7, 2, 5, 7, 0, 3, 4
2. `0x1301` (**Write**): miss, map VPN `0x13` to PPN `0x17`, valid and dirty, LRUs: 2, 0, 3, 6, 7, 1, 4, 5
3. `0x20ae` (**Write**): hit, dirty, LRUs: 3, 1, 4, 0, 7, 2, 5, 6
4. `0x2332` (**Write**): miss, map VPN `0x23` to PPN `0x18`, valid and dirty, LRUs: 4, 2, 5, 1, 0, 3, 6, 7
5. `0x20ff` (**Read**): hit, LRUs: 4, 2, 5, 0, 1, 3, 6, 7
6. `0x3415` (**Write**): miss and replace last entry, map VPN `0x34` to `0x19`, dirty, LRUs, 5, 3, 6, 1, 2, 4, 7, 0

4 Polling & Interrupts

4.1 Fill out this table that compares polling and interrupts.

Operation	Definition	Pro/Good for	Con
Polling	Forces the hardware to wait on ready bit (alternatively, if timing of device is known, the ready bit can be polled at the frequency of the device).	• Low Latency • Low overhead when data is available • Good For: devices that are always busy or when you can't make progress until the device replies	• Can't do anything else while polling • Can't sleep while polling (CPU always at full speed)
Interrupts	Hardware fires an "exception" when it becomes ready. CPU changes PC register to execute code in the interrupt handler when this occurs.	• Can do useful work while waiting for response • Can wait on many things at once • Good for: Devices that take a long time to respond, especially if you can do other work while waiting.	• Nondeterministic when interrupt occurs • interrupt handler has some overhead (e.g. saves all registers, flush pipeline, etc.) • Higher latency per event • Worse throughput

5 Memory Mapped I/O

5.1 For this question, the following addresses correspond to registers in some I/O devices and not regular user memory.

- `0xFFFF0000`—Receiver Control: LSB is the ready bit, there may be other bits set that we don't need right now.
- `0xFFFF0004`—Receiver Data: Received data stored at lowest byte.
- `0xFFFF0008`—Transmitter Control: LSB is the ready bit, there may be other

bit set that we don't need right now.

- `0xFFFF000C`—Transmitter Data: Transmitted data stored at lowest byte.

Recall that receiver will only have data for us when the corresponding ready bit is 1, and that we can only write data to the transmitter when its ready bit is 1. Write RISC-V code that reads byte from the receiver and writes that byte to the transmitter (busy-waiting if necessary).

```
                lui t0 0xffff0
receive_wait:   lw t1 0(t0)
                andi t1 t1 1            # poll on ready of receiver
                beq t1 x0 receive_wait
                lb t2 4(t0)             # load data
transmit_wait:  lw t1 8(t0)             # poll on ready of transmitter
                andi t1 t1 1
                beq t1 x0 transmit_wait # write to transmitter
                sb t2 12(t0)
```